Notebook 01: **Preliminary Work Recap and Baseline Lock**

Purpose of This Notebook

This notebook is the formal entry point of the **final pipeline**, but it is **not** the place where the final project starts making new claims. Its job is more disciplined than that.

The purpose of this notebook is to **recreate, in one compact and coherent notebook, the full preliminary / midterm evidence chain** that was previously distributed across the earlier notebooks on data loading, single-frame signal analysis, short-window motion analysis, single-frame semantic analysis, and short-window semantic consistency.

In other words, this notebook is the **baseline lock**.

Before moving into broader multi-window analysis, proxy comparison, failure-case analysis, video generation, or downstream usefulness tests, the final pipeline needs one trusted notebook that says:

this is the exact preliminary story we are carrying forward, this is how it was obtained, this is what actually worked, and these are the limits that remain non-negotiable.

This notebook therefore plays a very specific role: it does **not** replace the archived midterm notebooks, it does **not** mutate the historical preliminary work, and it does **not** pretend that the final-stage extensions have already happened.

Instead, it takes the valid preliminary staircase and rewrites it into one clean starting document for the final project.

Why This Notebook Exists in the Final Pipeline

The midterm work already established that the project idea is not empty.

The preliminary phase showed five important things:

1. The SemanticKITTI loading and label pipeline works correctly.
2. A simple range-aware pseudo-reflectivity proxy can be computed cleanly:
$$\hat{\rho}_i = I_i \cdot R_i$$
3. That proxy behaves differently from raw intensity on a single frame.
4. The proxy remains temporally stable over short motion windows.
5. The proxy can produce modest but meaningful semantic gains, but those gains are **scene-dependent rather than uniform**.

That means the project has already crossed the feasibility threshold.

So the final pipeline should not begin in chaos. It should begin from a **reproducible, honest, and unified recap notebook** that preserves the preliminary logic before the project expands into stronger evaluation.

This notebook exists so that every later notebook in `final_pipeline/` has a stable answer to the question:

What exactly is the baseline we are extending?

What This Notebook Is Reproducing

This notebook reproduces, in condensed form, the core logic of the preliminary stage:

A. Data-loading and structural sanity

We verify the dataset path, sequence structure, frame availability, label availability, and point-label alignment using SemanticKITTI sequence 00.

B. Single-frame signal behavior

We reload a reference LiDAR frame and reconstruct the raw channels:

- coordinates (x, y, z).
- raw intensity I.
- range R.

We then compute the same lightweight range-aware proxy used throughout the preliminary work:

$$\hat{\rho} = I \cdot R$$

C. Single-frame reflectivity-style interpretation

We recap the original question: if raw intensity is entangled with range and measurement geometry, does a simple range-aware transformation produce a more interpretable signal?

This notebook will reproduce the baseline statistical and visual comparison between:

- raw intensity.
- pseudo-reflectivity $(I \cdot R)$.

D. Short-window motion behavior

We then revisit the short temporal story on a small consecutive frame window, reproducing the core observation that the proxy remains numerically stable and visually structured across motion, while still remaining range-dependent.

E. Single-frame semantic behavior

We reproduce the baseline class-aware analysis showing that pseudo-reflectivity can reshape class-wise signal structure and produce a modest improvement in semantic separability relative to raw intensity.

F. Short-window semantic consistency

Finally, we reproduce the preliminary temporal-semantic result showing that the class-wise structure seen on one frame does not instantly collapse under motion, and that pseudo-reflectivity preserves short-horizon class ordering with a mild average advantage.

Scientific Motivation

LiDAR intensity is useful, but it is not a clean intrinsic surface descriptor.

For a point

$$p_i = (x_i, y_i, z_i, I_i)$$

the Euclidean range is

$$R_i = \sqrt{x_i^2 + y_i^2 + z_i^2}$$

A simplified LiDAR return interpretation is:

$$I_i \propto \frac{\rho_i \cos(\alpha_i)}{R_i^2}$$

where:

- ρ_i is surface reflectivity,
- α_i is incidence angle,
- R_i is range.

This means raw intensity mixes together:

- surface behavior,
- distance effects,
- measurement geometry,
- and sensor-specific effects.

So the project does **not** treat raw intensity as reflectivity.

Instead, the preliminary work introduced a practical proxy:

$$\hat{\rho}_i = I_i \cdot R_i$$

This is **not calibrated physical reflectivity**. It is a **heuristic range-aware feature** whose value lies in semantic interpretability and lightweight signal augmentation, not in exact reflectance recovery.

That honesty clause remains fully active in this notebook and throughout the final project.

What Earlier Preliminary Results Already Suggested

The preliminary stage did **not** prove a grand segmentation breakthrough. But it did produce a defensible baseline.

The most important baseline findings to be reproduced here are:

- On the single semantic frame used in the preliminary study, multiclass separability improved from **0.4482** for raw intensity to **0.5055** for pseudo-reflectivity.
- Across a short 10-frame semantic consistency window, class ordering remained highly stable for both signals, with mean Spearman rank consistency of **0.9534** for raw intensity and **0.9582** for pseudo-reflectivity.
- Across short motion windows, the proxy remained visually structured and numerically stable, but its usefulness was **not uniform** across all frames or scene regimes.
- The proxy is doing something meaningful, but it remains **range-dependent**, **scene-dependent**, and **strictly heuristic**.

Those findings are exactly strong enough to justify a broader final-stage study, but not strong enough to justify inflated claims.

That is why this notebook matters.

What This Notebook Will Actually Do

This recap notebook will stay compact, but it will still reproduce the complete preliminary logic in one readable arc.

Step A: Verify the dataset foundation

- confirm the expected SemanticKITTI sequence structure
- confirm frame and label availability
- confirm that the pipeline starts from a trustworthy base

Step B: Rebuild the reference frame

- load one reference LiDAR frame
- load its matching labels
- separate coordinates, intensity, and label information
- compute range and pseudo-reflectivity

Step C: Reproduce the baseline signal comparison

- compare raw intensity and $I \cdot R$ statistically.
- inspect distributions.
- inspect range dependence.
- visualize BEV / FOV style representations where appropriate.

Step D: Reproduce the short motion recap

- analyze a small contiguous motion window.
- rebuild frame-wise summaries.
- confirm that the proxy remains stable across motion.
- retain the conclusion that temporal structure exists without claiming invariance.

Step E: Reproduce the semantic recap

- attach semantic labels to the reference frame.
- compute class-wise summary behavior.
- reproduce the class-separability comparison.
- visualize how pseudo-reflectivity changes class structure relative to raw intensity.

Step F: Reproduce the short temporal-semantic recap

- keep only sufficiently supported classes across a small frame window.
- compute compact class-wise temporal summaries.
- evaluate ordering consistency across motion.
- confirm whether the semantic story survives beyond a single frame.

Step G: End with one baseline summary table

This notebook should end with a small set of trusted recap outputs:

- one compact figure board.
- one baseline metrics table.
- one written summary of what the preliminary stage established.
- one explicit statement of what is **still not solved**.

What This Notebook Is Not Allowed to Do

This is important.

This notebook is **not** the place to:

- introduce new proxy variants,
- compare many normalization schemes,
- claim sequence-wide generalization,
- claim material identification,
- claim calibrated reflectivity recovery,
- claim universal superiority of pseudo-reflectivity,
- claim segmentation performance improvements,
- or consume the full final-project agenda inside one overcooked recap file.

That work belongs later in the final pipeline.

This notebook must remain a **baseline recap notebook**, not a premature final-results notebook wearing a fake moustache.

Expected Outputs

By the end of this notebook, we should have:

1. a confirmed working baseline data pipeline,
2. a re-established reference frame with raw intensity, range, and pseudo-reflectivity,
3. a compact reproduction of the preliminary single-frame signal analysis,
4. a compact reproduction of the short-window motion analysis,
5. a compact reproduction of the preliminary semantic analysis,
6. a compact reproduction of the short-window semantic consistency analysis,
7. one baseline metrics table,
8. one recap figure board,
9. and one honest concluding baseline statement to anchor the rest of the final project.

Why This Notebook Matters for the Rest of the Final Work

Every later notebook in the final pipeline depends on this one.

If this recap notebook is clean, then the later final-stage notebooks can ask harder questions such as:

- does pseudo-reflectivity help across multiple windows rather than one local segment?
- which proxy variant performs best on average?
- which classes benefit consistently?
- which classes lose?
- where does the proxy fail?
- can the best proxy help in a lightweight downstream task?
- can videos and final artifacts display those findings faithfully?

But none of that should start from vague memory.

It should start from a locked baseline.

This notebook is that lock.

Core Honesty Statement

This project studies a **reflectivity-aware augmentation idea**, not solved reflectivity recovery.

The proxy

$$\hat{\rho} = I \cdot R$$

is:

- lightweight,
- practical,
- easy to compute,
- and semantically promising,

but it is also:

- not calibrated reflectivity,
- still range-dependent,
- still scene-dependent,
- and not uniformly better than raw intensity in every frame.

Those are not weaknesses to hide. They are part of the real result.

This notebook preserves that discipline from the very beginning of the final pipeline.

Final Takeaway of This Notebook

The correct role of this notebook is simple:

rebuild the full preliminary case in one place, preserve the valid evidence, preserve the limitations, and create a trusted baseline from which the actual final-stage expansion can proceed.

No fake-grand claims. No premature proxy wars. No segmentation hero cape yet.

Just one clean recap notebook that says:

the pipeline works, the proxy is meaningful, the gains are modest, the limits are real, and now the final project can begin properly.

Locate the dataset and verify the sequence structure

We define the SemanticKITTI sequence path and check whether the expected files and folders are present.

We are not loading any point clouds yet.

The goal here is only to confirm that the notebook is looking at the correct location and that the basic directory structure is valid before moving forward.

This is the same quiet sanity-check spirit as preliminary Notebook 01: first trust the path, then trust the data.

```
from pathlib import Path

# This recap notebook lives in final_pipeline/, so we allow both:
```

```python
# 1) the original preliminary path
# 2) the planned final-stage renamed path
candidate_roots = [
    Path("../data/semantickitti_subset/dataset/sequences/00"),
    Path("../data/semantickitti/dataset/sequences/00"),
]

DATASET_ROOT = next((p for p in candidate_roots if p.exists()),
candidate_roots[0])

expected_items = [
    "velodyne",
    "labels",
    "calib.txt",
    "poses.txt",
    "times.txt",
]

print("Dataset root:", DATASET_ROOT.resolve())
print("Exists:", DATASET_ROOT.exists())
print()

for item in expected_items:
    item_path = DATASET_ROOT / item
    print(f"{item:10} -> exists: {item_path.exists()} | path:
{item_path}")

print()
if DATASET_ROOT.exists():
    print("Contents of sequence folder:")
    for p in sorted(DATASET_ROOT.iterdir()):
        print(" -", p.name)
else:
    print("Dataset root not found. Checked these candidate paths:")
    for p in candidate_roots:
        print(" -", p.resolve())
```

```
Dataset root:
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti/dataset
/sequences/00
Exists: True

velodyne   -> exists: True | path:
../data/semantickitti/dataset/sequences/00/velodyne
labels     -> exists: True | path:
../data/semantickitti/dataset/sequences/00/labels
calib.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/calib.txt
poses.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/poses.txt
```

```
times.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/times.txt

Contents of sequence folder:
 - calib.txt
 - labels
 - poses.txt
 - times.txt
 - velodyne
```

List available frames and confirm file counts

We inspect the `velodyne` and `labels` folders to confirm that frame files are present and that both folders contain matching counts.

This gives us a quick structural sanity check before loading the first frame.

If the counts and frame IDs match, then the dataset structure is not only present, but internally consistent enough for frame-level loading in the next combo.

```
velodyne_dir = DATASET_ROOT / "velodyne"
labels_dir = DATASET_ROOT / "labels"

velodyne_files = sorted(velodyne_dir.glob("*.bin"))
label_files = sorted(labels_dir.glob("*.label"))

print("Number of .bin frames   :", len(velodyne_files))
print("Number of .label frames :", len(label_files))
print()

if velodyne_files:
    print("First 5 velodyne frames:")
    for f in velodyne_files[:5]:
        print(" -", f.name)

print()

if label_files:
    print("First 5 label frames:")
    for f in label_files[:5]:
        print(" -", f.name)

print()
if velodyne_files and label_files:
    velodyne_stems = [f.stem for f in velodyne_files]
    label_stems = [f.stem for f in label_files]

    print("Counts match   :", len(velodyne_files) == len(label_files))
    print("Frame IDs match:", velodyne_stems == label_stems)
```

```
    if velodyne_stems == label_stems:
        print("Sample frame ID:", velodyne_stems[0])
```

```
Number of .bin frames   : 4541
Number of .label frames : 4541

First 5 velodyne frames:
 - 000000.bin
 - 000001.bin
 - 000002.bin
 - 000003.bin
 - 000004.bin

First 5 label frames:
 - 000000.label
 - 000001.label
 - 000002.label
 - 000003.label
 - 000004.label

Counts match   : True
Frame IDs match: True
Sample frame ID: 000000
```

Load the first LiDAR frame

We load the first `.bin` file from the `velodyne` folder.

Each SemanticKITTI LiDAR frame stores raw point-cloud data as `float32` values arranged as four channels per point:

- `x`
- `y`
- `z`
- `intensity`

The goal here is to load the first frame, reshape it into an `N x 4` array, and inspect its basic structure.

This is the first actual contact with the raw LiDAR data.

```
import numpy as np

frame_id = velodyne_files[0].stem
bin_path = velodyne_dir / f"{frame_id}.bin"

points = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)

print("Frame ID:", frame_id)
print("File path:", bin_path)
```

```
print("Points array shape:", points.shape)
print("Data type:", points.dtype)
print()

print("First 5 points:")
print(points[:5])
```

```
Frame ID: 000000
File path:
../data/semantickitti/dataset/sequences/00/velodyne/000000.bin
Points array shape: (124668, 4)
Data type: float32

First 5 points:
[[5.2897942e+01 2.2989739e-02 1.9979945e+00 7.9999998e-02]
 [5.3750526e+01 1.9291429e-01 2.0269539e+00 0.0000000e+00]
 [5.3803116e+01 3.6183926e-01 2.0289137e+00 0.0000000e+00]
 [7.2600662e+01 1.2964664e+00 2.6647117e+00 0.0000000e+00]
 [7.2118256e+01 1.5133761e+00 2.6476634e+00 0.0000000e+00]]
```

Load the matching label frame

We load the label file corresponding to the same frame.

Each label is stored as an unsigned 32-bit integer. We first inspect the raw label array, and then in later combos we will separate semantic and instance information.

The goal here is simply to confirm that the matching label frame loads correctly and has the expected one-dimensional structure.

```
label_path = labels_dir / f"{frame_id}.label"

labels = np.fromfile(label_path, dtype=np.uint32)

print("Frame ID:", frame_id)
print("File path:", label_path)
print("Labels array shape:", labels.shape)
print("Data type:", labels.dtype)
print()

print("First 10 raw labels:")
print(labels[:10])
```

```
Frame ID: 000000
File path:
../data/semantickitti/dataset/sequences/00/labels/000000.label
Labels array shape: (124668,)
Data type: uint32
```

```
First 10 raw labels:
[0 0 0 0 0 0 0 0 0 0]
```

Verify point-label alignment

We confirm that the loaded point cloud and label array correspond to the same frame at the same point count.

This is a critical sanity check because each LiDAR point should have exactly one matching label entry.

If this check passes, then the raw geometric data and the label stream are aligned correctly, and later semantic analysis will stand on solid ground.

```
num_points = points.shape[0]
num_labels = labels.shape[0]

print("Number of points :", num_points)
print("Number of labels :", num_labels)
print("Counts match     :", num_points == num_labels)

assert num_points == num_labels, "Point-label mismatch detected!"
print("\nAlignment check passed.")
```

```
Number of points : 124668
Number of labels : 124668
Counts match     : True

Alignment check passed.
```

Separate semantic and instance label information

We unpack each raw 32-bit label into its semantic class ID and instance ID components.

This makes the labels interpretable and prepares them for basic upcoming inspection.

The raw label array is not yet human-friendly by itself. Splitting it into semantic and instance parts gives us the actual class information needed for later analysis.

```
semantic_labels = labels & 0xFFFF
instance_labels = labels >> 16

print("Semantic labels shape:", semantic_labels.shape)
print("Instance labels shape:", instance_labels.shape)
print()

print("First 10 semantic labels:")
print(semantic_labels[:10])
```

```
print("\nFirst 10 instance labels:")
print(instance_labels[:10])
```

```
Semantic labels shape: (124668,)
Instance labels shape: (124668,)

First 10 semantic labels:
[0 0 0 0 0 0 0 0 0 0]

First 10 instance labels:
[0 0 0 0 0 0 0 0 0 0]
```

Inspect the semantic classes present in the frame

We now identify the unique semantic class IDs present in the selected frame and count how many points belong to each class.

This gives a first snapshot of the frame's semantic composition.

We are still not doing deep semantic analysis yet. The goal here is only to confirm that the unpacked semantic labels are meaningful, nontrivial, and suitable for later frame-level interpretation.

```
unique_semantic, semantic_counts = np.unique(semantic_labels,
return_counts=True)

print("Number of unique semantic classes in frame:",
len(unique_semantic))
print()

print("First 20 semantic class IDs and counts:")
for class_id, count in zip(unique_semantic[:20],
semantic_counts[:20]):
    print(f"Class {class_id:>3}: {count}")
```

```
Number of unique semantic classes in frame: 17

First 20 semantic class IDs and counts:
Class   0: 2184
Class   1: 4
Class  10: 4234
Class  40: 34228
Class  44: 3268
Class  48: 26360
Class  50: 18268
Class  51: 370
Class  52: 1471
Class  60: 1053
Class  70: 27123
Class  71: 1192
```

```
Class  72: 2964
Class  80: 532
Class  81: 102
Class  99: 1227
Class 255: 88
```

Compute range and summarize the raw signal

We separate the point cloud into coordinates and intensity, compute the range of each point from the sensor, and print basic descriptive statistics.

This gives us the first quantitative summary of the frame geometry and signal values.

At this stage, we are still doing disciplined sanity work:

- isolate the raw channels,
- compute a basic derived geometric quantity,
- and check whether the values look numerically plausible for a real LiDAR frame.

```
xyz = points[:, :3]
intensity = points[:, 3]
ranges = np.linalg.norm(xyz, axis=1)

print("XYZ shape:", xyz.shape)
print("Intensity shape:", intensity.shape)
print("Ranges shape:", ranges.shape)
print()

print("XYZ min values:", xyz.min(axis=0))
print("XYZ max values:", xyz.max(axis=0))
print()

print("Intensity stats")
print(" - min :", intensity.min())
print(" - max :", intensity.max())
print(" - mean:", intensity.mean())
print(" - std :", intensity.std())
print()

print("Range stats")
print(" - min :", ranges.min())
print(" - max :", ranges.max())
print(" - mean:", ranges.mean())
print(" - std :", ranges.std())
```

```
XYZ shape: (124668, 3)
Intensity shape: (124668,)
Ranges shape: (124668,)

XYZ min values: [-78.087395 -55.72341  -11.556541]
```

```
XYZ max values: [77.96733   44.878613   2.8253412]

Intensity stats
 - min : 0.0
 - max : 0.99
 - mean: 0.29413402
 - std : 0.14768986

Range stats
 - min : 1.3483586
 - max : 79.73653
 - mean: 13.565046
 - std : 10.570971
```

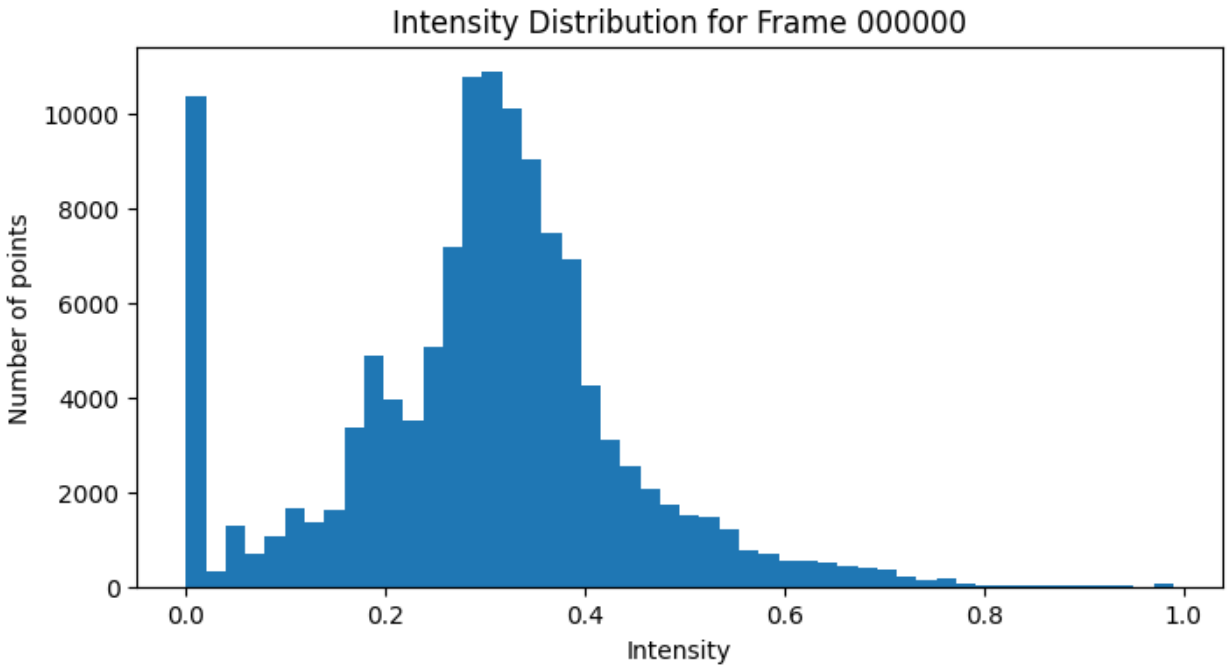
Visualize the intensity and range distributions

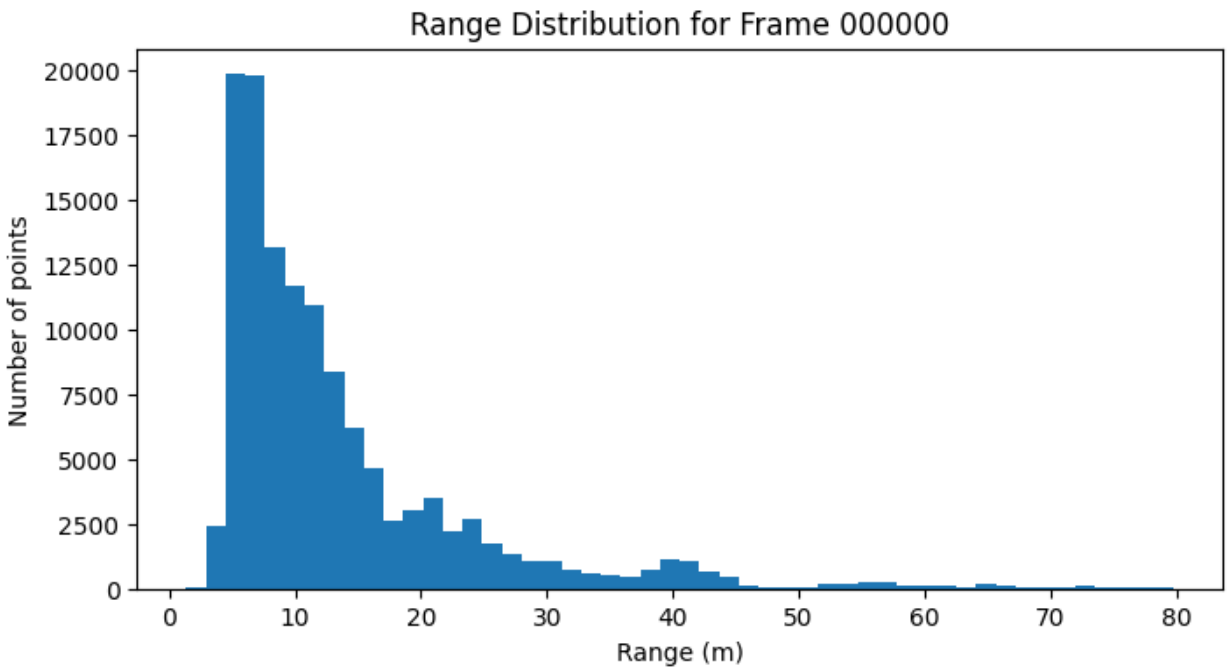
Here, we plot simple histograms for intensity and range to visually inspect how the raw signal and point distances are distributed within the frame.

This gives us a quick visual sanity check on the numeric summaries from the previous combo.

If these distributions look reasonable, then the frame is not only structurally valid but also statistically well-behaved enough for later reflectivity-style analysis.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))
plt.hist(intensity, bins=50)
plt.xlabel("Intensity")
plt.ylabel("Number of points")
plt.title("Intensity Distribution for Frame 000000")
plt.show()

plt.figure(figsize=(8, 4))
plt.hist(ranges, bins=50)
plt.xlabel("Range (m)")
plt.ylabel("Number of points")
plt.title("Range Distribution for Frame 000000")
plt.show()
```





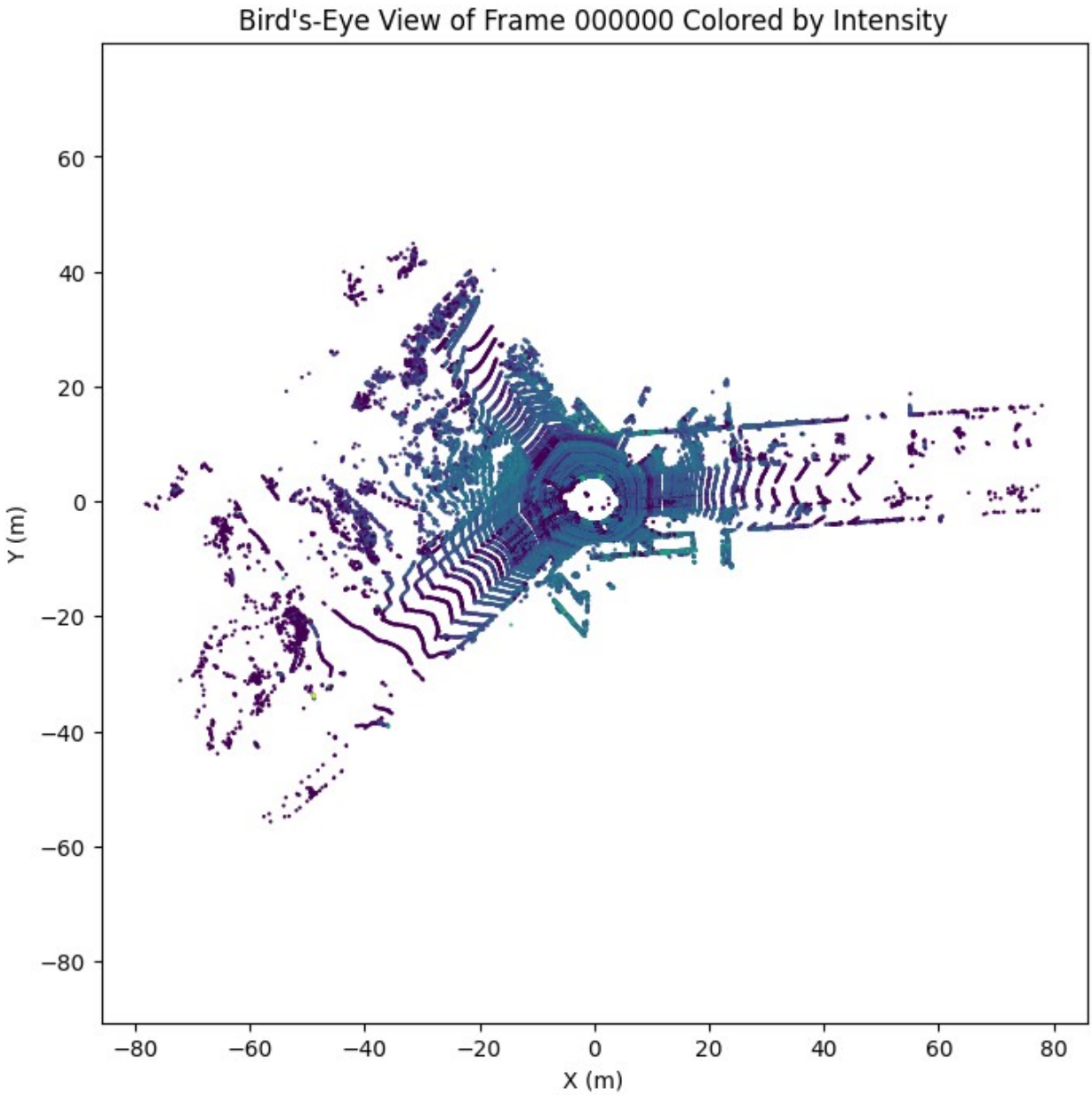
Create a bird's-eye view of the point cloud

We make a simple top-down scatter plot using the `x` and `y` coordinates, colored by intensity.

This gives a visual sanity check of the spatial structure of the frame and helps confirm that the point cloud resembles a real driving scene.

At this stage, the goal is still basic trust-building: the frame should look like a meaningful LiDAR scan, not cursed numeric soup.

```
plt.figure(figsize=(8, 8))
plt.scatter(
    xyz[:, 0],
    xyz[:, 1],
    c=intensity,
    s=0.5
)
plt.xlabel("X (m)")
plt.ylabel("Y (m)")
plt.title("Bird's-Eye View of Frame 000000 Colored by Intensity")
plt.axis("equal")
plt.show()
```



Visualize the semantic composition of the frame

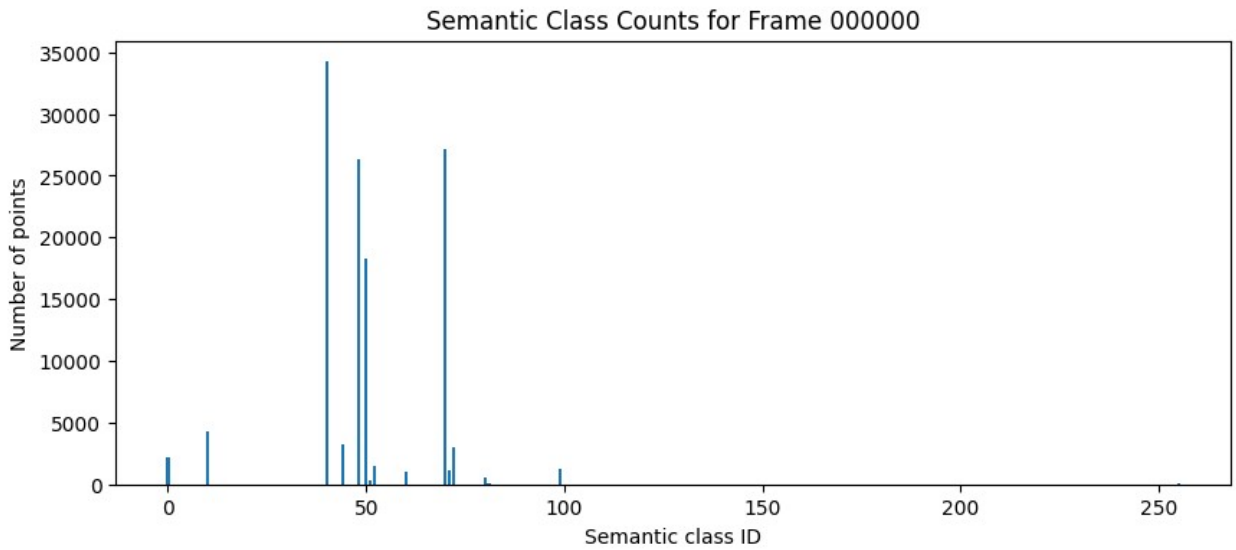
Here we create a bar chart of the semantic class IDs present in the selected frame and their corresponding point counts.

This gives a compact visual summary of the frame's label distribution.

The goal is not deep semantic interpretation yet. It is simply to confirm, in one visual check, that the semantic labels are present, nontrivial, and aligned with a realistic scene composition.

```
plt.figure(figsize=(10, 4))
plt.bar(unique_semantic, semantic_counts)
```

```
plt.xlabel("Semantic class ID")
plt.ylabel("Number of points")
plt.title("Semantic Class Counts for Frame 000000")
plt.show()
```



Rebuild the clean single-frame starting state for reflectivity-style analysis

Here we begin the next part of the recap notebook: the single-frame reflectivity-style analysis from the preliminary work.

We so far established that the SemanticKITTI loading pipeline is correct and that the raw LiDAR frame and labels are trustworthy.

Now we move one level deeper.

The goal of this combo is simple: reload one verified SemanticKITTI frame, reload its matching label file, and reconstruct the basic raw channels that the reflectivity-style analysis will build on top of.

We are not computing pseudo-reflectivity yet.

Right now, we are only rebuilding the clean starting state for the next section:

- point cloud,
- labels,
- coordinates,
- raw intensity,
- range,
- semantic labels,
- and instance labels.

This keeps the reflectivity-style analysis self-contained and makes the transition from pipeline sanity to signal interrogation explicit.

```
from pathlib import Path
import numpy as np

FRAME_ID = "000000"

bin_path = DATASET_ROOT / "velodyne" / f"{FRAME_ID}.bin"
label_path = DATASET_ROOT / "labels" / f"{FRAME_ID}.label"

points = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
labels = np.fromfile(label_path, dtype=np.uint32)

xyz = points[:, :3]
intensity = points[:, 3]
ranges = np.linalg.norm(xyz, axis=1)

semantic_labels = labels & 0xFFFF
instance_labels = labels >> 16

print("FRAME ID:", FRAME_ID)
print("Point cloud shape:", points.shape)
print("Label shape:", labels.shape)
print("XYZ shape:", xyz.shape)
print("Intensity shape:", intensity.shape)
print("Ranges shape:", ranges.shape)
print("Point-label aligned:", len(points) == len(labels))

print("\nFirst 3 points:")
print(points[:3])

print("\nFirst 10 semantic labels:")
print(semantic_labels[:10])

print("\nFirst 10 instance labels:")
print(instance_labels[:10])
```

```
FRAME ID: 000000
Point cloud shape: (124668, 4)
Label shape: (124668,)
XYZ shape: (124668, 3)
Intensity shape: (124668,)
Ranges shape: (124668,)
Point-label aligned: True

First 3 points:
[[5.2897942e+01 2.2989739e-02 1.9979945e+00 7.9999998e-02]
 [5.3750526e+01 1.9291429e-01 2.0269539e+00 0.0000000e+00]
 [5.3803116e+01 3.6183926e-01 2.0289137e+00 0.0000000e+00]]
```

```
First 10 semantic labels:
[0 0 0 0 0 0 0 0 0 0]

First 10 instance labels:
[0 0 0 0 0 0 0 0 0 0]
```

Define simple range-aware pseudo-reflectivity features

We construct simple reflectivity-style proxy signals from the verified single frame.

These are not physically calibrated reflectivity values. They are heuristic range-aware transformations of raw intensity that let us inspect whether compensating for distance reveals a more interpretable signal structure.

We begin with two simple variants:

$$\hat{\rho}^{(1)} = I \cdot R$$

and

$$\hat{\rho}^{(2)} = I \cdot R^2$$

The purpose of defining both is not to claim that both are good. It is to compare a moderate range-aware transformation against a stronger one and see whether the stronger compensation becomes too aggressive.

This is still a single-frame recap step. We are only building the candidate signals so that the next combos can compare them numerically and visually.

```
# Small epsilon for numerical safety, though ranges here should
already be > 0.
eps = 1e-6

pseudo_reflectivity_r1 = intensity * np.maximum(ranges, eps)
pseudo_reflectivity_r2 = intensity * np.maximum(ranges, eps) ** 2

print("Pseudo-reflectivity features created.")
print("r1 shape:", pseudo_reflectivity_r1.shape)
print("r2 shape:", pseudo_reflectivity_r2.shape)

print("\nFirst 10 raw intensity values:")
print(intensity[:10])

print("\nFirst 10 pseudo-reflectivity values: I * R")
print(pseudo_reflectivity_r1[:10])

print("\nFirst 10 pseudo-reflectivity values: I * R^2")
print(pseudo_reflectivity_r2[:10])
```

```
Pseudo-reflectivity features created.
r1 shape: (124668,)
```

```
r2 shape: (124668,)

First 10 raw intensity values:
[0.08 0.   0.   0.   0.   0.   0.   0.   0.   0.  ]

First 10 pseudo-reflectivity values: I * R
[4.2348533 0.        0.        0.        0.        0.        0.
 0.        0.        0.       ]

First 10 pseudo-reflectivity values: I * R^2
[224.17476   0.        0.        0.        0.        0.        0.
   0.        0.        0.     ]
```

Compare the three signals numerically

We will summarize the distributions of the three single-frame signals:

- raw intensity,
- pseudo-reflectivity $I \cdot R$,
- pseudo-reflectivity $I \cdot R^2$.

Before plotting anything, we first inspect the scale and spread numerically.

This helps us answer a basic question: how strongly do these range-aware transformations reshape the signal distribution?

If the stronger compensation grows too aggressively, we should see that directly in the summary statistics through a much heavier upper tail and a much larger spread.

```
def describe_signal(name, values):
    q = np.percentile(values, [0, 25, 50, 75, 95, 99, 100])
    print(f"{name}")
    print(f"  min   : {q[0]:.6f}")
    print(f"  p25   : {q[1]:.6f}")
    print(f"  p50   : {q[2]:.6f}")
    print(f"  p75   : {q[3]:.6f}")
    print(f"  p95   : {q[4]:.6f}")
    print(f"  p99   : {q[5]:.6f}")
    print(f"  max   : {q[6]:.6f}")
    print(f"  mean  : {values.mean():.6f}")
    print(f"  std   : {values.std():.6f}")
    print()

describe_signal("Raw intensity", intensity)
describe_signal("Pseudo-reflectivity (I * R)", pseudo_reflectivity_r1)
describe_signal("Pseudo-reflectivity (I * R^2)",
pseudo_reflectivity_r2)
```

```
Raw intensity
  min   : 0.000000
```

```
  p25   : 0.220000
  p50   : 0.300000
  p75   : 0.370000
  p95   : 0.540000
  p99   : 0.690000
  max   : 0.990000
  mean  : 0.294134
  std   : 0.147690

Pseudo-reflectivity (I * R)
  min   : 0.000000
  p25   : 1.669057
  p50   : 2.754553
  p75   : 5.201154
  p95   : 9.184167
  p99   : 12.255593
  max   : 58.805172
  mean  : 3.628692
  std   : 2.940125

Pseudo-reflectivity (I * R^2)
  min   : 0.000000
  p25   : 10.098474
  p50   : 24.684789
  p75   : 76.925367
  p95   : 233.098367
  p99   : 416.893155
  max   : 3991.761475
  mean  : 61.607342
  std   : 98.882828
```

Measure how each signal relates to distance from the sensor

We will now check how strongly each signal correlates with range.

This gives us a quick numerical sense of whether the transformed signals remain strongly distance-driven.

We compare:

- raw intensity vs range,
- pseudo-reflectivity $I \cdot R$ vs range,
- pseudo-reflectivity $I \cdot R^2$ vs range.

This is an important diagnostic step because the whole point of pseudo-reflectivity is not to pretend geometry vanishes, but to understand how the signal changes when range is explicitly brought into the feature.

If the stronger proxy overcompensates, that should show up here as an even stronger positive relationship with range.

```
corr_intensity_range = np.corrcoef(intensity, ranges)[0, 1]
corr_r1_range = np.corrcoef(pseudo_reflectivity_r1, ranges)[0, 1]
corr_r2_range = np.corrcoef(pseudo_reflectivity_r2, ranges)[0, 1]

print(f"Correlation between raw intensity and range           :
{corr_intensity_range:.6f}")
print(f"Correlation between pseudo-reflectivity I*R and range :
{corr_r1_range:.6f}")
print(f"Correlation between pseudo-reflectivity I*R^2 and range:
{corr_r2_range:.6f}")
```

```
Correlation between raw intensity and range           : -0.231389
Correlation between pseudo-reflectivity I*R and range : 0.398455
Correlation between pseudo-reflectivity I*R^2 and range: 0.568631
```

Visualize the signal distributions

We compare the distributions of the three single-frame signals visually.

Numerical summaries are useful, but plots make the distributional differences much easier to see.

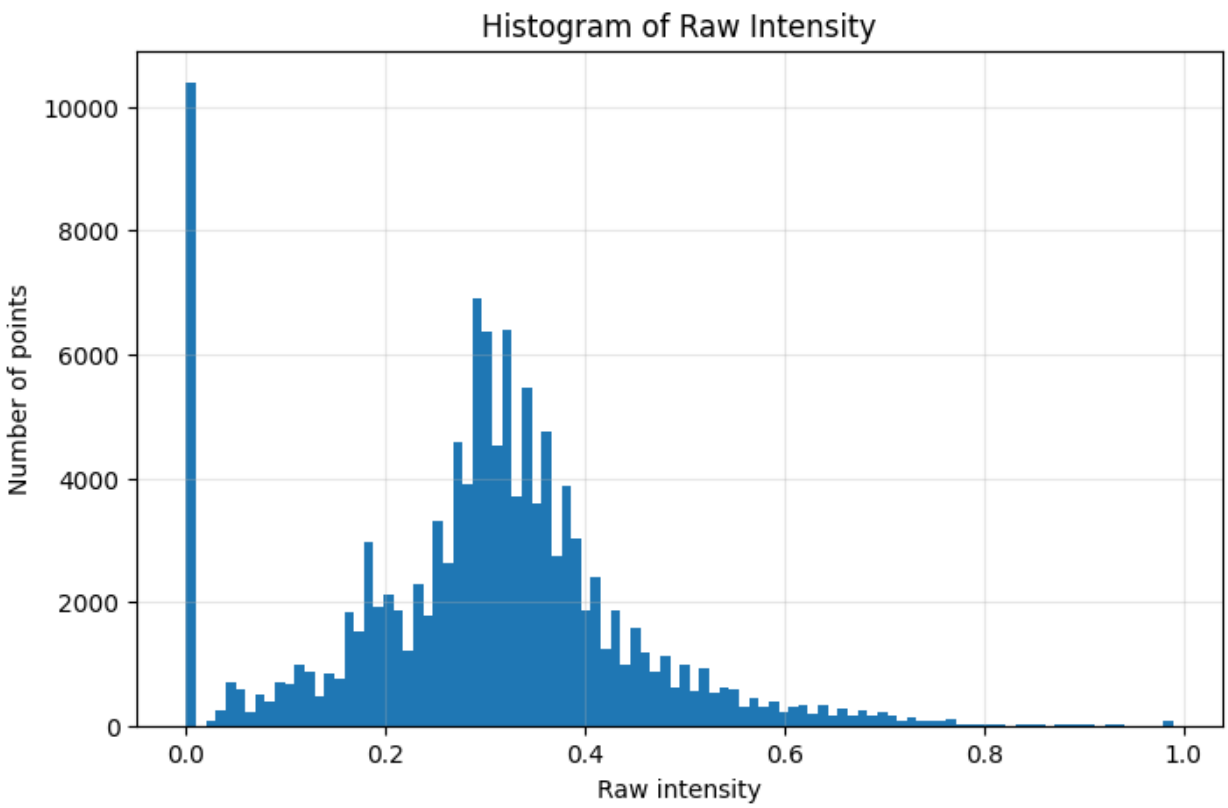
We now compare the histograms of:

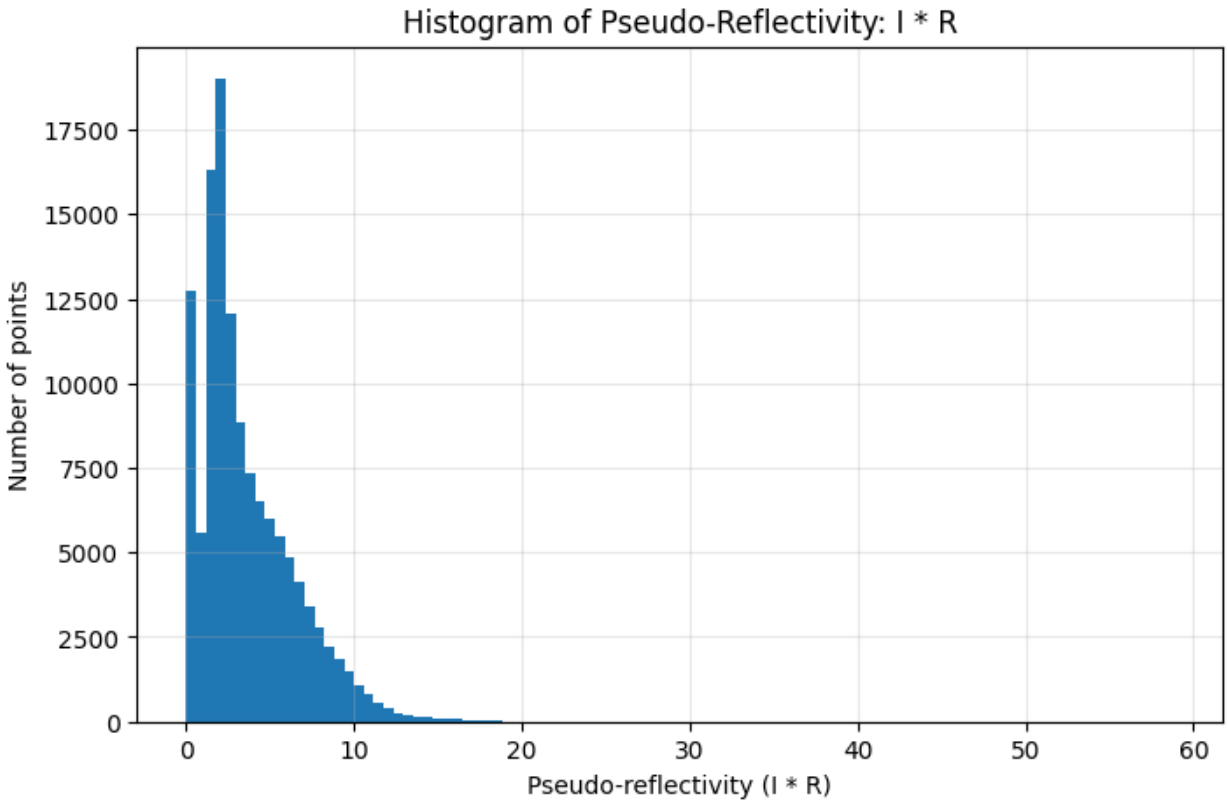
- raw intensity,
- pseudo-reflectivity $I \cdot R$,
- pseudo-reflectivity $I \cdot R^2$.

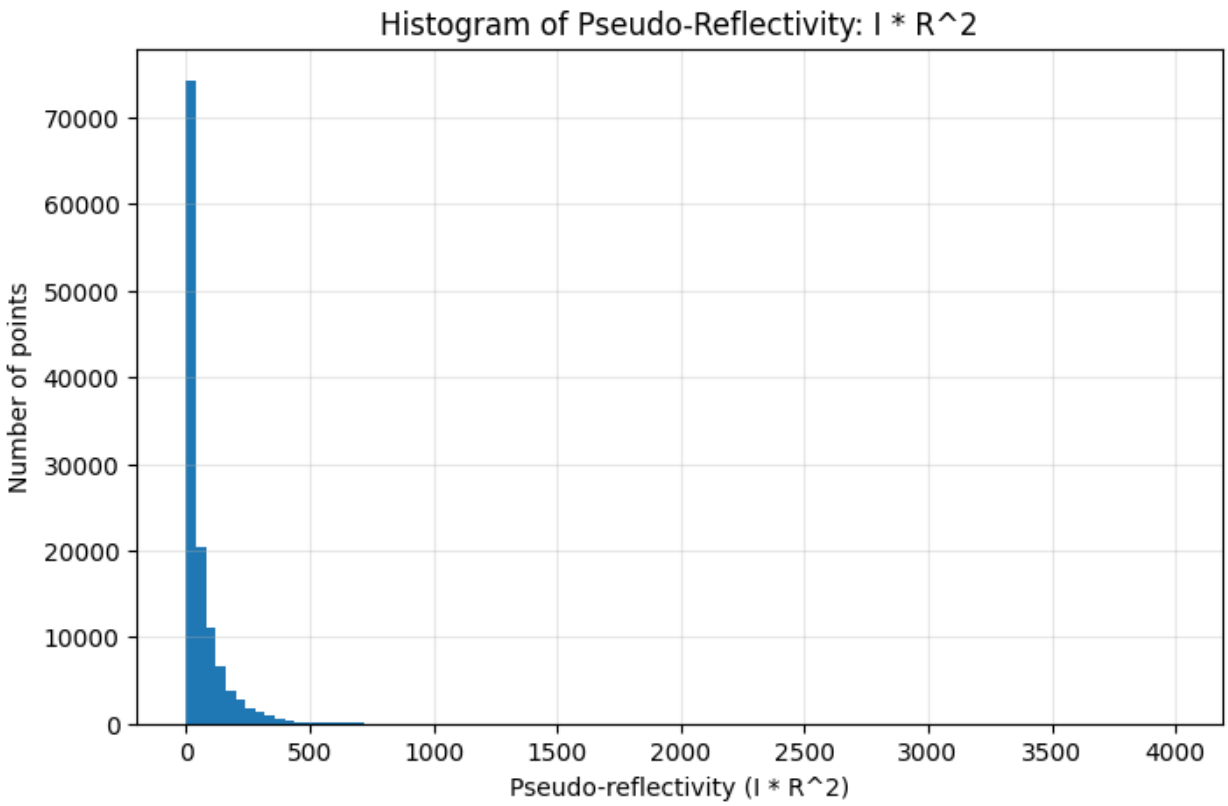
The goal is to see whether the moderate proxy stays interpretable and whether the stronger proxy develops an excessively heavy tail.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(intensity, bins=100)
plt.xlabel("Raw intensity")
plt.ylabel("Number of points")
plt.title("Histogram of Raw Intensity")
plt.grid(True, alpha=0.3)
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(pseudo_reflectivity_r1, bins=100)
plt.xlabel("Pseudo-reflectivity (I * R)")
plt.ylabel("Number of points")
plt.title("Histogram of Pseudo-Reflectivity: I * R")
plt.grid(True, alpha=0.3)
```

```
plt.show()

plt.figure(figsize=(8, 5))
plt.hist(pseudo_reflectivity_r2, bins=100)
plt.xlabel("Pseudo-reflectivity (I * R^2)")
plt.ylabel("Number of points")
plt.title("Histogram of Pseudo-Reflectivity: I * R^2")
plt.grid(True, alpha=0.3)
plt.show()
```




Histogram of Pseudo-Reflectivity: I * R
Number of points
17500
15000
12500
10000
7500
5000
2500
0
0
10
20
30
40
50
60
Pseudo-reflectivity (I * R)


Histogram of Pseudo-Reflectivity: I * R^2
Number of points
70000
60000
50000
40000
30000
20000
10000
0
0
500
1000
1500
2000
2500
3000
3500
4000
Pseudo-reflectivity (I * R^2)

Evaluate semantic usefulness numerically

We move from global signal distributions to semantic usefulness.

Global histograms tell us how the signal behaves overall, but they do not tell us whether the signal helps distinguish semantic classes.

So here we do the numerical part first:

1. identify the valid semantic classes in this frame,
2. keep only classes with enough support,
3. compute a simple Fisher-style multiclass separability score,
4. compare raw intensity against pseudo-reflectivity $I \cdot R$.

This lets us test, on the same single frame, whether the range-aware proxy produces a more semantically discriminative signal than raw intensity alone.

We are intentionally using $I \cdot R$ here, not $I \cdot R^2$, because the earlier cells already showed that the stronger proxy looks numerically overcompensated.

```
from collections import Counter

# Exclude unlabeled / ignored classes
ignore_labels = {0, 255}

# Count classes in this frame
class_counts = Counter(semantic_labels.tolist())

# Keep only classes with enough support
min_points = 500
valid_classes = sorted([
    cls for cls, cnt in class_counts.items()
    if cls not in ignore_labels and cnt >= min_points
])

print("Valid classes used for analysis:")
for cls in valid_classes:
    print(f"  Class {cls}: {class_counts[cls]} points")

def fisher_separability(signal, labels, classes):
    class_means = []
    class_vars = []
    class_ns = []

    for cls in classes:
        vals = signal[labels == cls]
        class_means.append(vals.mean())
        class_vars.append(vals.var())
        class_ns.append(len(vals))
```

```python
    class_means = np.array(class_means)
    class_vars = np.array(class_vars)
    class_ns = np.array(class_ns)

    global_mean = np.average(class_means, weights=class_ns)
    between = np.sum(class_ns * (class_means - global_mean) ** 2)
    within = np.sum(class_ns * class_vars)

    score = between / (within + 1e-12)
    return score, class_means, class_vars, class_ns

score_intensity, means_I, vars_I, ns = fisher_separability(
    intensity, semantic_labels, valid_classes
)
score_r1, means_R1, vars_R1, _ = fisher_separability(
    pseudo_reflectivity_r1, semantic_labels, valid_classes
)

print("\nClass-separability score (higher is better):")
print(f"  Raw intensity           : {score_intensity:.8f}")
print(f"  Pseudo-reflectivity I*R : {score_r1:.8f}")

print("\nPer-class mean values:")
for i, cls in enumerate(valid_classes):
    print(
        f"  Class {cls:>3} | "
        f"mean(I) = {means_I[i]:.6f} | "
        f"mean(I*R) = {means_R1[i]:.6f} | "
        f"n = {ns[i]}"
    )
```

```
Valid classes used for analysis:
  Class 10: 4234 points
  Class 40: 34228 points
  Class 44: 3268 points
  Class 48: 26360 points
  Class 50: 18268 points
  Class 52: 1471 points
  Class 60: 1053 points
  Class 70: 27123 points
  Class 71: 1192 points
  Class 72: 2964 points
  Class 80: 532 points
  Class 99: 1227 points

Class-separability score (higher is better):
  Raw intensity           : 0.38142828
  Pseudo-reflectivity I*R : 0.57723763

Per-class mean values:
```

```
Class  10 | mean(I) = 0.153462 | mean(I*R) = 2.553735 | n = 4234
Class  40 | mean(I) = 0.213374 | mean(I*R) = 1.895020 | n = 34228
Class  44 | mean(I) = 0.299865 | mean(I*R) = 3.833581 | n = 3268
Class  48 | mean(I) = 0.300424 | mean(I*R) = 2.472348 | n = 26360
Class  50 | mean(I) = 0.428269 | mean(I*R) = 6.293034 | n = 18268
Class  52 | mean(I) = 0.344582 | mean(I*R) = 4.121638 | n = 1471
Class  60 | mean(I) = 0.408528 | mean(I*R) = 2.683908 | n = 1053
Class  70 | mean(I) = 0.333848 | mean(I*R) = 4.861657 | n = 27123
Class  71 | mean(I) = 0.305696 | mean(I*R) = 6.802622 | n = 1192
Class  72 | mean(I) = 0.311606 | mean(I*R) = 6.411902 | n = 2964
Class  80 | mean(I) = 0.203421 | mean(I*R) = 2.421643 | n = 532
Class  99 | mean(I) = 0.279446 | mean(I*R) = 4.727847 | n = 1227
```

Visualize class-wise signal distributions

In this code, we visualize the class-wise distributions for the valid semantic classes.

The separability score already told us numerically that pseudo-reflectivity $I \cdot R$ is more semantically discriminative than raw intensity on this frame.

Now we inspect that story visually.

For each valid semantic class, we plot:

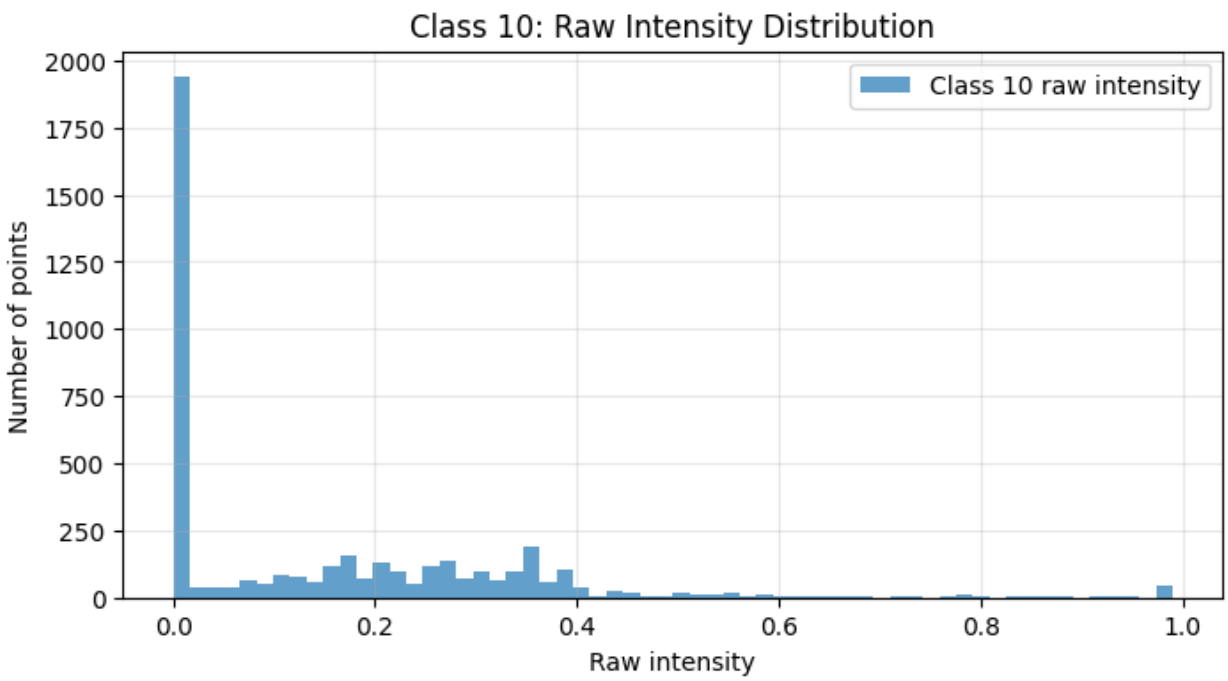
- the raw intensity distribution,
- and the pseudo-reflectivity $I \cdot R$ distribution.

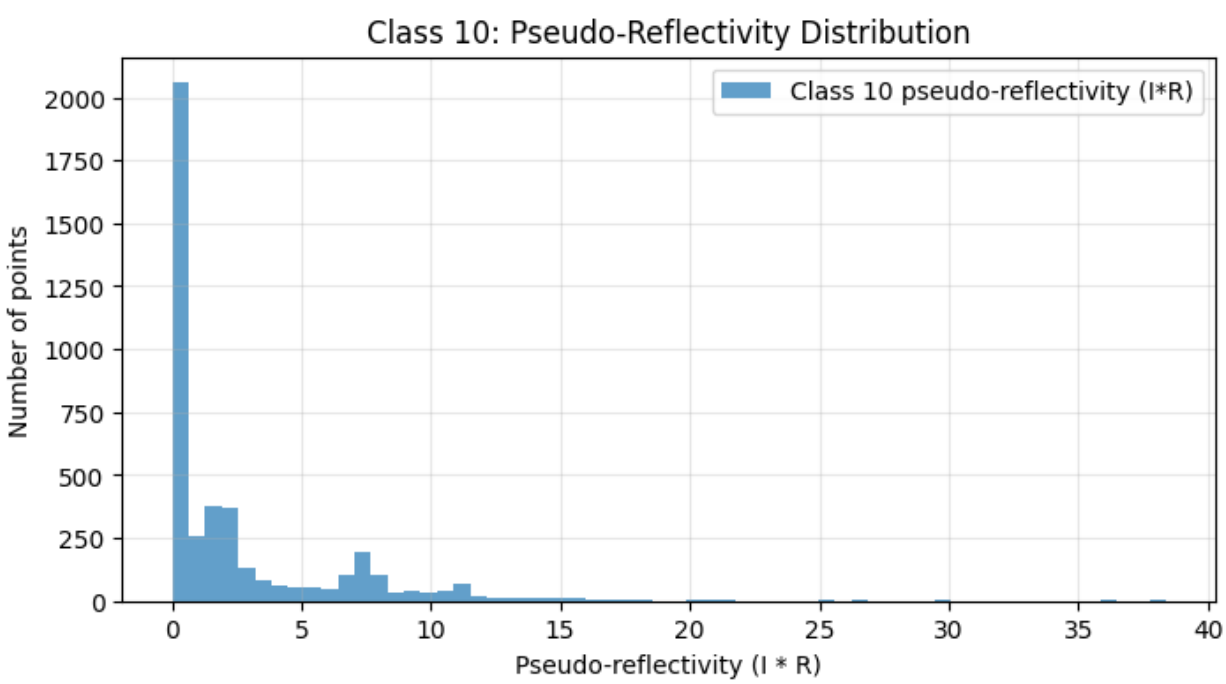
This is a heavier code than the earlier ones because it will generate many figures.

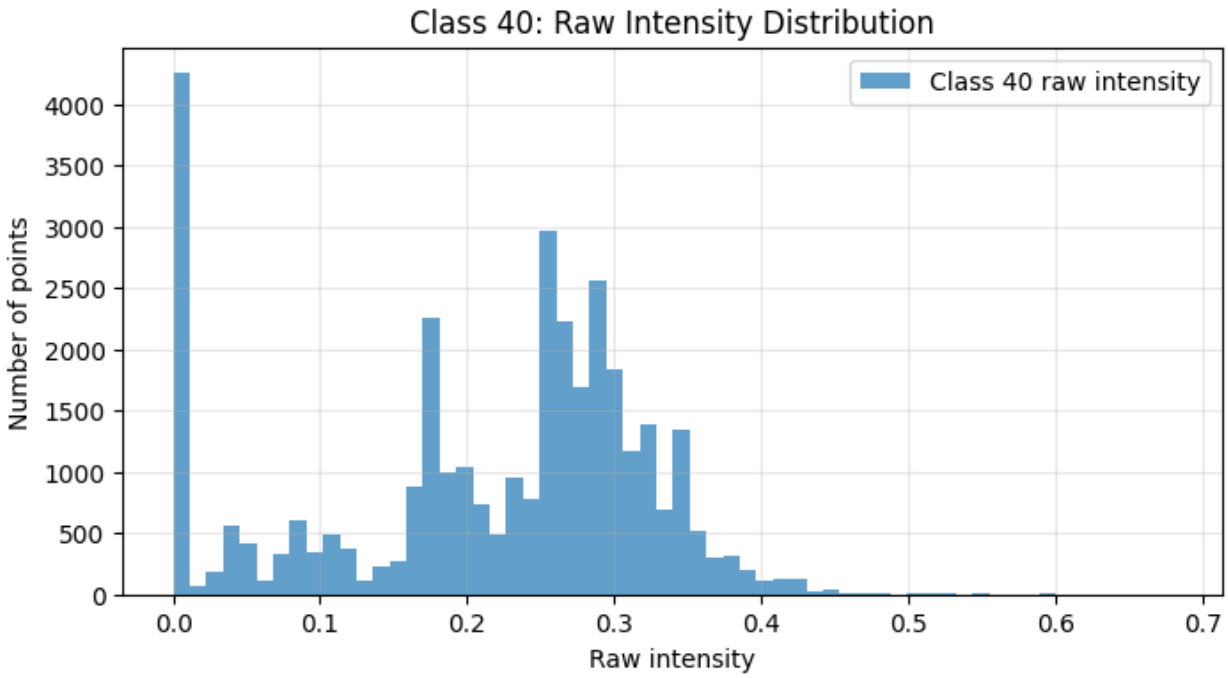
The goal is to see whether the transformed signal changes the class distributions in a way that makes at least some classes more distinguishable than with raw intensity alone.

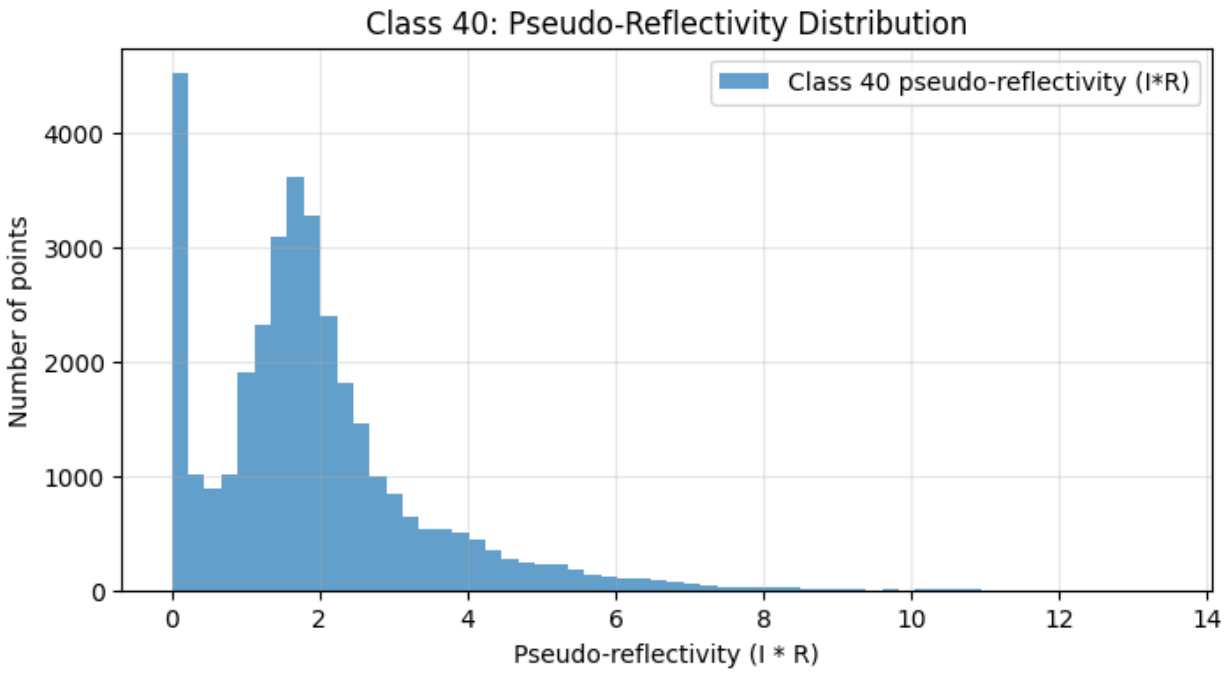
```
for cls in valid_classes:
    vals_I = intensity[semantic_labels == cls]
    vals_R1 = pseudo_reflectivity_r1[semantic_labels == cls]

    plt.figure(figsize=(8, 4))
    plt.hist(vals_I, bins=60, alpha=0.7, label=f"Class {cls} raw
intensity")
    plt.xlabel("Raw intensity")
    plt.ylabel("Number of points")
    plt.title(f"Class {cls}: Raw Intensity Distribution")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()

    plt.figure(figsize=(8, 4))
    plt.hist(vals_R1, bins=60, alpha=0.7, label=f"Class {cls} pseudo-
reflectivity (I*R)")
    plt.xlabel("Pseudo-reflectivity (I * R)")
```

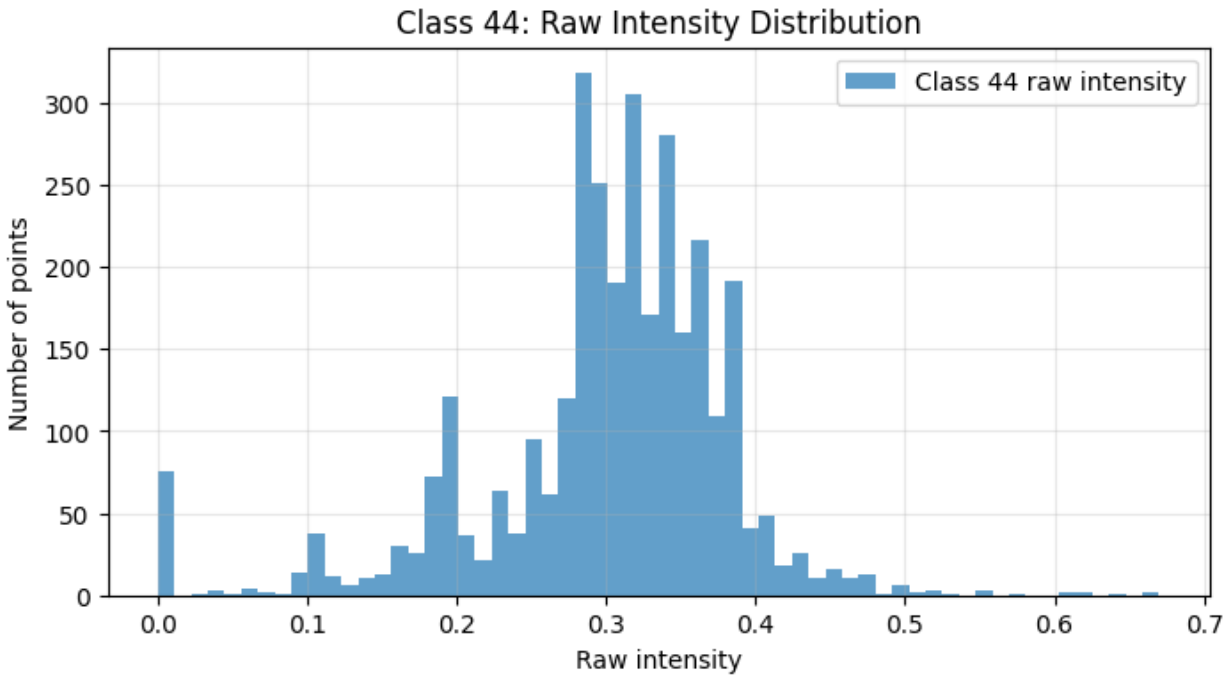
```
    plt.ylabel("Number of points")
    plt.title(f"Class {cls}: Pseudo-Reflectivity Distribution")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()
```

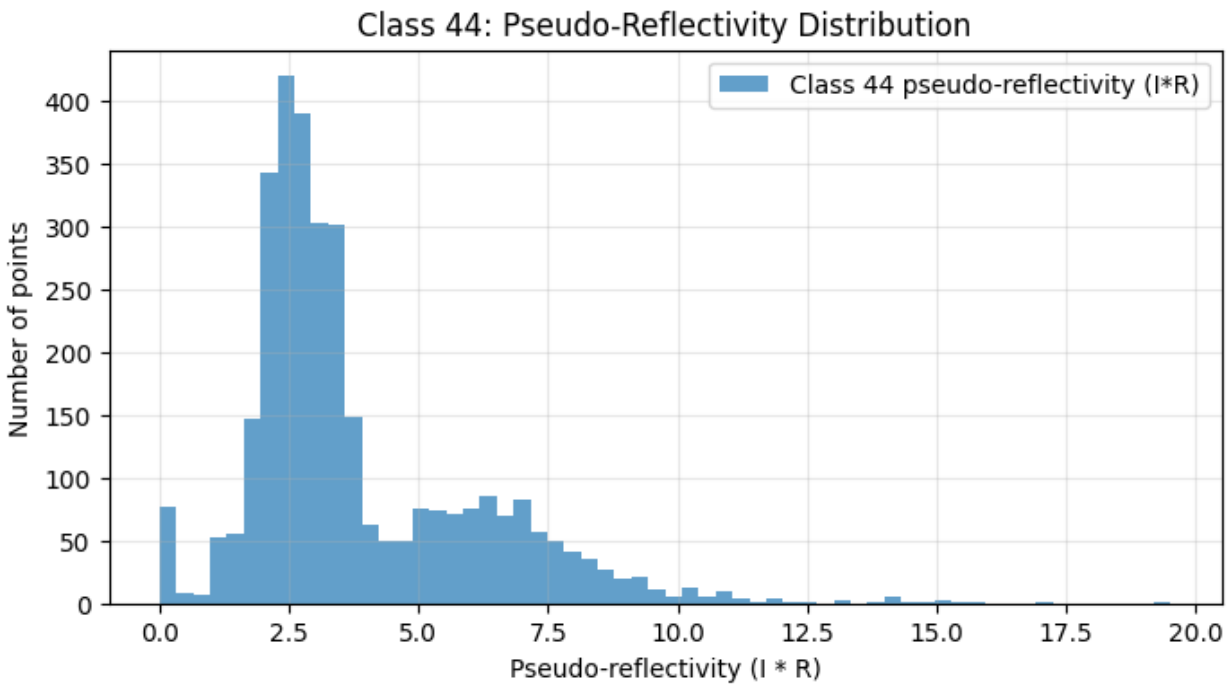


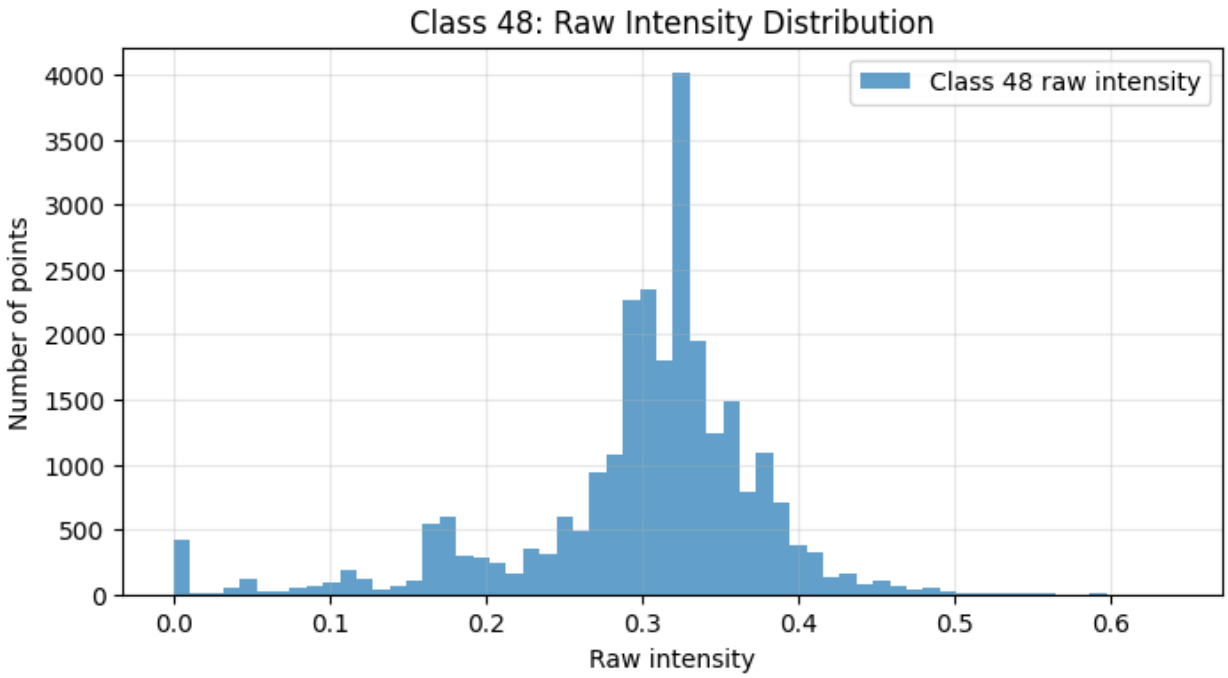


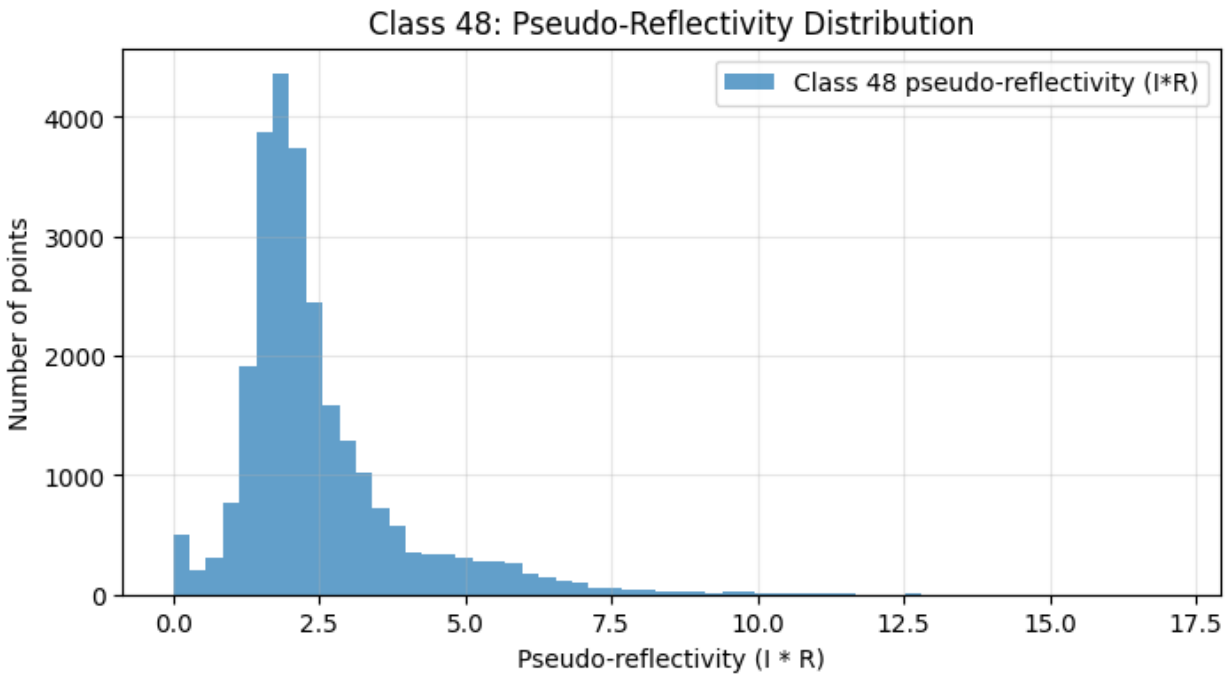

Class 40: Raw Intensity Distribution
Class 40 raw intensity
Number of points
Raw intensity
0
500
1000
1500
2000
2500
3000
3500
4000
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7

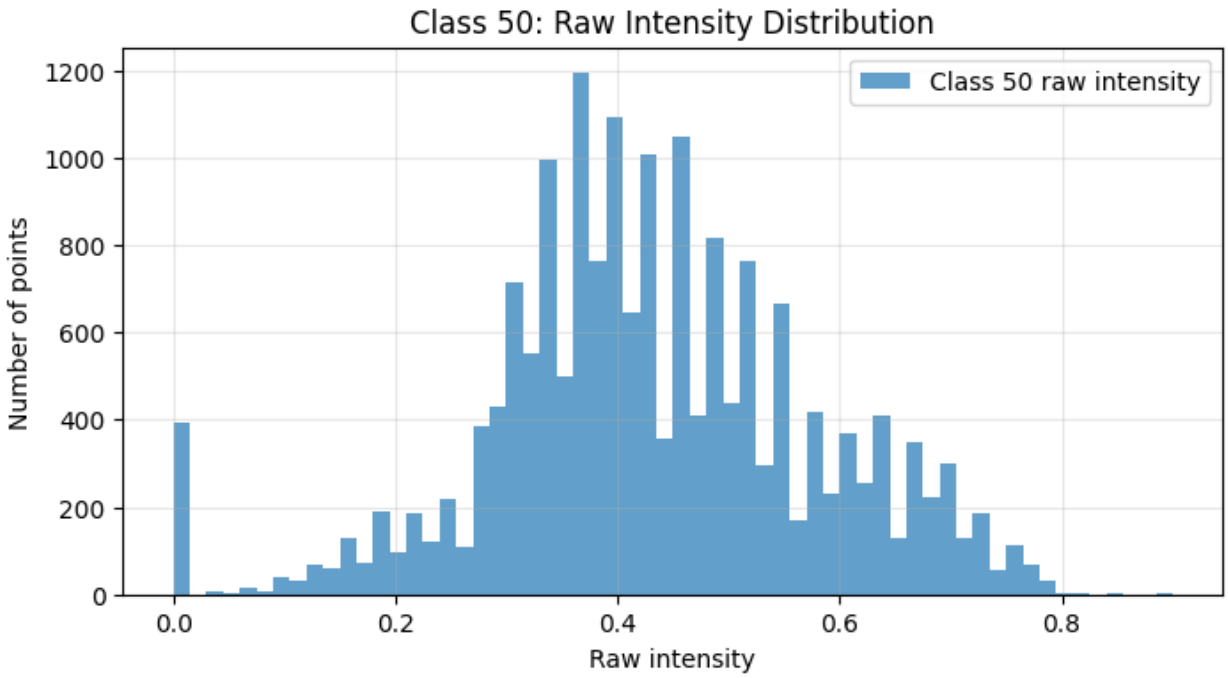

Class 40: Pseudo-Reflectivity Distribution
Class 40 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
1000
2000
3000
4000
0
2
4
6
8
10
12
14

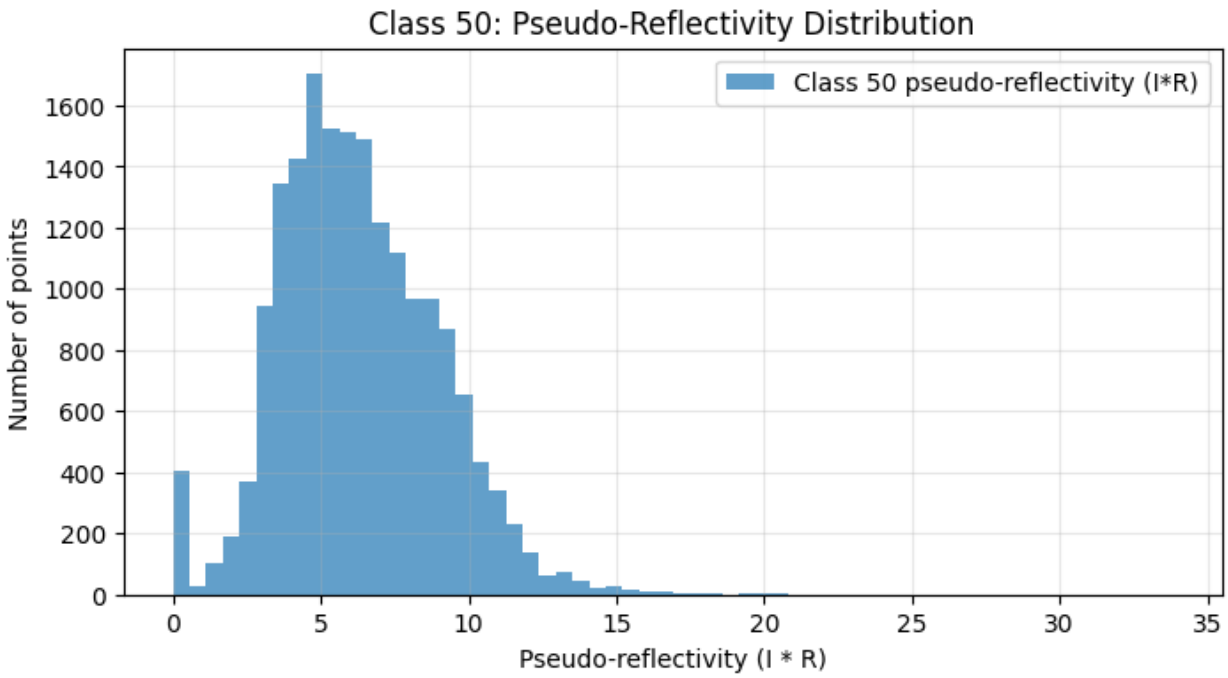

Class 44: Raw Intensity Distribution
Class 44 raw intensity
Number of points
Raw intensity
0
50
100
150
200
250
300
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7

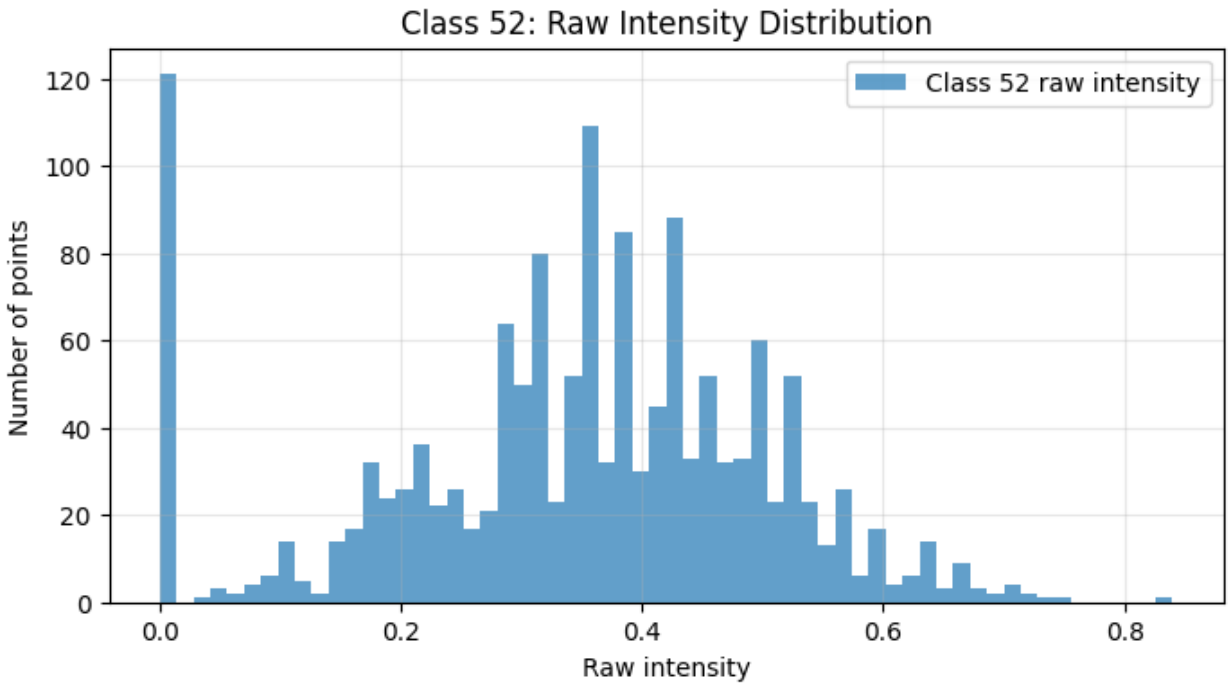

Class 44: Pseudo-Reflectivity Distribution
Class 44 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
50
100
150
200
250
300
350
400
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

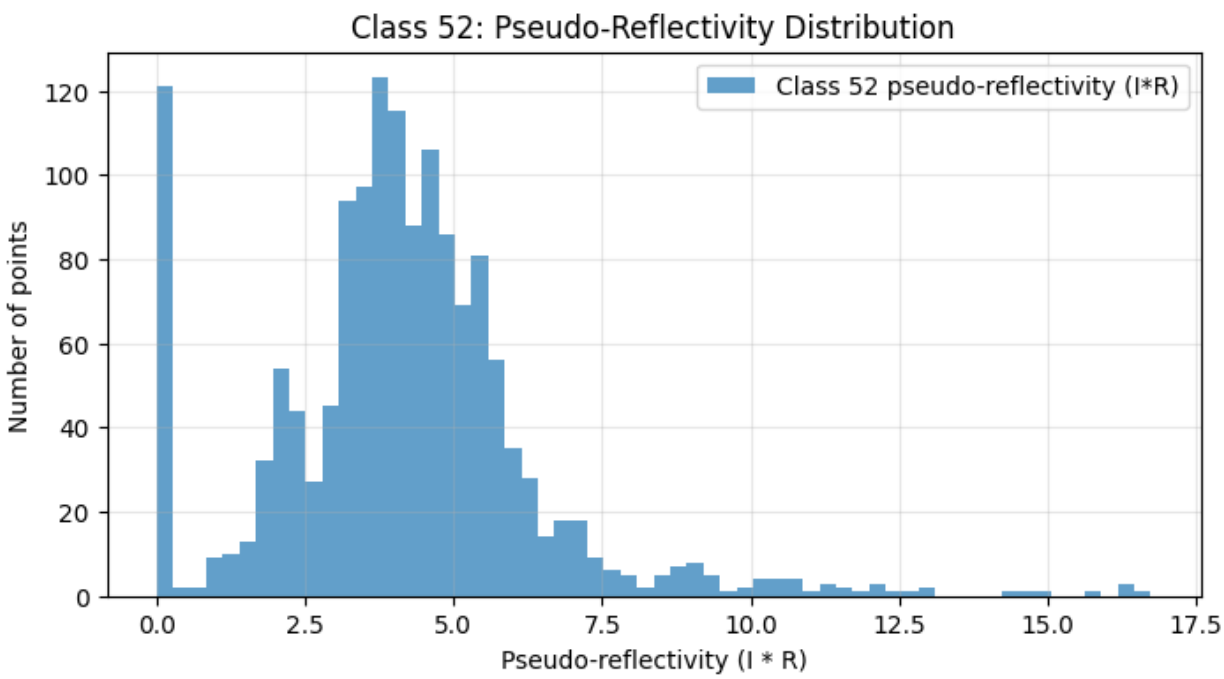

Class 48: Raw Intensity Distribution
Class 48 raw intensity
Number of points
Raw intensity
0
500
1000
1500
2000
2500
3000
3500
4000
0.0
0.1
0.2
0.3
0.4
0.5
0.6

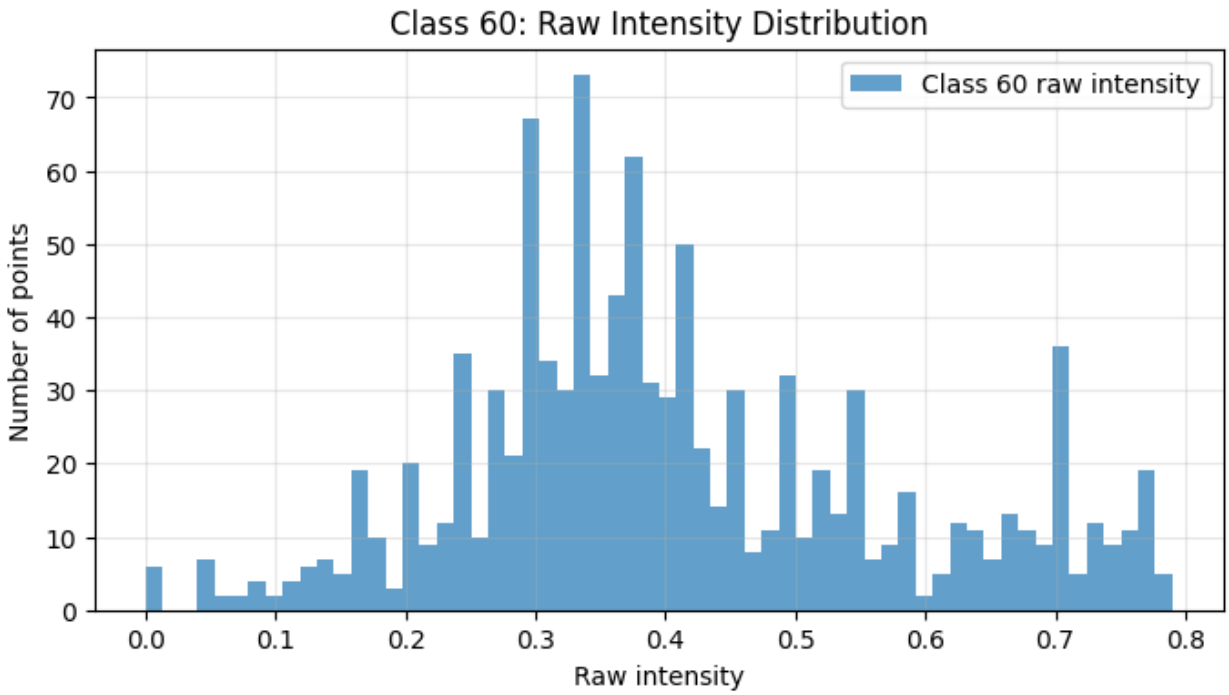

Class 48: Pseudo-Reflectivity Distribution
Class 48 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
1000
2000
3000
4000
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

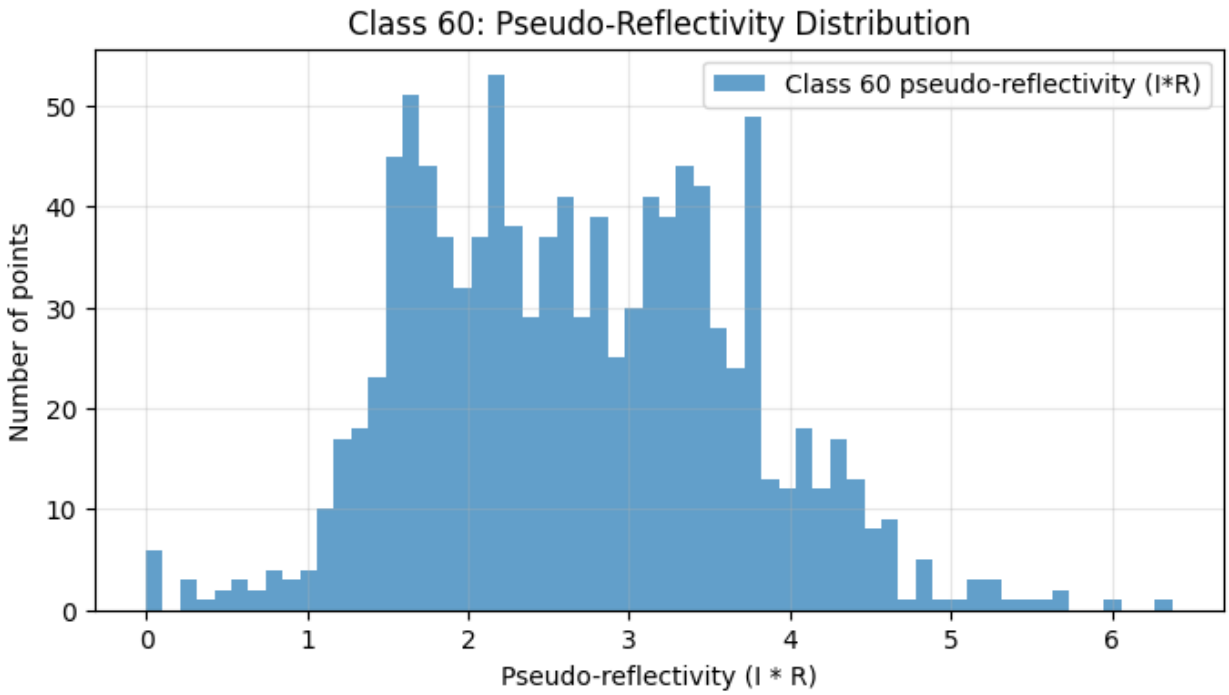

Class 50: Raw Intensity Distribution
Class 50 raw intensity
Number of points
Raw intensity
0
200
400
600
800
1000
1200
0.0
0.2
0.4
0.6
0.8

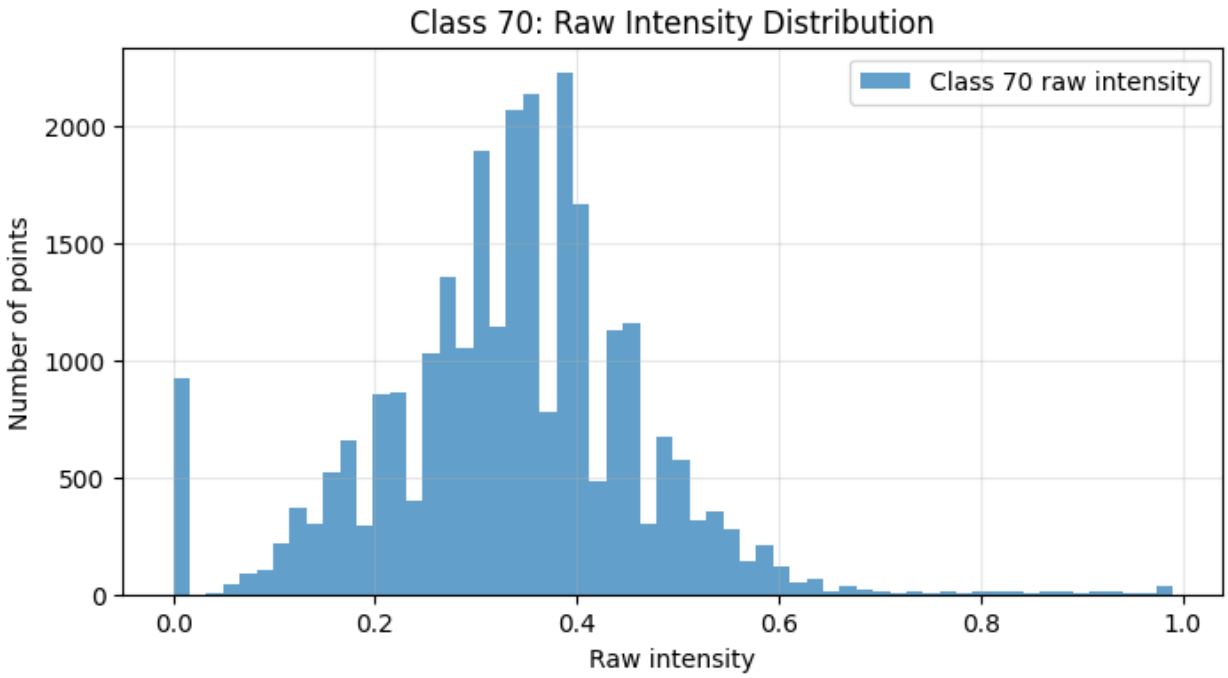

Class 50: Pseudo-Reflectivity Distribution
Class 50 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
200
400
600
800
1000
1200
1400
1600
0
5
10
15
20
25
30
35

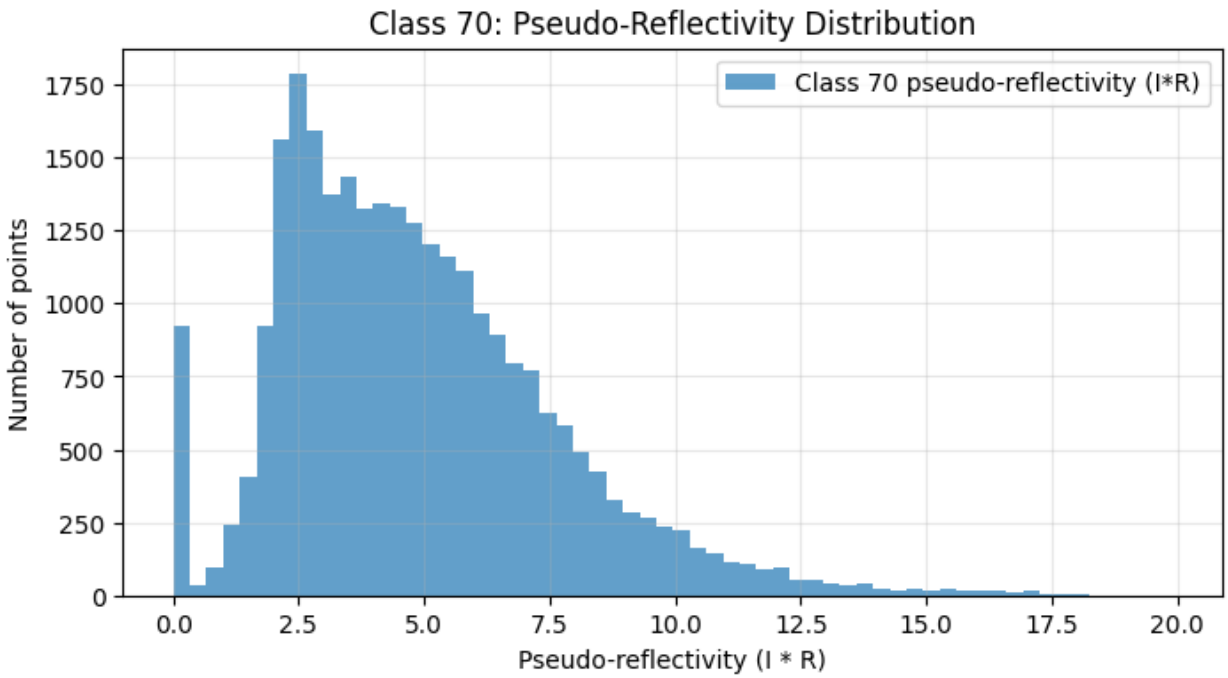

Class 52: Raw Intensity Distribution
Class 52 raw intensity
Number of points
Raw intensity
0
20
40
60
80
100
120
0.0
0.2
0.4
0.6
0.8

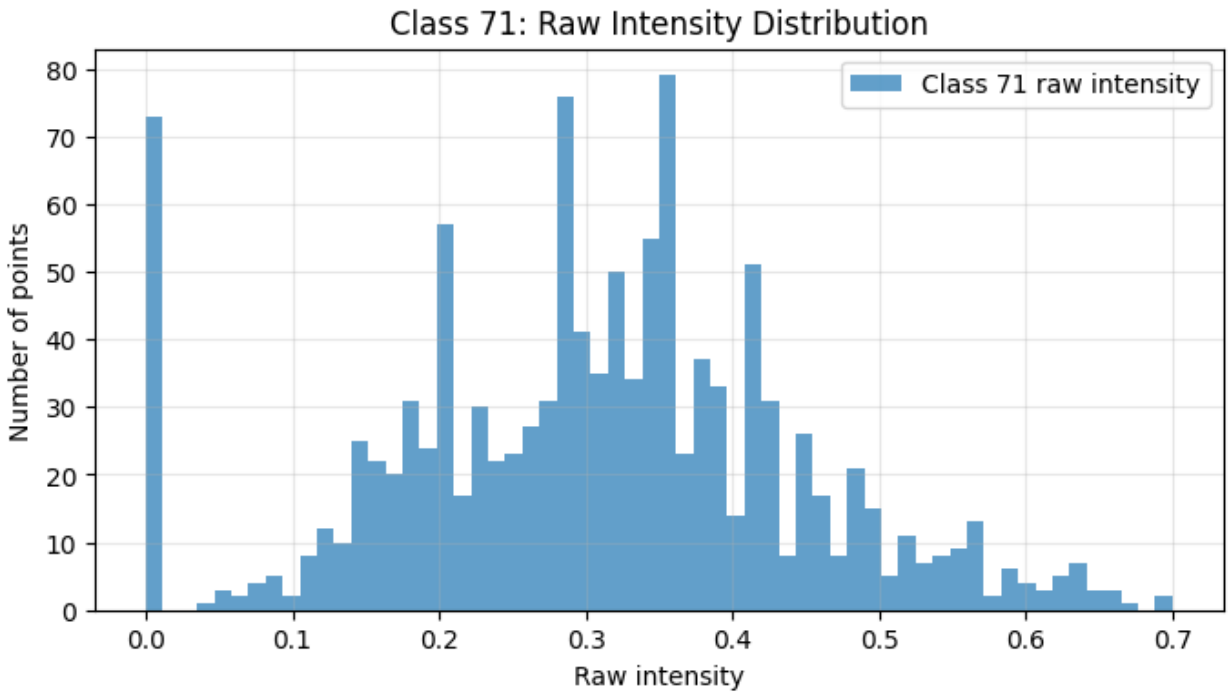

Class 52: Pseudo-Reflectivity Distribution
Class 52 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
20
40
60
80
100
120
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

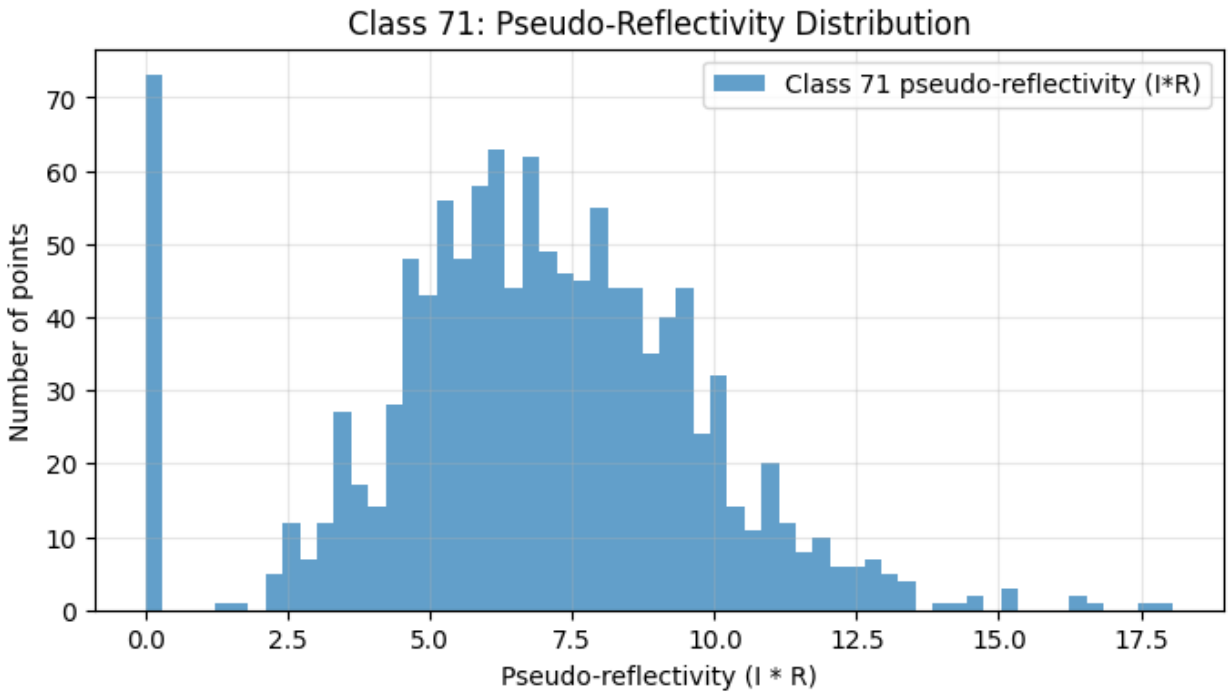

Class 60: Raw Intensity Distribution
Class 60 raw intensity
Number of points
Raw intensity
0
10
20
30
40
50
60
70
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8

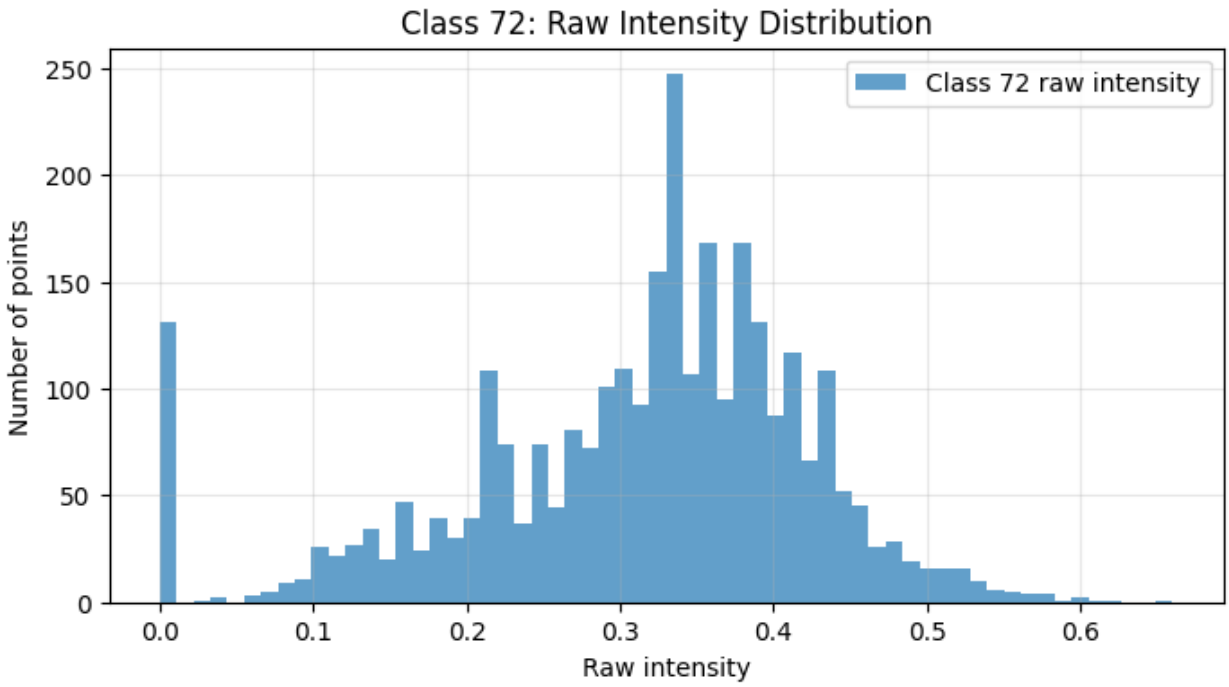

Class 60: Pseudo-Reflectivity Distribution
Class 60 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
10
20
30
40
50
0
1
2
3
4
5
6

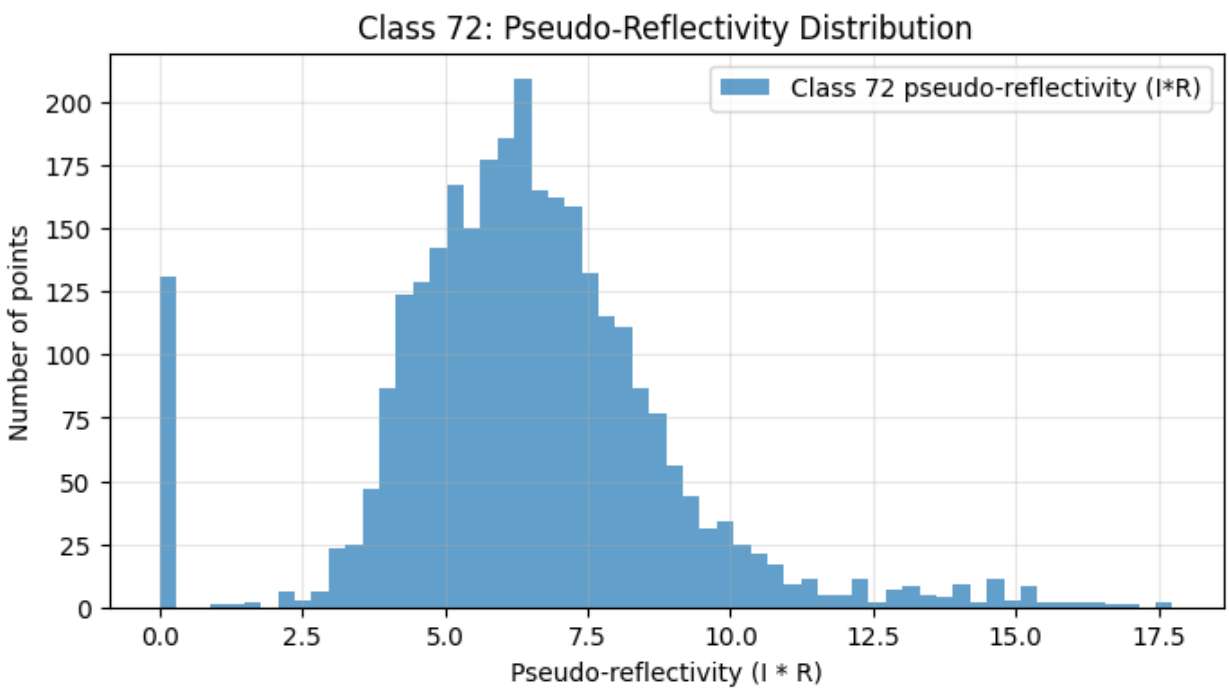

Class 70: Raw Intensity Distribution
Class 70 raw intensity
Number of points
0
500
1000
1500
2000
0.0
0.2
0.4
0.6
0.8
1.0
Raw intensity

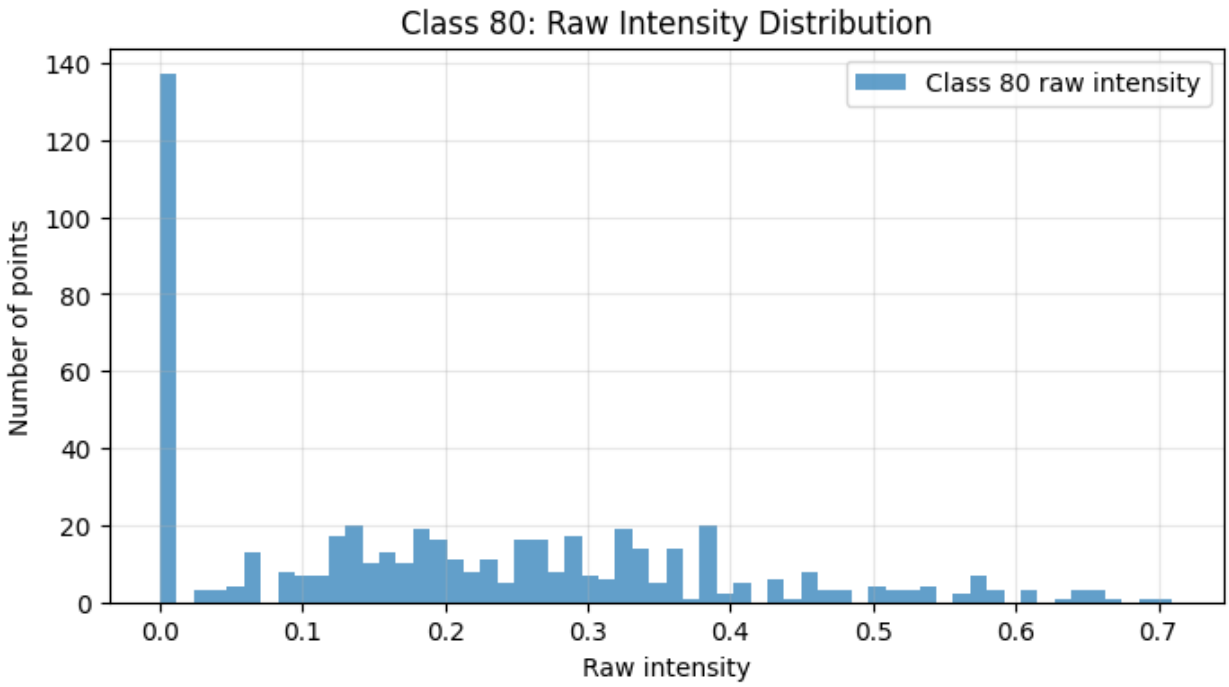

Class 70: Pseudo-Reflectivity Distribution
Class 70 pseudo-reflectivity (I*R)
Number of points
0
250
500
750
1000
1250
1500
1750
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Pseudo-reflectivity (I * R)

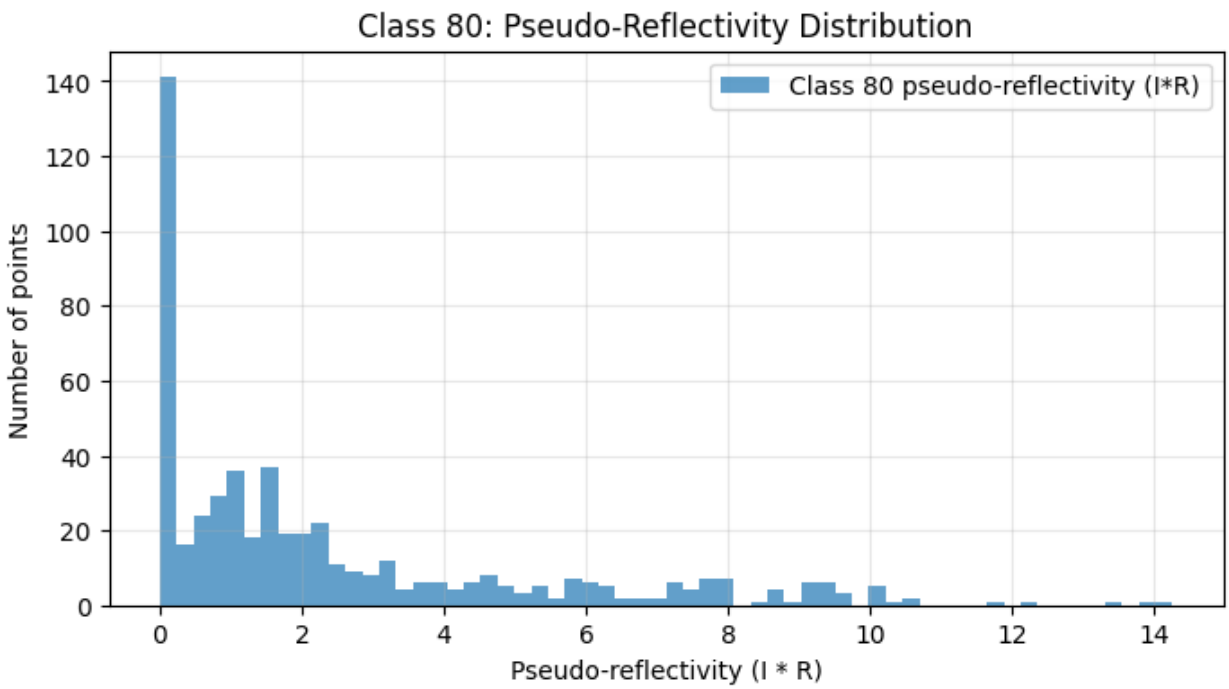

Class 71: Raw Intensity Distribution
Class 71 raw intensity
Number of points
Raw intensity
0
10
20
30
40
50
60
70
80
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7

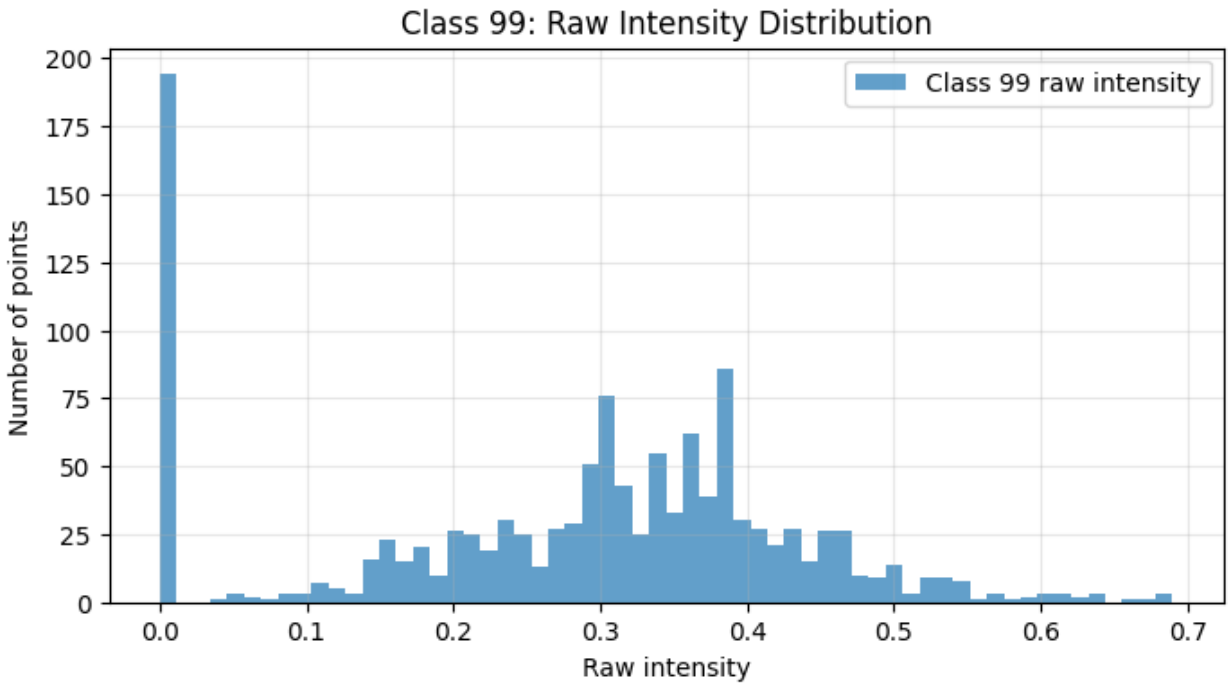

Class 71: Pseudo-Reflectivity Distribution
Class 71 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
10
20
30
40
50
60
70
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

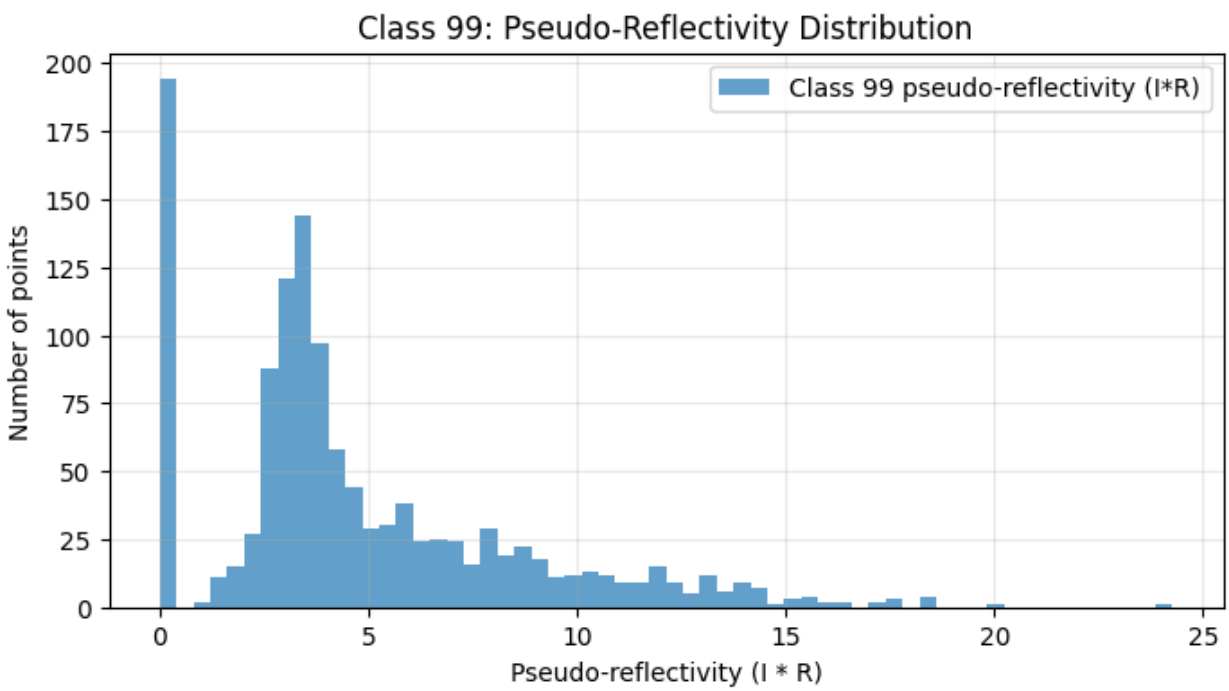

Class 72: Raw Intensity Distribution
Class 72 raw intensity
Number of points
Raw intensity
0
50
100
150
200
250
0.0
0.1
0.2
0.3
0.4
0.5
0.6


Class 72: Pseudo-Reflectivity Distribution
Class 72 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
25
50
75
100
125
150
175
200
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Class 80: Raw Intensity Distribution
Class 80 raw intensity
Number of points
Raw intensity
0
20
40
60
80
100
120
140
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7


Class 80: Pseudo-Reflectivity Distribution
Class 80 pseudo-reflectivity (I*R)
Number of points
Pseudo-reflectivity (I * R)
0
20
40
60
80
100
120
140
0
2
4
6
8
10
12
14





Build FOV projections and summarize them with projection-space metrics

In this code, we project the LiDAR frame into a simple angular field-of-view image.

This gives us sensor-view visualizations of:

- range,
- raw intensity,
- pseudo-reflectivity $I \cdot R$,
- semantic labels.

Along with the images, we also compute projection-space metrics such as:

- pixel coverage,
- collision count,
- valid-pixel statistics.

This lets us check whether the projected signals remain structurally meaningful in sensor-view space, not just in bird's-eye view or in 1D histograms.

The goal here is not to build a production LiDAR projection model. It is to create a compact, interpretable FOV representation that matches the preliminary single-frame analysis.

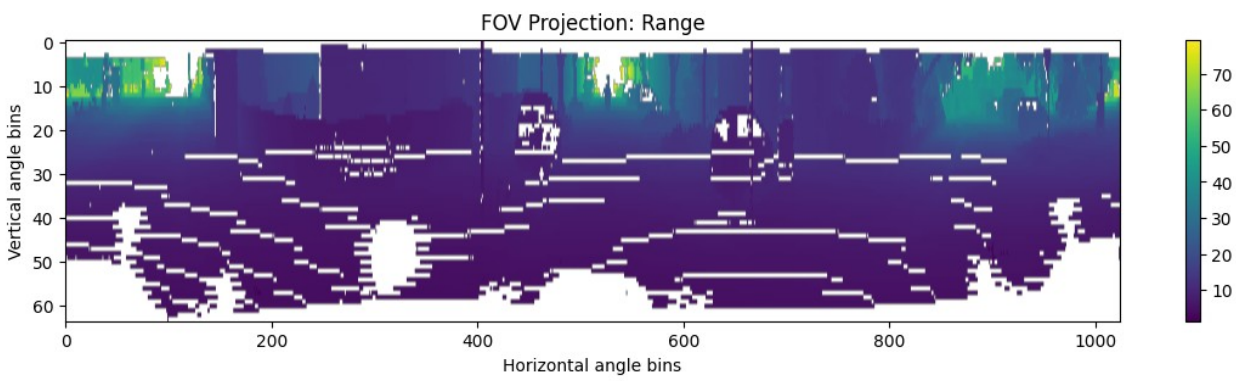
```
import numpy as np
import matplotlib.pyplot as plt

# 1) Spherical projection
x = xyz[:, 0]
y = xyz[:, 1]
z = xyz[:, 2]

r = np.linalg.norm(xyz, axis=1) + 1e-6
yaw = np.arctan2(y, x)
pitch = np.arcsin(z / r)

W = 1024
H = 64

yaw_min, yaw_max = -np.pi, np.pi
pitch_min, pitch_max = pitch.min(), pitch.max()

u = ((yaw - yaw_min) / (yaw_max - yaw_min) * (W - 1)).astype(np.int32)
v = ((pitch_max - pitch) / (pitch_max - pitch_min) * (H -
1)).astype(np.int32)

u = np.clip(u, 0, W - 1)
v = np.clip(v, 0, H - 1)

# 2) Projection buffers
range_img = np.full((H, W), np.nan, dtype=np.float32)
intensity_img = np.full((H, W), np.nan, dtype=np.float32)
pseudo_r1_img = np.full((H, W), np.nan, dtype=np.float32)
semantic_img = np.full((H, W), np.nan, dtype=np.float32)

filled_mask = np.zeros((H, W), dtype=bool)
collision_count = 0
```

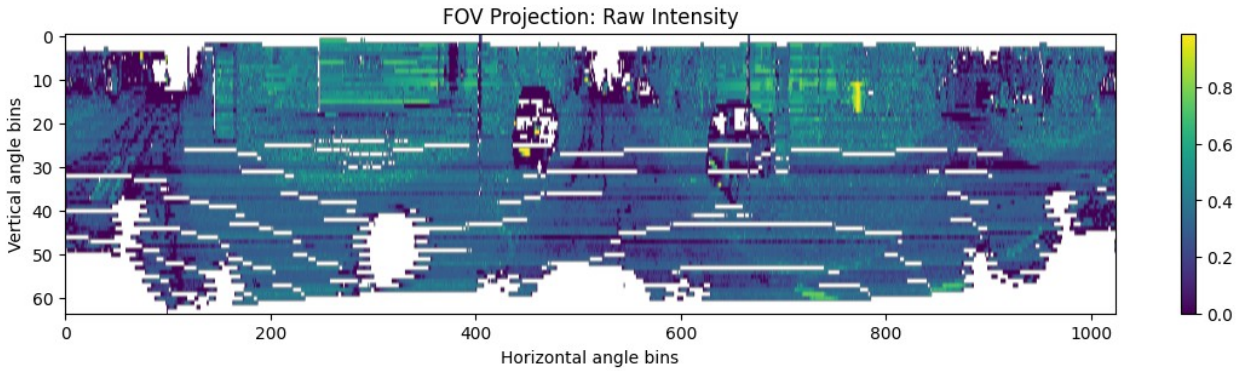
```python
# Far-to-near so nearer points overwrite farther ones
order = np.argsort(r)[::-1]

for idx in order:
    row, col = v[idx], u[idx]
    if filled_mask[row, col]:
        collision_count += 1
    filled_mask[row, col] = True

    range_img[row, col] = r[idx]
    intensity_img[row, col] = intensity[idx]
    pseudo_r1_img[row, col] = pseudo_reflectivity_r1[idx]
    semantic_img[row, col] = semantic_labels[idx]

# 3) Projection-space metrics
total_pixels = H * W
valid_pixels = int(np.sum(filled_mask))
coverage_ratio = valid_pixels / total_pixels
points_projected = len(r)
collisions_ratio = collision_count / points_projected

def valid_stats(name, img):
    vals = img[~np.isnan(img)]
    print(name)
    print(f"  valid pixels : {len(vals)}")
    print(f"  min          : {vals.min():.6f}")
    print(f"  p25          : {np.percentile(vals, 25):.6f}")
    print(f"  p50          : {np.percentile(vals, 50):.6f}")
    print(f"  p75          : {np.percentile(vals, 75):.6f}")
    print(f"  p95          : {np.percentile(vals, 95):.6f}")
    print(f"  max          : {vals.max():.6f}")
    print(f"  mean         : {vals.mean():.6f}")
    print(f"  std          : {vals.std():.6f}")
    print()

projected_classes =
np.unique(semantic_img[~np.isnan(semantic_img)]).astype(np.int32)

print("Projection-space metrics")
print(f"  Image size                : {H} x {W} = {total_pixels}
pixels")
print(f"  Total input points        : {points_projected}")
print(f"  Valid projected pixels    : {valid_pixels}")
print(f"  Pixel coverage ratio      : {coverage_ratio:.6f}")
print(f"  Pixel collision count     : {collision_count}")
print(f"  Pixel collision ratio     : {collisions_ratio:.6f}")
print(f"  Unique projected classes  : {len(projected_classes)}")
print(f"  Projected class IDs       : {projected_classes.tolist()}")
print()
```

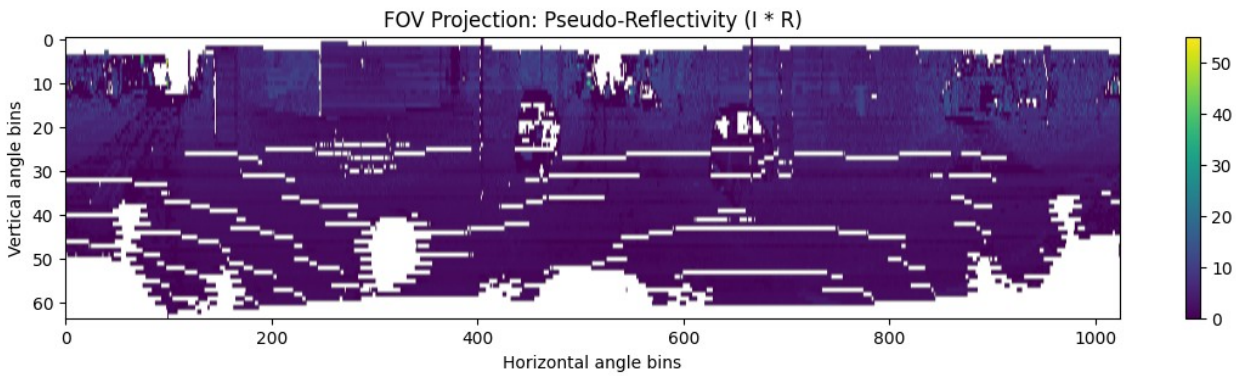
```
valid_stats("Projected range image stats", range_img)
valid_stats("Projected raw intensity image stats", intensity_img)
valid_stats("Projected pseudo-reflectivity (I * R) image stats",
pseudo_r1_img)

# 4) Visualizations
plt.figure(figsize=(14, 3))
plt.imshow(range_img, aspect="auto")
plt.title("FOV Projection: Range")
plt.xlabel("Horizontal angle bins")
plt.ylabel("Vertical angle bins")
plt.colorbar()
plt.show()

plt.figure(figsize=(14, 3))
plt.imshow(intensity_img, aspect="auto")
plt.title("FOV Projection: Raw Intensity")
plt.xlabel("Horizontal angle bins")
plt.ylabel("Vertical angle bins")
plt.colorbar()
plt.show()

plt.figure(figsize=(14, 3))
plt.imshow(pseudo_r1_img, aspect="auto")
plt.title("FOV Projection: Pseudo-Reflectivity (I * R)")
plt.xlabel("Horizontal angle bins")
plt.ylabel("Vertical angle bins")
plt.colorbar()
plt.show()

plt.figure(figsize=(14, 3))
plt.imshow(semantic_img, aspect="auto")
plt.title("FOV Projection: Semantic Labels")
plt.xlabel("Horizontal angle bins")
plt.ylabel("Vertical angle bins")
plt.colorbar()
plt.show()
```

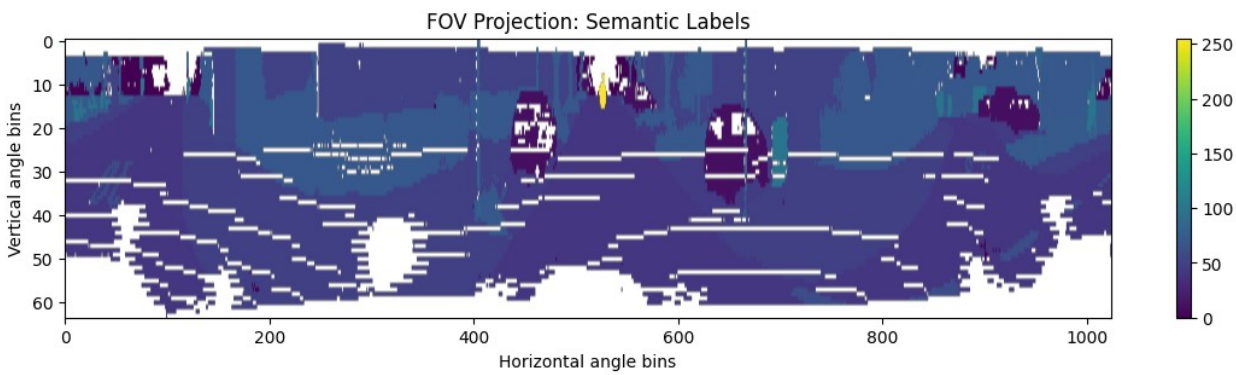
```
Projection-space metrics
  Image size                : 64 x 1024 = 65536 pixels
  Total input points        : 124668
  Valid projected pixels    : 50188
  Pixel coverage ratio      : 0.765808
  Pixel collision count     : 74480
  Pixel collision ratio     : 0.597427
  Unique projected classes  : 17
  Projected class IDs       : [0, 1, 10, 40, 44, 48, 50, 51, 52, 60,
70, 71, 72, 80, 81, 99, 255]

Projected range image stats
```

```
  valid pixels : 50188
  min          : 1.348360
  p25          : 6.416636
  p50          : 9.149655
  p75          : 14.091101
  p95          : 34.554153
  max          : 79.479065
  mean         : 12.547100
  std          : 9.983994

Projected raw intensity image stats
  valid pixels : 50188
  min          : 0.000000
  p25          : 0.210000
  p50          : 0.300000
  p75          : 0.360000
  p95          : 0.520000
  max          : 0.990000
  mean         : 0.288925
  std          : 0.146353

Projected pseudo-reflectivity (I * R) image stats
  valid pixels : 50188
  min          : 0.000000
  p25          : 1.572921
  p50          : 2.455885
  p75          : 4.683060
  p95          : 8.881825
  max          : 55.162960
  mean         : 3.357735
  std          : 2.867177
```









Summarizing the main quantitative findings of the single-frame reflectivity recap

In this code, we gather the most important single-frame results into one compact summary.

This gives a clean final view of how raw intensity and the main pseudo-reflectivity proxy behaved in terms of:

- range dependence,
- signal distribution,
- semantic separability,
- and FOV projection quality.

The purpose of this code is to lock the main takeaway of the single-frame reflectivity section before we move on to the short-window motion recap.

```
def signal_summary_dict(name, values):
    return {
        "name": name,
        "min": float(np.min(values)),
        "p25": float(np.percentile(values, 25)),
        "p50": float(np.percentile(values, 50)),
        "p75": float(np.percentile(values, 75)),
        "p95": float(np.percentile(values, 95)),
        "mean": float(np.mean(values)),
        "std": float(np.std(values)),
    }

summary_intensity = signal_summary_dict("Raw intensity", intensity)
summary_r1 = signal_summary_dict("Pseudo-reflectivity (I * R)",
pseudo_reflectivity_r1)

print("SINGLE-FRAME REFLECTIVITY RECAP: FINAL QUANTITATIVE SUMMARY")

print("\n[Signal Distribution Summary]")
for summary in [summary_intensity, summary_r1]:
    print(f"{summary['name']}")
    print(f"  min   : {summary['min']:.6f}")
    print(f"  p25   : {summary['p25']:.6f}")
    print(f"  p50   : {summary['p50']:.6f}")
    print(f"  p75   : {summary['p75']:.6f}")
    print(f"  p95   : {summary['p95']:.6f}")
    print(f"  mean  : {summary['mean']:.6f}")
    print(f"  std   : {summary['std']:.6f}")
    print()

print("[Range Dependence]")
print(f"corr(raw intensity, range)           =
{corr_intensity_range:.6f}")
print(f"corr(pseudo-reflectivity I*R, range) = {corr_r1_range:.6f}")
print(f"corr(pseudo-reflectivity I*R^2, range) = {corr_r2_range:.6f}")
print()

print("[Semantic Separability]")
print(f"raw intensity score                  = {score_intensity:.8f}")
print(f"pseudo-reflectivity I*R score        = {score_r1:.8f}")
print(f"absolute gain                        = {score_r1 -
score_intensity:.8f}")
print(f"relative gain                        = {(score_r1 -
score_intensity) / (score_intensity + 1e-12):.8f}")
print()

print("[Projection-Space Structure]")
```

```
print(f"valid projected pixels                = {valid_pixels}")
print(f"pixel coverage ratio                  = {coverage_ratio:.6f}")
print(f"pixel collision count                 = {collision_count}")
print(f"pixel collision ratio                 =
{collisions_ratio:.6f}")
print(f"unique projected classes              =
{len(projected_classes)}")
```

```
SINGLE-FRAME REFLECTIVITY RECAP: FINAL QUANTITATIVE SUMMARY

[Signal Distribution Summary]
Raw intensity
  min    : 0.000000
  p25    : 0.220000
  p50    : 0.300000
  p75    : 0.370000
  p95    : 0.540000
  mean   : 0.294134
  std    : 0.147690

Pseudo-reflectivity (I * R)
  min    : 0.000000
  p25    : 1.669057
  p50    : 2.754553
  p75    : 5.201154
  p95    : 9.184167
  mean   : 3.628692
  std    : 2.940125

[Range Dependence]
corr(raw intensity, range)           = -0.231389
corr(pseudo-reflectivity I*R, range) = 0.398455
corr(pseudo-reflectivity I*R^2, range) = 0.568631

[Semantic Separability]
raw intensity score                  = 0.38142828
pseudo-reflectivity I*R score        = 0.57723763
absolute gain                        = 0.19580935
relative gain                        = 0.51335823

[Projection-Space Structure]
valid projected pixels               = 50188
pixel coverage ratio                 = 0.765808
pixel collision count                = 74480
pixel collision ratio                = 0.597427
unique projected classes             = 17
```

Locate the sequence and define the short motion window

In this code, we begin the multi-frame motion recap section of this notebook.

The single-frame analysis showed that a simple range-aware pseudo-reflectivity proxy can behave differently from raw intensity and can become more semantically informative on one frame.

Now we move from a still frame to a short motion sequence.

The purpose of this code is only to define the short contiguous frame window that will be used for the motion recap.

We are not loading all point clouds yet.

Right now, we only:

- confirm the sequence structure again in the motion-analysis context,
- list the available frame IDs,
- choose a short contiguous motion window,
- and verify that all matching label files exist.

```
from pathlib import Path

VELODYNE_DIR = DATASET_ROOT / "velodyne"
LABELS_DIR = DATASET_ROOT / "labels"

expected_items = ["velodyne", "labels", "calib.txt", "poses.txt",
"times.txt"]

print("Dataset root:", DATASET_ROOT.resolve())
print("Exists:", DATASET_ROOT.exists())
print()

for item in expected_items:
    item_path = DATASET_ROOT / item
    print(f"{item:10} -> exists: {item_path.exists()} | path:
{item_path}")

print()

if VELODYNE_DIR.exists():
    frame_ids = sorted(p.stem for p in VELODYNE_DIR.glob("*.bin"))
    print("Total .bin frames found:", len(frame_ids))
    print("First 10 frame IDs:", frame_ids[:10])
    print("Last 10 frame IDs :", frame_ids[-10:])
else:
    frame_ids = []
    print("Velodyne directory not found.")

print()

# Short motion window for the recap
START_IDX = 0
NUM_FRAMES = 30
```

```python
selected_frame_ids = frame_ids[START_IDX:START_IDX + NUM_FRAMES]

print("Selected motion window")
print(" - start index :", START_IDX)
print(" - num frames  :", NUM_FRAMES)
print(" - first frame :", selected_frame_ids[0] if selected_frame_ids
else None)
print(" - last frame  :", selected_frame_ids[-1] if selected_frame_ids
else None)
print(" - valid window:", len(selected_frame_ids) == NUM_FRAMES)

if selected_frame_ids:
    all_labels_exist = all((LABELS_DIR / f"{fid}.label").exists() for
fid in selected_frame_ids)
    print(" - all matching label files exist:", all_labels_exist)
```

```
Dataset root:
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti/dataset
/sequences/00
Exists: True

velodyne   -> exists: True | path:
../data/semantickitti/dataset/sequences/00/velodyne
labels     -> exists: True | path:
../data/semantickitti/dataset/sequences/00/labels
calib.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/calib.txt
poses.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/poses.txt
times.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/times.txt

Total .bin frames found: 4541
First 10 frame IDs: ['000000', '000001', '000002', '000003', '000004',
'000005', '000006', '000007', '000008', '000009']
Last 10 frame IDs : ['004531', '004532', '004533', '004534', '004535',
'004536', '004537', '004538', '004539', '004540']

Selected motion window
 - start index : 0
 - num frames  : 30
 - first frame : 000000
 - last frame  : 000029
 - valid window: True
 - all matching label files exist: True
```

Load the selected frames and compute the core per-point signals

In this code, we load the selected 30 LiDAR frames for the short motion recap.

For each frame, we extract:

- 3D coordinates (x, y, z),
- raw intensity I,
- geometric range $R = \sqrt{x^2 + y^2 + z^2}$,
- pseudo-reflectivity $\hat{\rho} = I \cdot R$.

This builds the basic multi-frame data structure that the rest of the motion recap will analyze.

We are still staying disciplined here: no heavy interpretation yet, no semantic metrics yet.

```
import numpy as np

def load_semantickitti_bin(bin_path):
    scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
    xyz = scan[:, :3]
    intensity = scan[:, 3]
    return xyz, intensity

multi_frame_data = []

for fid in selected_frame_ids:
    bin_path = VELODYNE_DIR / f"{fid}.bin"

    xyz, intensity = load_semantickitti_bin(bin_path)
    ranges = np.linalg.norm(xyz, axis=1)
    pseudo_reflectivity = intensity * ranges

    frame_dict = {
        "frame_id": fid,
        "xyz": xyz,
        "intensity": intensity,
        "ranges": ranges,
        "pseudo_reflectivity": pseudo_reflectivity,
        "num_points": xyz.shape[0],
    }
    multi_frame_data.append(frame_dict)

print("Frames loaded:", len(multi_frame_data))
print()

first_frame = multi_frame_data[0]
last_frame = multi_frame_data[-1]
```

```
print("First frame summary")
print(" - frame_id              :", first_frame["frame_id"])
print(" - xyz shape             :", first_frame["xyz"].shape)
print(" - intensity shape       :", first_frame["intensity"].shape)
print(" - ranges shape          :", first_frame["ranges"].shape)
print(" - pseudo-reflectivity   :",
first_frame["pseudo_reflectivity"].shape)
print(" - num_points            :", first_frame["num_points"])
print()

print("Last frame summary")
print(" - frame_id              :", last_frame["frame_id"])
print(" - xyz shape             :", last_frame["xyz"].shape)
print(" - intensity shape       :", last_frame["intensity"].shape)
print(" - ranges shape          :", last_frame["ranges"].shape)
print(" - pseudo-reflectivity   :",
last_frame["pseudo_reflectivity"].shape)
print(" - num_points            :", last_frame["num_points"])
print()

point_counts = [f["num_points"] for f in multi_frame_data]
print("Point count over selected window")
print(" - min :", np.min(point_counts))
print(" - max :", np.max(point_counts))
print(" - mean:", np.mean(point_counts))
```

```
Frames loaded: 30

First frame summary
 - frame_id              : 000000
 - xyz shape             : (124668, 3)
 - intensity shape       : (124668,)
 - ranges shape          : (124668,)
 - pseudo-reflectivity   : (124668,)
 - num_points            : 124668

Last frame summary
 - frame_id              : 000029
 - xyz shape             : (124018, 3)
 - intensity shape       : (124018,)
 - ranges shape          : (124018,)
 - pseudo-reflectivity   : (124018,)
 - num_points            : 124018

Point count over selected window
 - min : 120295
 - max : 124668
 - mean: 123242.7
```

Building frame-wise summary statistics

In this code, we compress each frame in the selected motion window into a small numerical summary.

For every frame, we compute descriptive statistics for:

- raw intensity,
- range,
- pseudo-reflectivity.

This gives us a clean temporal table that will let us inspect how the signal evolves across the 30-frame motion window.

```
import pandas as pd

def summarize_array(arr):
    return {
        "min": float(np.min(arr)),
        "p25": float(np.percentile(arr, 25)),
        "p50": float(np.percentile(arr, 50)),
        "p75": float(np.percentile(arr, 75)),
        "p95": float(np.percentile(arr, 95)),
        "mean": float(np.mean(arr)),
        "std": float(np.std(arr)),
    }

frame_summaries = []

for frame in multi_frame_data:
    intensity_stats = summarize_array(frame["intensity"])
    range_stats = summarize_array(frame["ranges"])
    pr_stats = summarize_array(frame["pseudo_reflectivity"])

    row = {
        "frame_id": frame["frame_id"],
        "num_points": frame["num_points"],

        "intensity_min": intensity_stats["min"],
        "intensity_p25": intensity_stats["p25"],
        "intensity_p50": intensity_stats["p50"],
        "intensity_p75": intensity_stats["p75"],
        "intensity_p95": intensity_stats["p95"],
        "intensity_mean": intensity_stats["mean"],
        "intensity_std": intensity_stats["std"],

        "range_min": range_stats["min"],
        "range_p25": range_stats["p25"],
        "range_p50": range_stats["p50"],
        "range_p75": range_stats["p75"],
```

```
        "range_p95": range_stats["p95"],
        "range_mean": range_stats["mean"],
        "range_std": range_stats["std"],

        "pr_min": pr_stats["min"],
        "pr_p25": pr_stats["p25"],
        "pr_p50": pr_stats["p50"],
        "pr_p75": pr_stats["p75"],
        "pr_p95": pr_stats["p95"],
        "pr_mean": pr_stats["mean"],
        "pr_std": pr_stats["std"],
    }
    frame_summaries.append(row)

summary_df = pd.DataFrame(frame_summaries)

print("Summary table shape:", summary_df.shape)
print()
print(summary_df[[
    "frame_id",
    "num_points",
    "intensity_mean",
    "intensity_std",
    "range_mean",
    "range_std",
    "pr_mean",
    "pr_std"
]].head(10))
```

```
Summary table shape: (30, 23)

  frame_id  num_points  intensity_mean  intensity_std  range_mean
range_std  \
0   000000      124668        0.294134       0.147690   13.565046
10.570971
1   000001      124605        0.294588       0.148049   13.511380
10.627299
2   000002      124478        0.294162       0.148084   13.401234
10.591676
3   000003      124167        0.295665       0.149636   13.268131
10.503467
4   000004      123969        0.295585       0.151231   13.188173
10.503502
5   000005      123924        0.294694       0.153250   13.134429
10.483358
6   000006      123373        0.294909       0.156234   13.098121
10.506295
7   000007      122765        0.296495       0.159902   13.089967
10.547448
8   000008      122123        0.298687       0.161086   13.058884
```

```
10.536805
9   000009      121357        0.300832       0.161600   13.099307
10.632101

    pr_mean    pr_std
0  3.628692  2.940125
1  3.606743  2.945034
2  3.578735  2.920066
3  3.573944  2.920962
4  3.554410  2.910439
5  3.548236  2.921168
6  3.547228  2.920425
7  3.569736  2.951197
8  3.585915  2.963047
9  3.608539  3.002258
```

Ploting temporal signal statistics across frame index

In this code, we visualize how the frame-wise mean values evolve over the 30-frame motion window.

We begin with the mean values of:

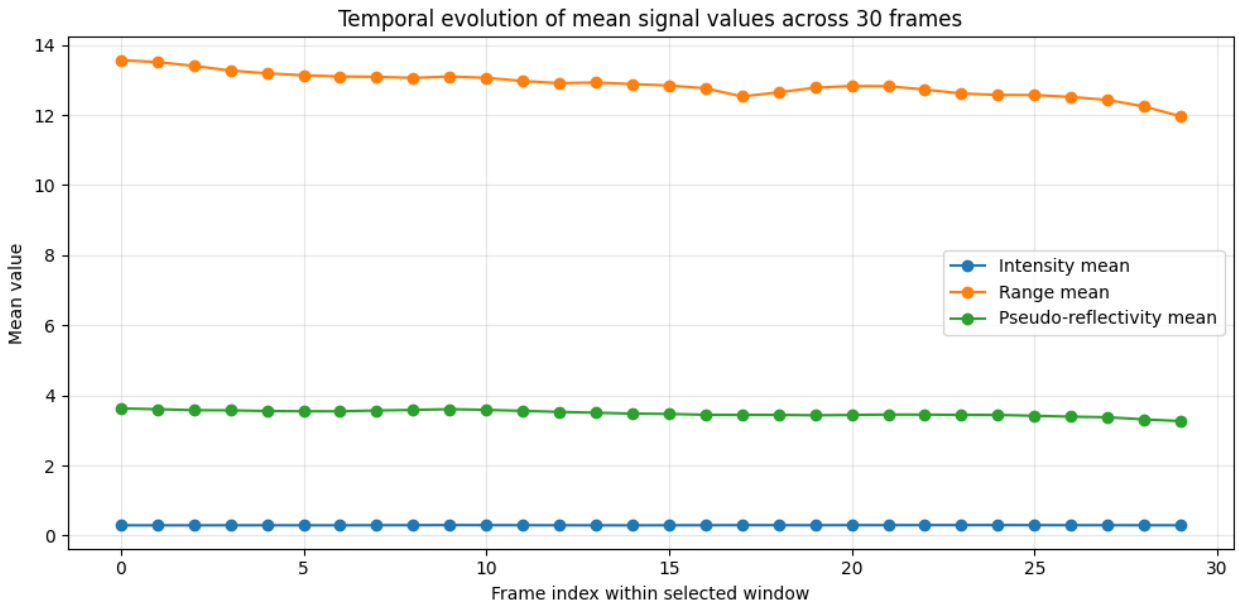
- raw intensity,
- range,
- pseudo-reflectivity.

This gives the first direct temporal view of whether the constructed signal remains stable or drifts noticeably as the platform moves.

Because these three signals live on different numeric scales, this plot is mainly for trend inspection, not for direct one-to-one visual comparison.

```
import matplotlib.pyplot as plt
import numpy as np

frame_idx = np.arange(len(summary_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, summary_df["intensity_mean"], marker="o",
label="Intensity mean")
plt.plot(frame_idx, summary_df["range_mean"], marker="o", label="Range
mean")
plt.plot(frame_idx, summary_df["pr_mean"], marker="o", label="Pseudo-
reflectivity mean")

plt.xlabel("Frame index within selected window")
plt.ylabel("Mean value")
plt.title("Temporal evolution of mean signal values across 30 frames")
plt.grid(True, alpha=0.3)
```

```
plt.legend()
plt.tight_layout()
plt.show()
```



Compare temporal trends on a normalized scale

We now normalize each mean curve by its value in the first frame.

The previous plot was useful for trend inspection, but direct visual comparison was difficult because the signals live on very different numeric scales.

By normalizing each curve relative to frame 0, we can compare how much each signal changes over the motion window in relative terms.

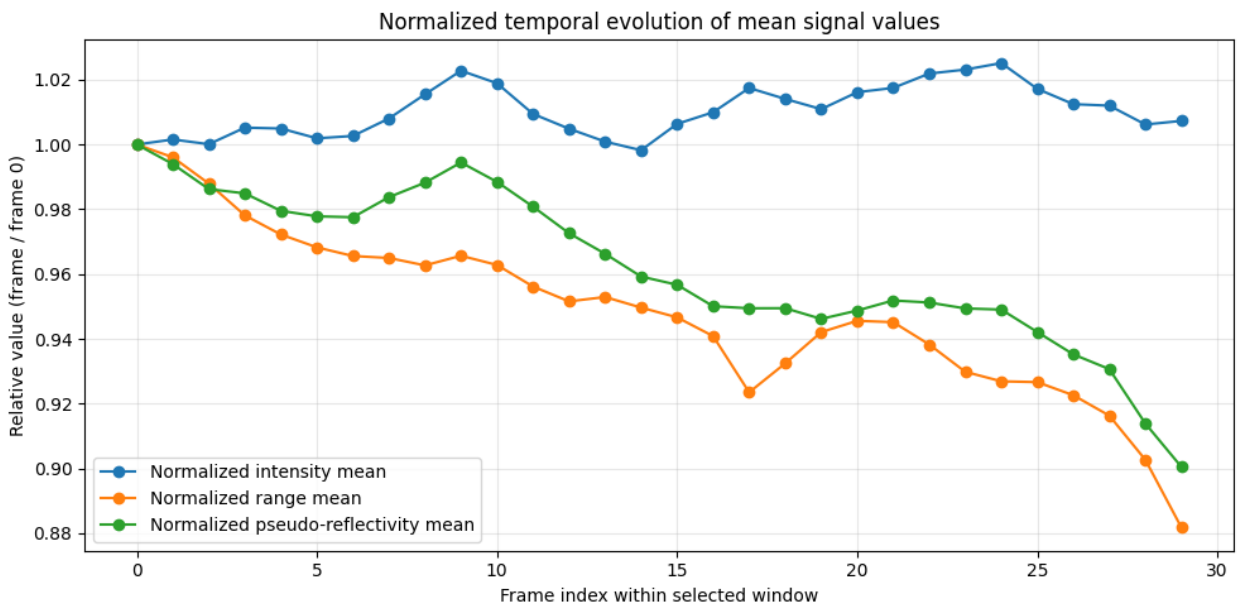
We compare:

- normalized raw intensity mean,
- normalized range mean,
- normalized pseudo-reflectivity mean.

This makes it easier to see whether pseudo-reflectivity changes more like raw intensity, more like range, or somewhere in between.

```
norm_intensity_mean = summary_df["intensity_mean"] /
summary_df["intensity_mean"].iloc[0]
norm_range_mean = summary_df["range_mean"] /
summary_df["range_mean"].iloc[0]
norm_pr_mean = summary_df["pr_mean"] / summary_df["pr_mean"].iloc[0]

frame_idx = np.arange(len(summary_df))
```

```
plt.figure(figsize=(10, 5))
plt.plot(frame_idx, norm_intensity_mean, marker="o", label="Normalized
intensity mean")
plt.plot(frame_idx, norm_range_mean, marker="o", label="Normalized
range mean")
plt.plot(frame_idx, norm_pr_mean, marker="o", label="Normalized
pseudo-reflectivity mean")

plt.xlabel("Frame index within selected window")
plt.ylabel("Relative value (frame / frame 0)")
plt.title("Normalized temporal evolution of mean signal values")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Plot temporal evolution of signal spread

Here we examine how the spread of the frame-wise signal distributions evolves across the 30-frame motion window.

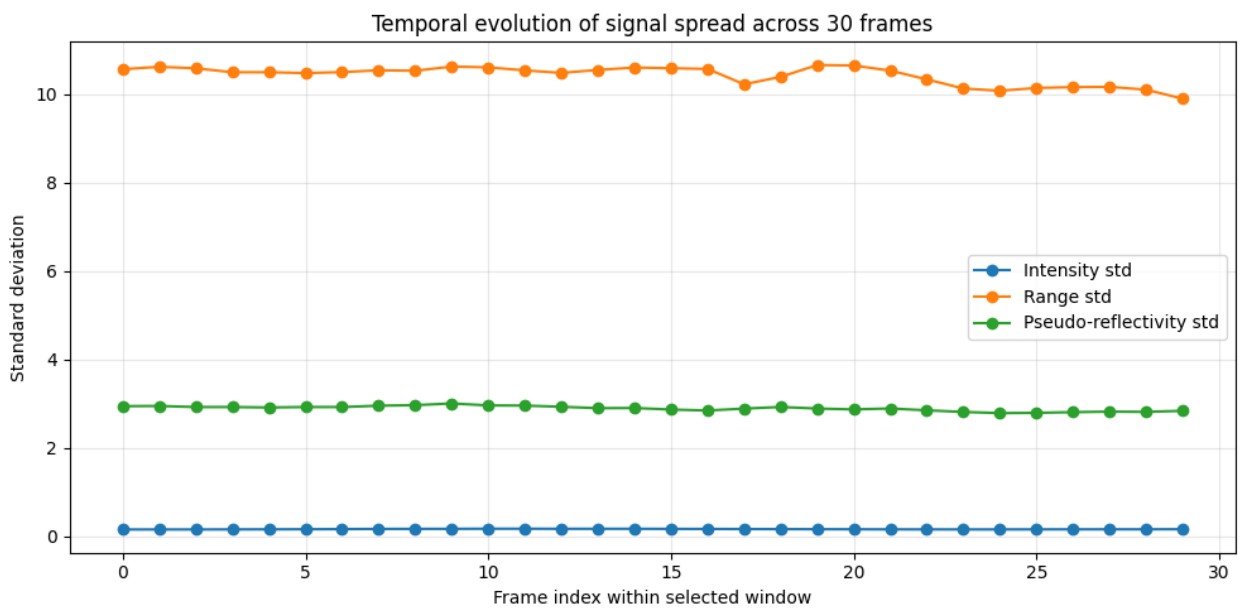
Mean values only describe the center of each distribution.

Here, we compare the temporal evolution of the standard deviation for:

- raw intensity,
- range,
- pseudo-reflectivity.

This helps us check whether the constructed pseudo-reflectivity signal remains numerically well-behaved through motion, or whether it becomes unstable as the platform moves.

```
frame_idx = np.arange(len(summary_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, summary_df["intensity_std"], marker="o",
label="Intensity std")
plt.plot(frame_idx, summary_df["range_std"], marker="o", label="Range
std")
plt.plot(frame_idx, summary_df["pr_std"], marker="o", label="Pseudo-
reflectivity std")

plt.xlabel("Frame index within selected window")
plt.ylabel("Standard deviation")
plt.title("Temporal evolution of signal spread across 30 frames")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Track temporal evolution of pseudo-reflectivity quantiles

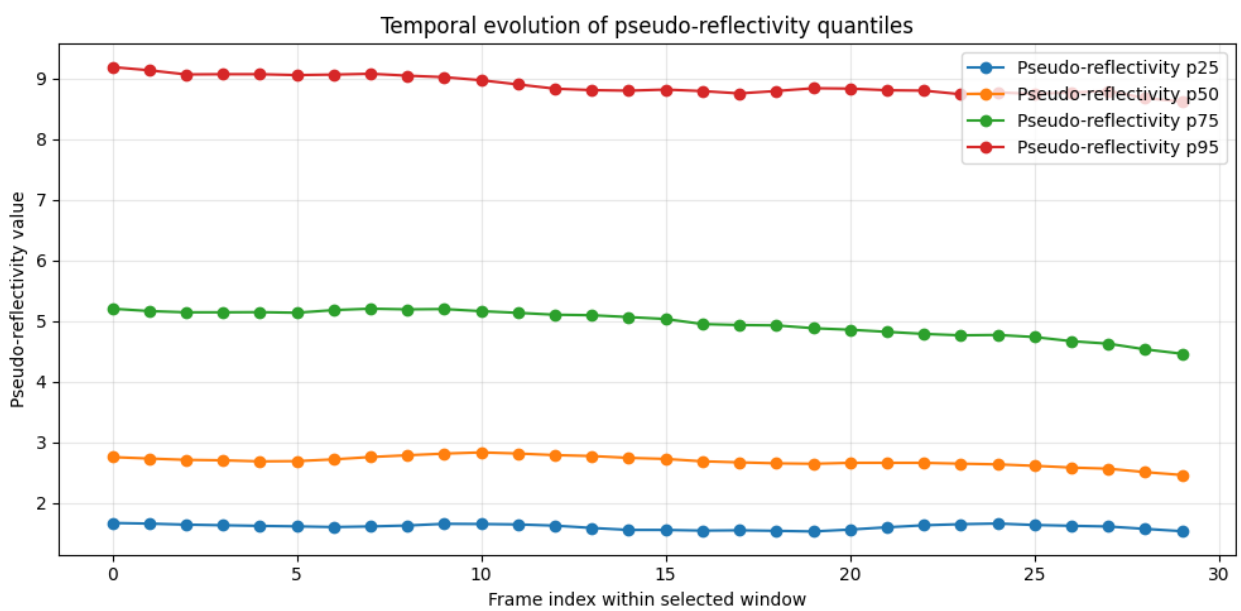
In this code, we look beyond the mean and standard deviation and track selected quantiles of the pseudo-reflectivity distribution across the 30-frame motion window.

This helps us see whether the distribution shifts uniformly or changes differently across:

- lower-value regions,
- the middle of the distribution,
- stronger pseudo-reflectivity responses.

We focus on pseudo-reflectivity here because the goal is to understand whether its temporal change is merely random noise or a more structured redistribution of the signal as the scene evolves through motion.

```
frame_idx = np.arange(len(summary_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, summary_df["pr_p25"], marker="o", label="Pseudo-
reflectivity p25")
plt.plot(frame_idx, summary_df["pr_p50"], marker="o", label="Pseudo-
reflectivity p50")
plt.plot(frame_idx, summary_df["pr_p75"], marker="o", label="Pseudo-
reflectivity p75")
plt.plot(frame_idx, summary_df["pr_p95"], marker="o", label="Pseudo-
reflectivity p95")

plt.xlabel("Frame index within selected window")
plt.ylabel("Pseudo-reflectivity value")
plt.title("Temporal evolution of pseudo-reflectivity quantiles")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Measure frame-wise range dependence across motion

We now examine a more diagnostic temporal quantity: the correlation between signal value and range in each frame.

So far, we have studied temporal statistics of the signal values themselves.

Now we measure, frame by frame:

- correlation between raw intensity and range,
- correlation between pseudo-reflectivity $I \cdot R$ and range.

This helps us test whether the reflectivity-style proxy remains more or less range-entangled as the platform moves through the selected motion window.

The goal is not to prove geometry invariance. The goal is to characterize how stable the signal-range relationship remains across time.

```
framewise_corr_rows = []

for frame in multi_frame_data:
    intensity_vals = frame["intensity"]
    range_vals = frame["ranges"]
    pr_vals = frame["pseudo_reflectivity"]

    corr_intensity_range = np.corrcoef(intensity_vals, range_vals)[0,
1]
    corr_pr_range = np.corrcoef(pr_vals, range_vals)[0, 1]

    framewise_corr_rows.append({
        "frame_id": frame["frame_id"],
        "corr_intensity_range": float(corr_intensity_range),
        "corr_pr_range": float(corr_pr_range),
    })

corr_df = pd.DataFrame(framewise_corr_rows)

print(corr_df.head(10))
print()
print("Correlation summary over 30 frames")
print(" - raw intensity vs range")
print("   mean:", corr_df["corr_intensity_range"].mean())
print("   min :", corr_df["corr_intensity_range"].min())
print("   max :", corr_df["corr_intensity_range"].max())
print()
print(" - pseudo-reflectivity vs range")
print("   mean:", corr_df["corr_pr_range"].mean())
print("   min :", corr_df["corr_pr_range"].min())
print("   max :", corr_df["corr_pr_range"].max())
```

```
  frame_id  corr_intensity_range  corr_pr_range
0   000000             -0.231389       0.398455
1   000001             -0.237417       0.390665
2   000002             -0.231691       0.392047
3   000003             -0.222040       0.402657
4   000004             -0.216446       0.399708
5   000005             -0.200680       0.404770
6   000006             -0.192225       0.391858
```

```
7   000007              -0.184621       0.383704
8   000008              -0.185355       0.375663
9   000009              -0.193318       0.368467

Correlation summary over 30 frames
 - raw intensity vs range
   mean: -0.2065390528423717
   min : -0.241687721923002
   max : -0.16775861263260272

 - pseudo-reflectivity vs range
   mean: 0.3646205896583482
   min : 0.34222221241736783
   max : 0.4047703050908886
```

Visualize frame-wise range dependence across motion

We will be now plotting the frame-wise signal-range correlations over time.

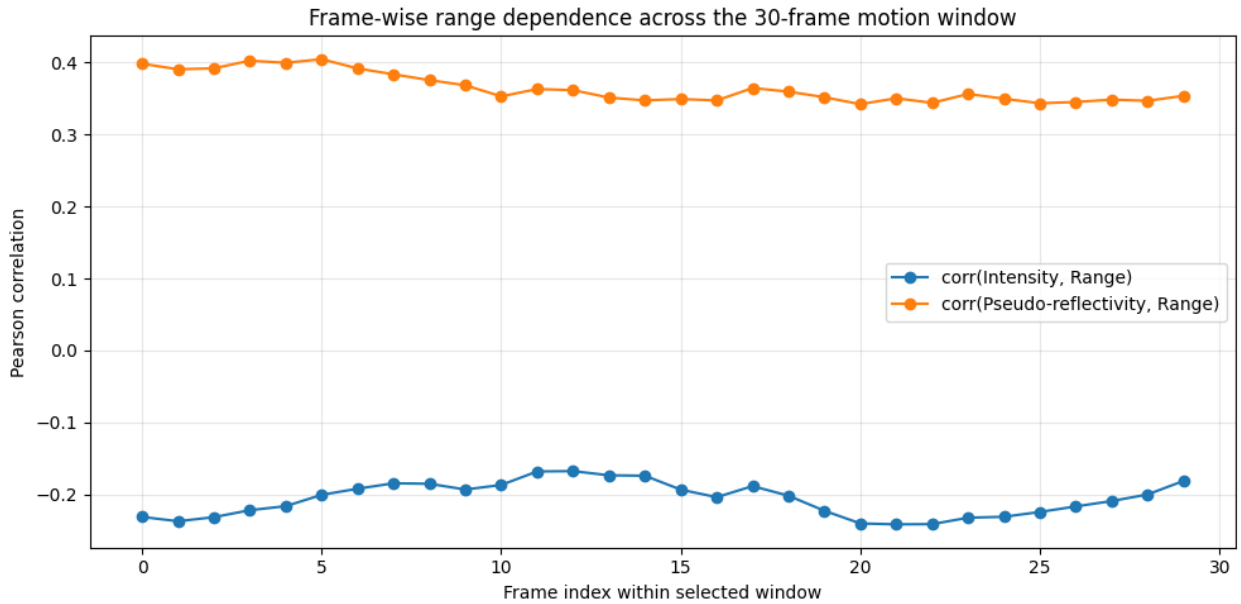
This lets us inspect whether the range dependence of:

- raw intensity,
- pseudo-reflectivity $I \cdot R$,

remains stable or changes meaningfully across the selected motion window.

The goal is to make the previous numerical summary easier to read as a temporal pattern. If both curves remain fairly smooth, that supports the claim that the signal-range relationship is structured and temporally stable rather than erratic.

```
frame_idx = np.arange(len(corr_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, corr_df["corr_intensity_range"], marker="o",
label="corr(Intensity, Range)")
plt.plot(frame_idx, corr_df["corr_pr_range"], marker="o",
label="corr(Pseudo-reflectivity, Range)")

plt.xlabel("Frame index within selected window")
plt.ylabel("Pearson correlation")
plt.title("Frame-wise range dependence across the 30-frame motion
window")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Recompute temporal semantic separability across the motion window

In this code, we bring semantics back into the motion recap.

So far, the multi-frame analysis has shown that the raw signal statistics and the range dependence of the proxy remain structured across the 30-frame window.

Now we ask the more important question:

does pseudo-reflectivity remain more semantically discriminative than raw intensity as the scene moves through time?

To keep this comparison methodologically consistent with the earlier single-frame analysis, we reuse the same Fisher-style class-separability setup and the same class-filtering rule across frames.

For each frame in the selected motion window, we will:

- load the semantic labels,
- keep only sufficiently supported semantic classes,
- compute the separability score for raw intensity,
- compute the separability score for pseudo-reflectivity $I \cdot R$,
- and record the frame-wise gain.

This gives us a direct temporal-semantic comparison across the full motion window.

```
def load_semantickitti_labels(label_path):
    raw = np.fromfile(label_path, dtype=np.uint32)
    semantic = raw & 0xFFFF
    return semantic
```

```python
def get_valid_classes_notebook02_style(labels,
min_points_per_class=500, ignored_classes=(0, 1, 255)):
    valid_classes = []
    unique_classes, counts = np.unique(labels, return_counts=True)

    for cls, count in zip(unique_classes, counts):
        if cls in ignored_classes:
            continue
        if count >= min_points_per_class:
            valid_classes.append(int(cls))

    return sorted(valid_classes)

def fisher_separability(values, labels, valid_classes):
    class_means = []
    class_vars = []
    class_ns = []

    for cls in valid_classes:
        cls_values = values[labels == cls]
        class_means.append(np.mean(cls_values))
        class_vars.append(np.var(cls_values))
        class_ns.append(len(cls_values))

    class_means = np.array(class_means, dtype=np.float64)
    class_vars = np.array(class_vars, dtype=np.float64)
    class_ns = np.array(class_ns, dtype=np.float64)

    global_mean = np.sum(class_ns * class_means) / np.sum(class_ns)
    between = np.sum(class_ns * (class_means - global_mean) ** 2)
    within = np.sum(class_ns * class_vars)

    score = between / (within + 1e-12)
    return float(score), class_means, class_vars, class_ns

semantic_rows = []

for frame in multi_frame_data:
    fid = frame["frame_id"]
    label_path = LABELS_DIR / f"{fid}.label"
    semantic_labels_frame = load_semantickitti_labels(label_path)

    valid_classes_frame = get_valid_classes_notebook02_style(
        semantic_labels_frame,
        min_points_per_class=500,
        ignored_classes=(0, 1, 255)
    )

    score_intensity_frame, _, _, ns = fisher_separability(
```

```
        frame["intensity"],
        semantic_labels_frame,
        valid_classes_frame
    )

    score_pr_frame, _, _, _ = fisher_separability(
        frame["pseudo_reflectivity"],
        semantic_labels_frame,
        valid_classes_frame
    )

    semantic_rows.append({
        "frame_id": fid,
        "num_valid_classes": len(valid_classes_frame),
        "valid_classes": valid_classes_frame,
        "intensity_sep_score": score_intensity_frame,
        "pr_sep_score": score_pr_frame,
        "absolute_gain": score_pr_frame - score_intensity_frame,
        "relative_gain": (score_pr_frame - score_intensity_frame) /
(score_intensity_frame + 1e-12),
    })

semantic_df = pd.DataFrame(semantic_rows)

print("Frame 000000 sanity check")
row0 = semantic_df.iloc[0]
print(" - valid classes       :", row0["valid_classes"])
print(" - num valid classes   :", row0["num_valid_classes"])
print(" - intensity score     :",
f'{row0["intensity_sep_score"]:.6f}')
print(" - pseudo-reflectivity :", f'{row0["pr_sep_score"]:.6f}')
print(" - absolute gain       :", f'{row0["absolute_gain"]:.6f}')
print(" - relative gain       :", f'{row0["relative_gain"]:.6f}')
print()

print("First 10 frame-wise results")
print(semantic_df[[
    "frame_id",
    "num_valid_classes",
    "intensity_sep_score",
    "pr_sep_score",
    "absolute_gain",
    "relative_gain"
]].head(10))
print()

print("30-frame semantic separability summary")
print(" - intensity score mean        :",
semantic_df["intensity_sep_score"].mean())
print(" - pseudo-reflectivity mean    :",
```

```
semantic_df["pr_sep_score"].mean())
print(" - absolute gain mean          :",
semantic_df["absolute_gain"].mean())
print(" - relative gain mean          :",
semantic_df["relative_gain"].mean())
```

```
Frame 000000 sanity check
 - valid classes       : [10, 40, 44, 48, 50, 52, 60, 70, 71, 72, 80,
99]
 - num valid classes   : 12
 - intensity score     : 0.381428
 - pseudo-reflectivity : 0.577238
 - absolute gain       : 0.195809
 - relative gain       : 0.513358

First 10 frame-wise results
  frame_id  num_valid_classes  intensity_sep_score  pr_sep_score  \
0   000000                 12             0.381428      0.577238
1   000001                 12             0.387196      0.562421
2   000002                 12             0.401164      0.535149
3   000003                 12             0.419656      0.520715
4   000004                 12             0.423778      0.505296
5   000005                 13             0.454864      0.512665
6   000006                 13             0.470120      0.514486
7   000007                 12             0.492163      0.513640
8   000008                 12             0.523754      0.498392
9   000009                 12             0.547563      0.495476

   absolute_gain  relative_gain
0       0.195809       0.513358
1       0.175224       0.452547
2       0.133986       0.333993
3       0.101058       0.240812
4       0.081518       0.192361
5       0.057801       0.127073
6       0.044366       0.094372
7       0.021477       0.043638
8      -0.025361      -0.048422
9      -0.052087      -0.095125

30-frame semantic separability summary
 - intensity score mean        : 0.4504474749539853
 - pseudo-reflectivity mean    : 0.4752031096261892
 - absolute gain mean          : 0.024755634672203822
 - relative gain mean          : 0.09162314535748868
```

Visualize temporal semantic separability across motion

In this part, we plot the frame-wise semantic separability scores across the 30-frame motion window.

This shows how the semantic discriminative strength of:

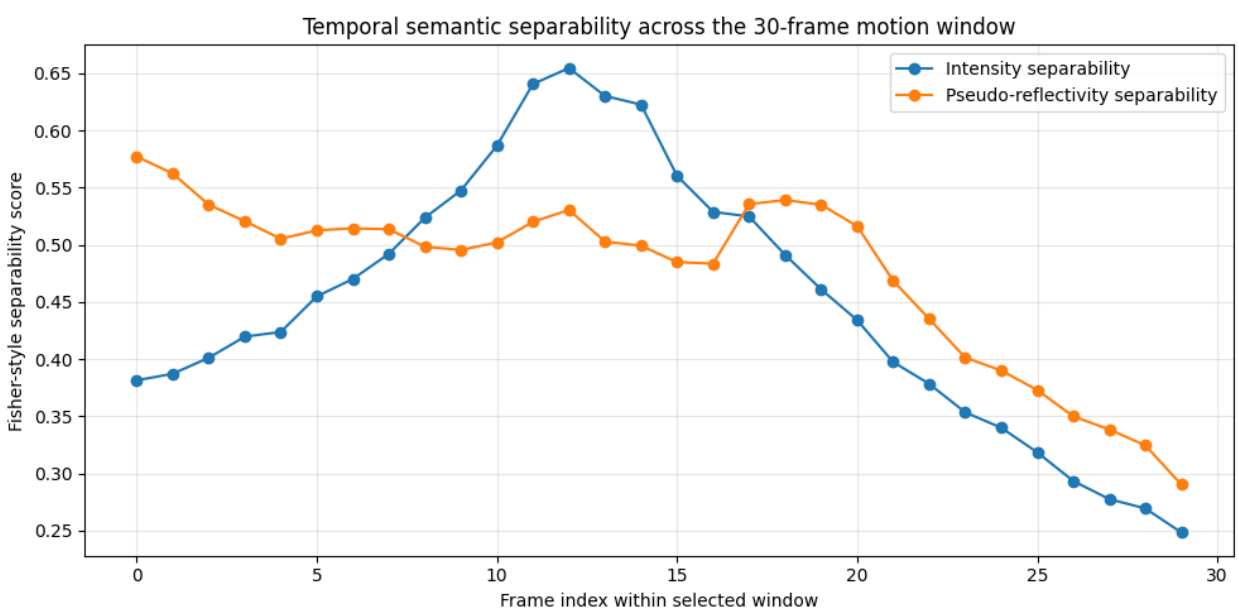
- raw intensity,
- pseudo-reflectivity $I \cdot R$,

changes through motion when evaluated with the same Fisher-style metric used in the single-frame recap.

The goal is to make the temporal comparison visually explicit.

If the two curves cross, that is not a problem. It is part of the actual result: the usefulness of pseudo-reflectivity is scene-dependent over time rather than uniformly dominant.

```
frame_idx = np.arange(len(semantic_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, semantic_df["intensity_sep_score"], marker="o",
label="Intensity separability")
plt.plot(frame_idx, semantic_df["pr_sep_score"], marker="o",
label="Pseudo-reflectivity separability")

plt.xlabel("Frame index within selected window")
plt.ylabel("Fisher-style separability score")
plt.title("Temporal semantic separability across the 30-frame motion
window")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Plotting the separability gain of pseudo-reflectivity over raw intensity

Here, we make the temporal comparison more explicit by plotting the frame-wise gain:

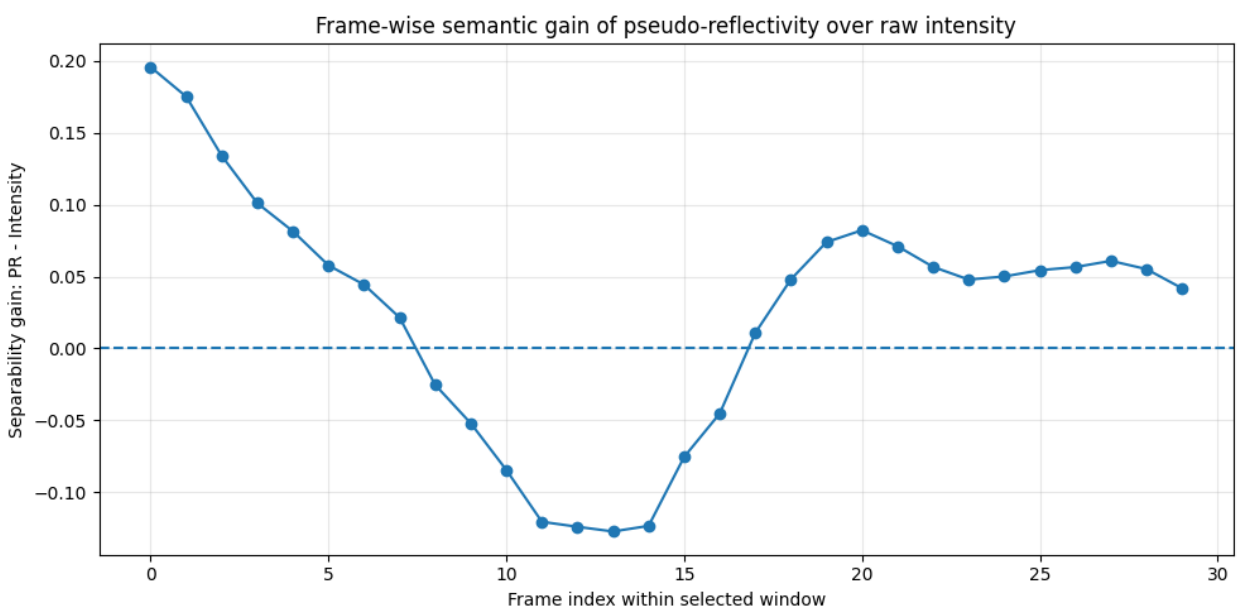
$$\Delta_{\text{sep}} = \text{Sep}(I \cdot R) - \text{Sep}(I)$$

Positive values mean pseudo-reflectivity is more semantically separable than raw intensity in that frame.

Negative values mean raw intensity is more semantically separable.

This plot is useful because it strips away the two separate curves and shows the relative advantage directly as a single temporal signal.

```
frame_idx = np.arange(len(semantic_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, semantic_df["absolute_gain"], marker="o")
plt.axhline(0.0, linestyle="--")

plt.xlabel("Frame index within selected window")
plt.ylabel("Separability gain: PR - Intensity")
plt.title("Frame-wise semantic gain of pseudo-reflectivity over raw
intensity")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Identifying the strongest and weakest frames for pseudo-reflectivity gain

We now summarize where pseudo-reflectivity helps the most and where it helps the least.

This gives us concrete anchor frames for later interpretation and possible visual inspection.

We will extract:

- the top frames where pseudo-reflectivity gives the largest positive semantic gain,
- and the top frames where pseudo-reflectivity hurts the most relative to raw intensity.

This is useful because it turns the temporal curve into named examples rather than just a trend.

```
top_k = 5

best_gain_frames = semantic_df.nlargest(top_k, "absolute_gain")[
    ["frame_id", "intensity_sep_score", "pr_sep_score",
"absolute_gain", "relative_gain"]
].reset_index(drop=True)

worst_gain_frames = semantic_df.nsmallest(top_k, "absolute_gain")[
    ["frame_id", "intensity_sep_score", "pr_sep_score",
"absolute_gain", "relative_gain"]
].reset_index(drop=True)

print("Top 5 frames where pseudo-reflectivity helps the most")
print(best_gain_frames)
print()
print("Top 5 frames where pseudo-reflectivity helps the least / hurts
the most")
print(worst_gain_frames)
```

```
Top 5 frames where pseudo-reflectivity helps the most
  frame_id  intensity_sep_score  pr_sep_score  absolute_gain
relative_gain
0   000000             0.381428      0.577238       0.195809
0.513358
1   000001             0.387196      0.562421       0.175224
0.452547
2   000002             0.401164      0.535149       0.133986
0.333993
3   000003             0.419656      0.520715       0.101058
0.240812
4   000020             0.434072      0.516331       0.082260
0.189507

Top 5 frames where pseudo-reflectivity helps the least / hurts the
most
  frame_id  intensity_sep_score  pr_sep_score  absolute_gain
```

```
relative_gain
0   000013              0.630274      0.502886      -0.127387      -
0.202115
1   000012              0.654662      0.530535      -0.124127      -
0.189604
2   000014              0.622705      0.499203      -0.123502      -
0.198331
3   000011              0.640741      0.520092      -0.120650      -
0.188297
4   000010              0.586710      0.502133      -0.084577      -
0.144154
```

Print a compact final temporal summary

We now collect the main findings of the short-window motion recap into one compact summary block.

This includes:

- average range dependence over the 30-frame window,
- average semantic separability over the 30-frame window,
- mean gain of pseudo-reflectivity over raw intensity,
- and the best and worst gain frames for later reference.

```
best_row = semantic_df.loc[semantic_df["absolute_gain"].idxmax()]
worst_row = semantic_df.loc[semantic_df["absolute_gain"].idxmin()]

print("MOTION RECAP: FINAL TEMPORAL SUMMARY")
print()

print("[Range Dependence Across 30 Frames]")
print(f"mean corr(raw intensity, range)    =
{corr_df['corr_intensity_range'].mean():.6f}")
print(f"mean corr(I * R, range)            =
{corr_df['corr_pr_range'].mean():.6f}")
print()

print("[Semantic Separability Across 30 Frames]")
print(f"mean raw intensity score           =
{semantic_df['intensity_sep_score'].mean():.6f}")
print(f"mean pseudo-reflectivity score     =
{semantic_df['pr_sep_score'].mean():.6f}")
print(f"mean absolute gain                 =
{semantic_df['absolute_gain'].mean():.6f}")
print(f"mean relative gain                 =
{semantic_df['relative_gain'].mean():.6f}")
print()

print("[Best Frame for Pseudo-Reflectivity Gain]")
```

```
print(f"frame_id                          = {best_row['frame_id']}")
print(f"intensity score                   =
{best_row['intensity_sep_score']:.6f}")
print(f"pseudo-reflectivity score         =
{best_row['pr_sep_score']:.6f}")
print(f"absolute gain                     =
{best_row['absolute_gain']:.6f}")
print(f"relative gain                     =
{best_row['relative_gain']:.6f}")
print()

print("[Worst Frame for Pseudo-Reflectivity Gain]")
print(f"frame_id                          = {worst_row['frame_id']}")
print(f"intensity score                   =
{worst_row['intensity_sep_score']:.6f}")
print(f"pseudo-reflectivity score         =
{worst_row['pr_sep_score']:.6f}")
print(f"absolute gain                     =
{worst_row['absolute_gain']:.6f}")
print(f"relative gain                     =
{worst_row['relative_gain']:.6f}")
```

```
MOTION RECAP: FINAL TEMPORAL SUMMARY

[Range Dependence Across 30 Frames]
mean corr(raw intensity, range)    = -0.206539
mean corr(I * R, range)            = 0.364621

[Semantic Separability Across 30 Frames]
mean raw intensity score           = 0.450447
mean pseudo-reflectivity score     = 0.475203
mean absolute gain                 = 0.024756
mean relative gain                 = 0.091623

[Best Frame for Pseudo-Reflectivity Gain]
frame_id                           = 000000
intensity score                    = 0.381428
pseudo-reflectivity score          = 0.577238
absolute gain                      = 0.195809
relative gain                      = 0.513358

[Worst Frame for Pseudo-Reflectivity Gain]
frame_id                           = 000013
intensity score                    = 0.630274
pseudo-reflectivity score          = 0.502886
absolute gain                      = -0.127387
relative gain                      = -0.202115
```

Inspecting which semantic classes exist in this frame

We begin the dedicated single-frame semantic recap.

We already rebuilt the verified frame, labels, range, and pseudo-reflectivity earlier in this notebook. Now we shift into the class-wise semantic view.

In this step, we will:

- list all semantic class IDs present in the frame,
- count how many points belong to each class,
- and identify which classes are large enough to support meaningful class-wise signal analysis.

This gives us the semantic battlefield before we compare raw intensity and pseudo-reflectivity class by class.

To stay faithful to the preliminary semantic notebook, we use a minimum support threshold and keep every class that satisfies it, including class 0 if it is sufficiently populated.

```
unique_classes, class_counts = np.unique(semantic_labels,
return_counts=True)

print("Number of unique semantic classes in frame:",
len(unique_classes))
print()

print("Semantic class counts:")
for cls, cnt in zip(unique_classes, class_counts):
    print(f"Class {cls:>3}: {cnt}")

MIN_POINTS_PER_CLASS = 500

valid_classes_semantic = unique_classes[class_counts >=
MIN_POINTS_PER_CLASS]
valid_counts_semantic = class_counts[class_counts >=
MIN_POINTS_PER_CLASS]

print()
print(f"Classes with at least {MIN_POINTS_PER_CLASS} points:")
for cls, cnt in zip(valid_classes_semantic, valid_counts_semantic):
    print(f"Class {cls:>3}: {cnt}")

print()
print("Valid class IDs for downstream semantic analysis:")
print(valid_classes_semantic.tolist())
```

```
Number of unique semantic classes in frame: 17

Semantic class counts:
Class   0: 2184
```

```
Class   1: 4
Class  10: 4234
Class  40: 34228
Class  44: 3268
Class  48: 26360
Class  50: 18268
Class  51: 370
Class  52: 1471
Class  60: 1053
Class  70: 27123
Class  71: 1192
Class  72: 2964
Class  80: 532
Class  81: 102
Class  99: 1227
Class 255: 88

Classes with at least 500 points:
Class   0: 2184
Class  10: 4234
Class  40: 34228
Class  44: 3268
Class  48: 26360
Class  50: 18268
Class  52: 1471
Class  60: 1053
Class  70: 27123
Class  71: 1192
Class  72: 2964
Class  80: 532
Class  99: 1227

Valid class IDs for downstream semantic analysis:
[0, 10, 40, 44, 48, 50, 52, 60, 70, 71, 72, 80, 99]
```

Building class-wise summary statistics for the core signals

In this code, we move from semantic presence to semantic signal behavior.

For each sufficiently populated class, we compute compact descriptive statistics for:

- raw intensity,
- range,
- pseudo-reflectivity.

This tells us whether different semantic classes occupy different signal regimes before we even start plotting distributions.

The goal is to turn the labeled frame into a small semantic summary table that we can inspect directly and use in the next semantic recap codes.

```python
import pandas as pd
import numpy as np

# Rebuild the dedicated single-frame semantic analysis state
explicitly
FRAME_ID_SEM = "000000"

bin_path_sem = DATASET_ROOT / "velodyne" / f"{FRAME_ID_SEM}.bin"
label_path_sem = DATASET_ROOT / "labels" / f"{FRAME_ID_SEM}.label"

points_sem = np.fromfile(bin_path_sem, dtype=np.float32).reshape(-1,
4)
labels_sem = np.fromfile(label_path_sem, dtype=np.uint32)

xyz_sem = points_sem[:, :3]
intensity_sem = points_sem[:, 3]
ranges_sem = np.linalg.norm(xyz_sem, axis=1)
pseudo_reflectivity_sem = intensity_sem * ranges_sem

semantic_labels_sem = labels_sem & 0xFFFF
instance_labels_sem = labels_sem >> 16

print("Single-frame semantic recap state")
print("FRAME_ID:", FRAME_ID_SEM)
print("points shape            :", points_sem.shape)
print("semantic labels shape   :", semantic_labels_sem.shape)
print("intensity shape         :", intensity_sem.shape)
print("ranges shape            :", ranges_sem.shape)
print("pseudo-reflectivity shape:", pseudo_reflectivity_sem.shape)
print("aligned                 :", len(points_sem) ==
len(semantic_labels_sem))
print()

rows = []

for cls in valid_classes_semantic:
    mask = semantic_labels_sem == cls

    cls_intensity = intensity_sem[mask]
    cls_ranges = ranges_sem[mask]
    cls_pseudo = pseudo_reflectivity_sem[mask]

    rows.append({
        "class_id": int(cls),
        "count": int(mask.sum()),

        "intensity_mean": float(np.mean(cls_intensity)),
        "intensity_std": float(np.std(cls_intensity)),
        "intensity_p25": float(np.percentile(cls_intensity, 25)),
        "intensity_p50": float(np.percentile(cls_intensity, 50)),
        "intensity_p75": float(np.percentile(cls_intensity, 75)),
```

```
        "range_mean": float(np.mean(cls_ranges)),
        "range_std": float(np.std(cls_ranges)),
        "range_p25": float(np.percentile(cls_ranges, 25)),
        "range_p50": float(np.percentile(cls_ranges, 50)),
        "range_p75": float(np.percentile(cls_ranges, 75)),

        "pseudo_mean": float(np.mean(cls_pseudo)),
        "pseudo_std": float(np.std(cls_pseudo)),
        "pseudo_p25": float(np.percentile(cls_pseudo, 25)),
        "pseudo_p50": float(np.percentile(cls_pseudo, 50)),
        "pseudo_p75": float(np.percentile(cls_pseudo, 75)),
    })

class_stats_df = pd.DataFrame(rows).sort_values("count",
ascending=False).reset_index(drop=True)

print("Class-wise summary statistics:")
display(class_stats_df)
```

```
Single-frame semantic recap state
FRAME_ID: 000000
points shape            : (124668, 4)
semantic labels shape   : (124668,)
intensity shape         : (124668,)
ranges shape            : (124668,)
pseudo-reflectivity shape: (124668,)
aligned                 : True

Class-wise summary statistics:

    class_id  count  intensity_mean  intensity_std  intensity_p25  \
0         40  34228        0.213374       0.113441           0.16
1         70  27123        0.333848       0.131191           0.27
2         48  26360        0.300424       0.079737           0.28
3         50  18268        0.428269       0.148804           0.34
4         10   4234        0.153462       0.193044           0.00
5         44   3268        0.299865       0.085356           0.27
6         72   2964        0.311606       0.115178           0.25
7          0   2184        0.047019       0.116439           0.00
8         52   1471        0.344582       0.159827           0.26
9         99   1227        0.279446       0.155952           0.20
10        71   1192        0.305696       0.139994           0.21
11        60   1053        0.408528       0.166554           0.30
12        80    532        0.203421       0.173505           0.00

    intensity_p50  intensity_p75  range_mean  range_std  range_p25
range_p50  \
0            0.25           0.29    9.779534   6.546706   5.605285
7.266638
```

```
1            0.34           0.41   16.486662  10.300393   8.144478
13.146496
2            0.31           0.35    8.518487   5.227359   5.460627
6.657840
3            0.42           0.52   15.399200   6.694814  10.730637
13.061844
4            0.09           0.27   14.442315   9.118223   8.085788
8.730825
5            0.32           0.35   13.468061   7.984953   8.016994
9.541819
6            0.33           0.39   22.927259   9.109071  15.432064
20.181004
7            0.00           0.00   59.040615  11.136915  54.508484
57.576302
8            0.36           0.45   13.619083   6.919503  10.642494
10.746443
9            0.31           0.38   21.013813  12.509447   9.031216
16.166605
10           0.31           0.39   24.887270   9.196290  15.995169
20.621986
11           0.38           0.51    7.093295   2.614435   4.940204
6.063283
12           0.18           0.32   11.477625   8.760163   5.673513
6.861239

    range_p75  pseudo_mean  pseudo_std  pseudo_p25  pseudo_p50
pseudo_p75
0   11.138959     1.895020    1.460192    1.022925    1.712127
2.432171
1   21.098911     4.861657    2.745001    2.819527    4.443832
6.407302
2    9.428081     2.472348    1.448439    1.640431    2.077798
2.861552
3   17.301601     6.293034    2.691933    4.419292    6.030804
8.061449
4   21.078747     2.553735    4.030944    0.000000    0.784169
3.145671
5   18.266766     3.833581    2.288389    2.387962    3.068133
5.064581
6   28.056694     6.411902    2.519396    5.051509    6.313788
7.673467
7   64.999191     2.567785    6.472702    0.000000    0.000000
0.000000
8   10.884012     4.121638    2.271700    3.090988    4.062512
5.139290
9   37.970516     4.727847    3.791297    2.697250    3.680454
6.416998
10  32.449940     6.802622    2.946005    5.217062    6.806355
8.676571
```

```
11   8.620554     2.683908    0.998142    1.885840    2.631746
3.408531
12  13.640254     2.421643    2.884859    0.000000    1.446301
3.247145
```

Quantifying semantic separability across valid classes

In this code, we measure directly whether pseudo-reflectivity separates the valid semantic classes more clearly than raw intensity on this single frame.

We use a Fisher-style multiclass separability score across the valid semantic classes.

This gives one compact number for:

- raw intensity,
- pseudo-reflectivity $I \cdot R$,

and lets us compare the class-wise semantic usefulness numerically before moving on.

```
import numpy as np

def fisher_style_multiclass_score(values, labels, classes):
    overall_mean = np.mean(values)

    between = 0.0
    within = 0.0

    for cls in classes:
        cls_vals = values[labels == cls]
        cls_mean = np.mean(cls_vals)
        cls_var = np.var(cls_vals)
        n_cls = len(cls_vals)

        between += n_cls * (cls_mean - overall_mean) ** 2
        within += n_cls * cls_var

    return between / within if within > 0 else np.nan

intensity_score_sem = fisher_style_multiclass_score(
    intensity_sem, semantic_labels_sem, valid_classes_semantic
)
pseudo_score_sem = fisher_style_multiclass_score(
    pseudo_reflectivity_sem, semantic_labels_sem,
valid_classes_semantic
)

print("Multiclass semantic separability score")
print(f"Raw intensity          : {intensity_score_sem:.6f}")
print(f"Pseudo-reflectivity    : {pseudo_score_sem:.6f}")
print(f"Absolute improvement   : {pseudo_score_sem -
```

```
intensity_score_sem:.6f}")
print(f"Relative improvement   : {(pseudo_score_sem -
intensity_score_sem) / intensity_score_sem:.6f}")
```

```
Multiclass semantic separability score
Raw intensity          : 0.448162
Pseudo-reflectivity    : 0.505541
Absolute improvement   : 0.057379
Relative improvement   : 0.128031
```

Visualizing the frame spatially with semantics, intensity, and pseudo-reflectivity

In this code, we return from summary statistics to the actual scene geometry.

We visualize the same single frame in bird's-eye view using:

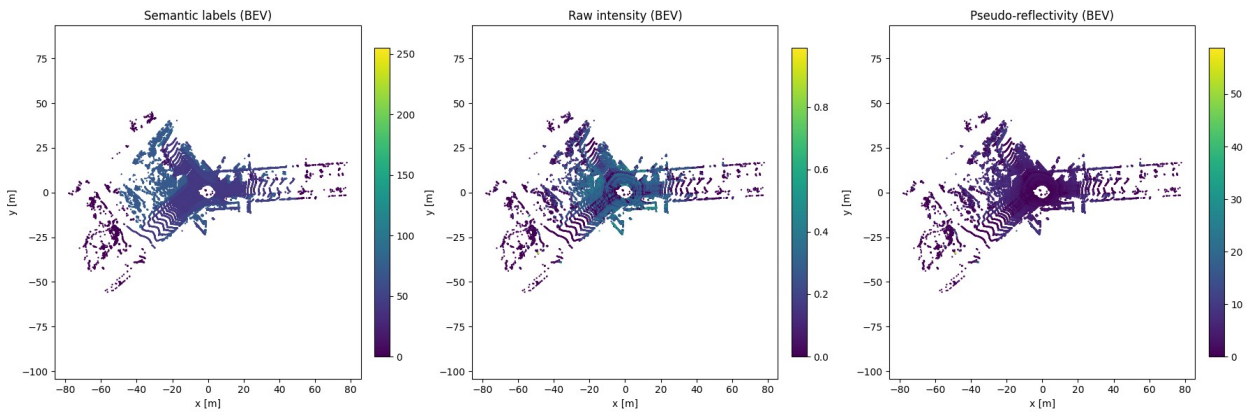
- semantic labels,
- raw intensity,
- pseudo-reflectivity $I \cdot R$.

This helps connect the class-wise signal behavior back to where those patterns live in the point cloud.

The goal here is to check whether the semantic structure and the signal structure are aligned spatially, and whether pseudo-reflectivity changes the visual emphasis of the scene in a meaningful way.

```
import matplotlib.pyplot as plt

x_sem = xyz_sem[:, 0]
y_sem = xyz_sem[:, 1]

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# 1) Semantic labels
sc0 = axes[0].scatter(x_sem, y_sem, c=semantic_labels_sem, s=0.5)
axes[0].set_title("Semantic labels (BEV)")
axes[0].set_xlabel("x [m]")
axes[0].set_ylabel("y [m]")
axes[0].axis("equal")
plt.colorbar(sc0, ax=axes[0], fraction=0.046, pad=0.04)

# 2) Raw intensity
sc1 = axes[1].scatter(x_sem, y_sem, c=intensity_sem, s=0.5)
axes[1].set_title("Raw intensity (BEV)")
axes[1].set_xlabel("x [m]")
axes[1].set_ylabel("y [m]")
axes[1].axis("equal")
```

```
plt.colorbar(sc1, ax=axes[1], fraction=0.046, pad=0.04)

# 3) Pseudo-reflectivity
sc2 = axes[2].scatter(x_sem, y_sem, c=pseudo_reflectivity_sem, s=0.5)
axes[2].set_title("Pseudo-reflectivity (BEV)")
axes[2].set_xlabel("x [m]")
axes[2].set_ylabel("y [m]")
axes[2].axis("equal")
plt.colorbar(sc2, ax=axes[2], fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()
```



Comparing class mean values directly

In this part, we make the class differences easier to read at a glance.

We plot the mean value of each valid semantic class for:

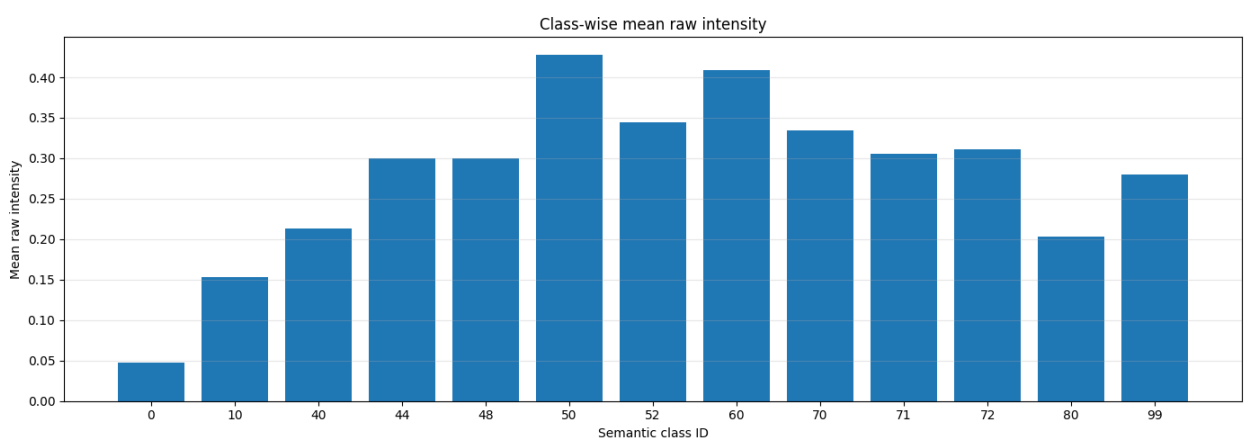
- raw intensity,
- range,
- pseudo-reflectivity $I \cdot R$.

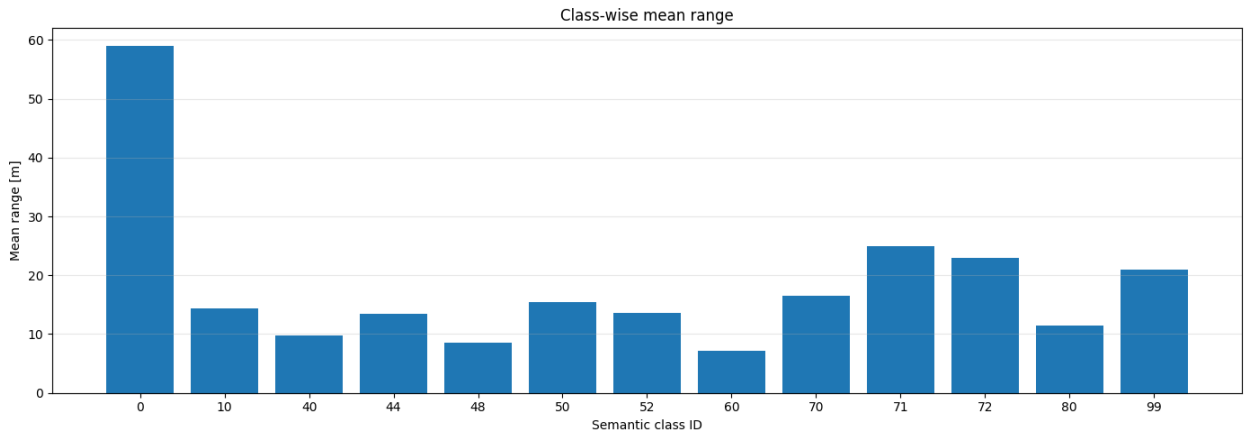
This gives a compact class-level comparison of where each signal places each semantic group.

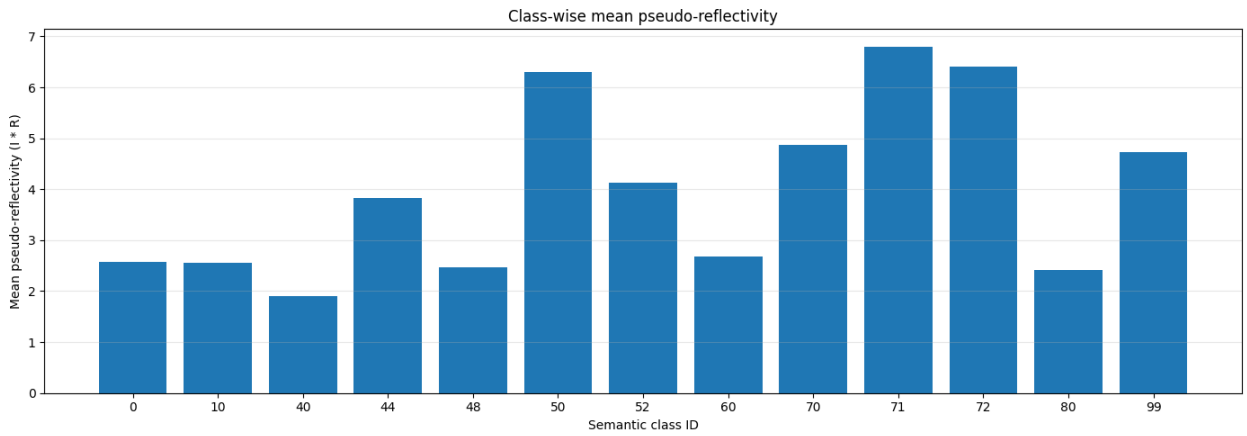
The goal is to see whether raw intensity and pseudo-reflectivity rank the classes in the same way, or whether the range-aware proxy changes the semantic ordering.

```
import matplotlib.pyplot as plt
import numpy as np

plot_df =
class_stats_df.sort_values("class_id").reset_index(drop=True)

class_ids = plot_df["class_id"].astype(str).tolist()
xpos = np.arange(len(class_ids))
```

```
plt.figure(figsize=(14, 5))
plt.bar(xpos, plot_df["intensity_mean"])
plt.xticks(xpos, class_ids)
plt.xlabel("Semantic class ID")
plt.ylabel("Mean raw intensity")
plt.title("Class-wise mean raw intensity")
plt.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 5))
plt.bar(xpos, plot_df["range_mean"])
plt.xticks(xpos, class_ids)
plt.xlabel("Semantic class ID")
plt.ylabel("Mean range [m]")
plt.title("Class-wise mean range")
plt.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 5))
plt.bar(xpos, plot_df["pseudo_mean"])
plt.xticks(xpos, class_ids)
plt.xlabel("Semantic class ID")
plt.ylabel("Mean pseudo-reflectivity (I * R)")
plt.title("Class-wise mean pseudo-reflectivity")
plt.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```







Final takeaway of the single-frame semantic recap

By this point, we have:

- inspected the semantic composition of the frame,
- built class-wise summary statistics,
- measured multiclass semantic separability,
- visualized the frame spatially with semantics, intensity, and pseudo-reflectivity,
- and compared class-mean behavior directly.

The goal now is to lock the main semantic takeaway in one compact summary before we move to the final recap section: short-window semantic consistency through motion.

```
print("SINGLE-FRAME SEMANTIC RECAP: FINAL SUMMARY\n")

print("[Semantic Frame Composition]")
print(f"number of unique semantic classes     =
{len(unique_classes)}")
print(f"valid classes used for analysis       =
{len(valid_classes_semantic)}")
print(f"valid class IDs                       =
{valid_classes_semantic.tolist()}")
```

```
print()

print("[Multiclass Semantic Separability]")
print(f"raw intensity score                    =
{intensity_score_sem:.6f}")
print(f"pseudo-reflectivity score              =
{pseudo_score_sem:.6f}")
print(f"absolute improvement                   = {pseudo_score_sem -
intensity_score_sem:.6f}")
print(f"relative improvement                   = {(pseudo_score_sem -
intensity_score_sem) / intensity_score_sem:.6f}")
print()

top_intensity = class_stats_df.sort_values("intensity_mean",
ascending=False).head(3)[["class_id", "intensity_mean"]]
top_pseudo = class_stats_df.sort_values("pseudo_mean",
ascending=False).head(3)[["class_id", "pseudo_mean"]]
top_range = class_stats_df.sort_values("range_mean",
ascending=False).head(3)[["class_id", "range_mean"]]

print("[Top 3 Classes by Mean Raw Intensity]")
for _, row in top_intensity.iterrows():
    print(f"class {int(row['class_id']):>3} :
{row['intensity_mean']:.6f}")
print()

print("[Top 3 Classes by Mean Pseudo-Reflectivity]")
for _, row in top_pseudo.iterrows():
    print(f"class {int(row['class_id']):>3} :
{row['pseudo_mean']:.6f}")
print()

print("[Top 3 Classes by Mean Range]")
for _, row in top_range.iterrows():
    print(f"class {int(row['class_id']):>3} :
{row['range_mean']:.6f}")
print()
```

```
SINGLE-FRAME SEMANTIC RECAP: FINAL SUMMARY

[Semantic Frame Composition]
number of unique semantic classes     = 17
valid classes used for analysis       = 13
valid class IDs                       = [0, 10, 40, 44, 48, 50, 52,
60, 70, 71, 72, 80, 99]

[Multiclass Semantic Separability]
raw intensity score                   = 0.448162
pseudo-reflectivity score             = 0.505541
absolute improvement                  = 0.057379
```

```
relative improvement                  = 0.128031

[Top 3 Classes by Mean Raw Intensity]
class  50 : 0.428269
class  60 : 0.408528
class  52 : 0.344582

[Top 3 Classes by Mean Pseudo-Reflectivity]
class  71 : 6.802622
class  72 : 6.411902
class  50 : 6.293034

[Top 3 Classes by Mean Range]
class   0 : 59.040615
class  71 : 24.887270
class  72 : 22.927259
```

Setup for multi-frame semantic consistency

In this part, we begin the final recap section of this notebook: the short-window semantic consistency analysis through motion.

The earlier semantic recap showed that pseudo-reflectivity can reshape class-wise behavior on one frame in a more interpretable way than raw intensity alone.

Now we ask the next question:

do those class-wise patterns persist across a short motion window, or were they only a single-frame accident?

To keep this recap faithful to the preliminary Notebook 05 structure, we start with a small, disciplined 10-frame consecutive window.

In this combo, we only:

- define the sequence paths,
- choose the 10-frame semantic-consistency window,
- and print the selected frame IDs.

No class-wise temporal statistics yet. This is just the setup handshake for the final semantic recap stage.

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Optional: cleaner plots / display
plt.rcParams["figure.figsize"] = (8, 4)
```

```
plt.rcParams["figure.dpi"] = 120
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 4)

# Semantic consistency recap paths
SEQ_DIR_05 = DATASET_ROOT
VELODYNE_DIR_05 = SEQ_DIR_05 / "velodyne"
LABEL_DIR_05 = SEQ_DIR_05 / "labels"

# Small, disciplined preliminary window
START_FRAME_05 = 0
NUM_FRAMES_05 = 10

frame_ids_05 = [f"{i:06d}" for i in range(START_FRAME_05,
START_FRAME_05 + NUM_FRAMES_05)]

print("Sequence directory :", SEQ_DIR_05.resolve())
print("Velodyne dir exists:", VELODYNE_DIR_05.exists())
print("Label dir exists   :", LABEL_DIR_05.exists())
print("Selected frame IDs :", frame_ids_05)
```

```
Sequence directory :
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti/dataset
/sequences/00
Velodyne dir exists: True
Label dir exists   : True
Selected frame IDs : ['000000', '000001', '000002', '000003',
'000004', '000005', '000006', '000007', '000008', '000009']
```

Define data-loading utilities for the semantic-consistency window

In this combo, we define the small helper functions needed for the short-window semantic consistency recap.

For each frame, we need to be able to:

- load LiDAR points,
- load semantic labels,
- compute range,
- and compute pseudo-reflectivity.

This keeps the 10-frame semantic-consistency analysis clean and self-contained.

We are still not building the temporal class table yet. This combo only prepares the utilities so the next combo can load the selected frames reliably and verify structural consistency.

```
def load_frame_points_05(frame_id, velodyne_dir=VELODYNE_DIR_05):
    bin_path = velodyne_dir / f"{frame_id}.bin"
```

```python
    points = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
    xyz = points[:, :3]
    intensity = points[:, 3]
    return xyz, intensity

def load_frame_labels_05(frame_id, label_dir=LABEL_DIR_05):
    label_path = label_dir / f"{frame_id}.label"
    raw_labels = np.fromfile(label_path, dtype=np.uint32)
    semantic_labels = raw_labels & 0xFFFF
    return semantic_labels

def compute_range_and_pseudo_reflectivity_05(xyz, intensity):
    ranges = np.linalg.norm(xyz, axis=1)
    pseudo_reflectivity = intensity * ranges
    return ranges, pseudo_reflectivity

# Quick sanity check on the first frame
test_frame_05 = frame_ids_05[0]
xyz_05, intensity_05 = load_frame_points_05(test_frame_05)
semantic_labels_05 = load_frame_labels_05(test_frame_05)
ranges_05, pseudo_reflectivity_05 =
compute_range_and_pseudo_reflectivity_05(xyz_05, intensity_05)

print("Test frame             :", test_frame_05)
print("XYZ shape              :", xyz_05.shape)
print("Intensity shape        :", intensity_05.shape)
print("Semantic labels shape  :", semantic_labels_05.shape)
print("Ranges shape           :", ranges_05.shape)
print("Pseudo-reflectivity    :", pseudo_reflectivity_05.shape)

print("\nConsistency check")
print(
    "Same number of points across arrays:",
    len(xyz_05) == len(intensity_05) == len(semantic_labels_05) ==
len(ranges_05) == len(pseudo_reflectivity_05)
)
```

```
Test frame             : 000000
XYZ shape              : (124668, 3)
Intensity shape        : (124668,)
Semantic labels shape  : (124668,)
Ranges shape           : (124668,)
Pseudo-reflectivity    : (124668,)

Consistency check
Same number of points across arrays: True
```

Build a compact per-frame summary table

In this code, we load all selected frames in the 10-frame semantic-consistency window and construct a small frame-level summary table.

For each frame, we record:

- number of points,
- number of semantic classes,
- basic intensity statistics,
- basic range statistics,
- basic pseudo-reflectivity statistics.

This is still a structural and numerical check before class-wise temporal analysis.

The goal is to confirm that the selected semantic-consistency window is well-behaved at the frame level before we build the class-wise temporal table.

```
frame_summaries_05 = []

for frame_id in frame_ids_05:
    xyz_05, intensity_05 = load_frame_points_05(frame_id)
    semantic_labels_05 = load_frame_labels_05(frame_id)
    ranges_05, pseudo_reflectivity_05 =
compute_range_and_pseudo_reflectivity_05(xyz_05, intensity_05)

    frame_summaries_05.append({
        "frame_id": frame_id,
        "num_points": len(intensity_05),
        "num_semantic_classes": len(np.unique(semantic_labels_05)),
        "intensity_mean": float(np.mean(intensity_05)),
        "intensity_std": float(np.std(intensity_05)),
        "range_mean": float(np.mean(ranges_05)),
        "range_std": float(np.std(ranges_05)),
        "pseudo_reflectivity_mean":
float(np.mean(pseudo_reflectivity_05)),
        "pseudo_reflectivity_std":
float(np.std(pseudo_reflectivity_05)),
    })

frame_summary_df_05 = pd.DataFrame(frame_summaries_05)

frame_summary_df_05
```

```
  frame_id  num_points  num_semantic_classes  intensity_mean
intensity_std  \
0   000000      124668                    17          0.2941
0.1477
1   000001      124605                    17          0.2946
0.1480
```

```
2   000002      124478                      17          0.2942
0.1481
3   000003      124167                      17          0.2957
0.1496
4   000004      123969                      17          0.2956
0.1512
5   000005      123924                      17          0.2947
0.1532
6   000006      123373                      18          0.2949
0.1562
7   000007      122765                      18          0.2965
0.1599
8   000008      122123                      19          0.2987
0.1611
9   000009      121357                      18          0.3008
0.1616

   range_mean  range_std  pseudo_reflectivity_mean
pseudo_reflectivity_std
0     13.5650    10.5710                    3.6287
2.9401
1     13.5114    10.6273                    3.6067
2.9450
2     13.4012    10.5917                    3.5787
2.9201
3     13.2681    10.5035                    3.5739
2.9210
4     13.1882    10.5035                    3.5544
2.9104
5     13.1344    10.4834                    3.5482
2.9212
6     13.0981    10.5063                    3.5472
2.9204
7     13.0900    10.5474                    3.5697
2.9512
8     13.0589    10.5368                    3.5859
2.9630
9     13.0993    10.6321                    3.6085
3.0023
```

Building class-wise statistics for every frame

In this code, we compute class-wise summary statistics for each frame in the 10-frame semantic-consistency window.

We keep only classes with enough points to be meaningful.

For each retained class in each frame, we record:

- point count,

- mean and standard deviation of raw intensity,
- mean and standard deviation of pseudo-reflectivity,
- mean and standard deviation of range.

This creates the temporal semantic table that the final recap section is really about.

Once this table exists, we can ask the key question: do the class-wise patterns from the single-frame semantic recap persist across motion?

```
MIN_POINTS_PER_CLASS_05 = 200

class_rows_05 = []

for frame_id in frame_ids_05:
    xyz_05, intensity_05 = load_frame_points_05(frame_id)
    semantic_labels_05 = load_frame_labels_05(frame_id)
    ranges_05, pseudo_reflectivity_05 =
compute_range_and_pseudo_reflectivity_05(xyz_05, intensity_05)

    unique_classes_05, counts_05 = np.unique(semantic_labels_05,
return_counts=True)

    for class_id, count in zip(unique_classes_05, counts_05):
        if count < MIN_POINTS_PER_CLASS_05:
            continue

        mask = semantic_labels_05 == class_id

        class_rows_05.append({
            "frame_id": frame_id,
            "class_id": int(class_id),
            "num_points": int(count),
            "intensity_mean": float(np.mean(intensity_05[mask])),
            "intensity_std": float(np.std(intensity_05[mask])),
            "pseudo_reflectivity_mean":
float(np.mean(pseudo_reflectivity_05[mask])),
            "pseudo_reflectivity_std":
float(np.std(pseudo_reflectivity_05[mask])),
            "range_mean": float(np.mean(ranges_05[mask])),
            "range_std": float(np.std(ranges_05[mask])),
        })

class_summary_df_05 = (
    pd.DataFrame(class_rows_05)
    .sort_values(by=["frame_id", "class_id"])
    .reset_index(drop=True)
)

print("MIN_POINTS_PER_CLASS:", MIN_POINTS_PER_CLASS_05)
print("Number of retained class-frame rows:",
```

```
len(class_summary_df_05))
print("Unique retained classes:",
sorted(class_summary_df_05["class_id"].unique().tolist()))
print()

class_summary_df_05.head(20)
```

```
MIN_POINTS_PER_CLASS: 200
Number of retained class-frame rows: 140
Unique retained classes: [0, 10, 40, 44, 48, 50, 51, 52, 60, 70, 71,
72, 80, 99]


   frame_id  class_id  num_points  intensity_mean  intensity_std  \
0    000000         0        2184          0.0470         0.1164
1    000000        10        4234          0.1535         0.1930
2    000000        40       34228          0.2134         0.1134
3    000000        44        3268          0.2999         0.0854
4    000000        48       26360          0.3004         0.0797
5    000000        50       18268          0.4283         0.1488
6    000000        51         370          0.2885         0.1211
7    000000        52        1471          0.3446         0.1598
8    000000        60        1053          0.4085         0.1666
9    000000        70       27123          0.3338         0.1312
10   000000        71        1192          0.3057         0.1400
11   000000        72        2964          0.3116         0.1152
12   000000        80         532          0.2034         0.1735
13   000000        99        1227          0.2794         0.1560
14   000001         0        2170          0.0491         0.1302
15   000001        10        4387          0.1530         0.1911
16   000001        40       33233          0.2125         0.1134
17   000001        44        3831          0.2994         0.0830
18   000001        48       26040          0.3009         0.0799
19   000001        50       19180          0.4281         0.1520

    pseudo_reflectivity_mean  pseudo_reflectivity_std  range_mean
range_std
0                     2.5678                   6.4727     59.0406
11.1369
1                     2.5537                   4.0309     14.4423
9.1182
2                     1.8950                   1.4602      9.7795
6.5467
3                     3.8336                   2.2884     13.4681
7.9850
4                     2.4723                   1.4484      8.5185
5.2274
5                     6.2930                   2.6919     15.3992
6.6948
6                     6.0659                   2.4863     21.3816
```

```
2.1585
7                    4.1216                    2.2717     13.6191
6.9195
8                    2.6839                    0.9981      7.0933
2.6144
9                    4.8617                    2.7450     16.4867
10.3004
10                   6.8026                    2.9460     24.8873
9.1963
11                   6.4119                    2.5194     22.9273
9.1091
12                   2.4216                    2.8849     11.4776
8.7602
13                   4.7278                    3.7913     21.0138
12.5094
14                   2.7169                    7.0997     59.5173
11.1551
15                   2.4465                    3.7950     14.1880
9.4371
16                   1.9004                    1.4638      9.9190
6.7176
17                   3.5215                    2.2271     12.3583
7.7030
18                   2.4490                    1.4421      8.4375
5.2630
19                   6.2019                    2.6865     15.1860
6.6215
```

Keeping only temporally persistent classes

In this part, we keep only those semantic classes that survive across all selected frames.

This gives us a clean basis for short-window semantic consistency analysis.

The reason for doing this is simple:

if a class appears only in some frames, then its temporal behavior is harder to compare fairly across the full window.

So before measuring class-wise stability or class-order consistency, we first restrict the analysis to classes that are present in all 10 selected frames.

```
num_selected_frames_05 = len(frame_ids_05)

class_frame_counts_05 = (
    class_summary_df_05.groupby("class_id")["frame_id"]
    .nunique()
    .reset_index(name="num_frames_present")
)
```

```
persistent_classes_05 = class_frame_counts_05.loc[
    class_frame_counts_05["num_frames_present"] ==
num_selected_frames_05,
    "class_id"
].tolist()

persistent_summary_df_05 = class_summary_df_05[
    class_summary_df_05["class_id"].isin(persistent_classes_05)
].copy().reset_index(drop=True)

print("Selected frames:", num_selected_frames_05)
print("Persistent classes:", persistent_classes_05)
print("Number of persistent classes:", len(persistent_classes_05))
print("Persistent summary shape:", persistent_summary_df_05.shape)
print()

class_frame_counts_05.sort_values(
    ["num_frames_present", "class_id"],
    ascending=[False, True]
)
```

```
Selected frames: 10
Persistent classes: [0, 10, 40, 44, 48, 50, 51, 52, 60, 70, 71, 72,
80, 99]
Number of persistent classes: 14
Persistent summary shape: (140, 9)
```

	class_id	num_frames_present
0	0	10
1	10	10
2	40	10
3	44	10
4	48	10
5	50	10
6	51	10
7	52	10
8	60	10
9	70	10
10	71	10
11	72	10
12	80	10
13	99	10

Measuring class-wise temporal stability

In this part, we quantify how much each persistent class mean changes across the selected 10-frame motion window for both:

- raw intensity,

- pseudo-reflectivity.

This gives a first direct measure of short-horizon semantic consistency.

For each persistent class, we compute:

- mean value over time,
- standard deviation over time,
- coefficient of variation over time.

The goal is not to prove invariance. It is to check whether the class-wise signal behavior remains reasonably stable across motion.

```
stability_rows_05 = []

for class_id in persistent_classes_05:
    cls_df = persistent_summary_df_05[
        persistent_summary_df_05["class_id"] == class_id
    ].sort_values("frame_id")

    intensity_means = cls_df["intensity_mean"].values
    pseudo_means = cls_df["pseudo_reflectivity_mean"].values

    stability_rows_05.append({
        "class_id": int(class_id),
        "num_frames": len(cls_df),
        "intensity_mean_over_time": float(np.mean(intensity_means)),
        "intensity_std_over_time": float(np.std(intensity_means)),
        "intensity_cv_over_time": float(np.std(intensity_means) /
(np.mean(intensity_means) + 1e-12)),
        "pseudo_mean_over_time": float(np.mean(pseudo_means)),
        "pseudo_std_over_time": float(np.std(pseudo_means)),
        "pseudo_cv_over_time": float(np.std(pseudo_means) /
(np.mean(pseudo_means) + 1e-12)),
    })

stability_df_05 = (
    pd.DataFrame(stability_rows_05)
    .sort_values("class_id")
    .reset_index(drop=True)
)

stability_df_05
```

```
    class_id  num_frames  intensity_mean_over_time
intensity_std_over_time  \
0          0          10                    0.0468
0.0030
1         10          10                    0.1290
0.0163
2         40          10                    0.2118
```

```
0.0013
3         44          10                    0.3024
0.0021
4         48          10                    0.3002
0.0009
5         50          10                    0.4324
0.0067
6         51          10                    0.3187
0.0166
7         52          10                    0.3674
0.0209
8         60          10                    0.3963
0.0197
9         70          10                    0.3245
0.0048
10        71          10                    0.3125
0.0039
11        72          10                    0.2964
0.0134
12        80          10                    0.1798
0.0138
13        99          10                    0.3015
0.0255

    intensity_cv_over_time  pseudo_mean_over_time
pseudo_std_over_time  \
0                   0.0631                 2.6052
0.1384
1                   0.1261                 1.8978
0.4232
2                   0.0060                 1.9459
0.0343
3                   0.0070                 2.9334
0.4198
4                   0.0030                 2.5255
0.0974
5                   0.0155                 6.0199
0.1353
6                   0.0522                 5.8411
0.2264
7                   0.0569                 5.2178
0.6119
8                   0.0496                 3.4241
0.7067
9                   0.0147                 4.4751
0.1926
10                  0.0125                 5.9944
0.6117
11                  0.0451                 6.8468
```

```
0.2338
12                  0.0766                  1.8447
0.2957
13                  0.0846                  5.5547
0.9460

    pseudo_cv_over_time
0                0.0531
1                0.2230
2                0.0176
3                0.1431
4                0.0386
5                0.0225
6                0.0388
7                0.1173
8                0.2064
9                0.0430
10               0.1020
11               0.0341
12               0.1603
13               0.1703
```

Test whether class ordering persists across frames

In this code, we test whether the class-wise ordering remains stable across the 10-frame motion window.

Instead of asking whether every class mean stays numerically identical, we ask a more robust question:

does the relative ordering of classes remain similar from frame to frame?

To measure this, we:

- build a frame-by-class table for raw intensity means,
- build a frame-by-class table for pseudo-reflectivity means,
- compute a reference class profile by averaging across the full window,
- and then compute the Spearman rank correlation between each frame and that reference profile.

This gives a compact measure of short-window semantic consistency through motion.

```
from scipy.stats import spearmanr

# Pivot tables: rows = frame_id, columns = class_id
intensity_pivot_05 = persistent_summary_df_05.pivot(
    index="frame_id", columns="class_id", values="intensity_mean"
).sort_index()

pseudo_pivot_05 = persistent_summary_df_05.pivot(
```

```
    index="frame_id", columns="class_id",
values="pseudo_reflectivity_mean"
).sort_index()

# Reference profile = average class means across the full frame window
intensity_reference_05 = intensity_pivot_05.mean(axis=0)
pseudo_reference_05 = pseudo_pivot_05.mean(axis=0)

ordering_rows_05 = []

for frame_id in intensity_pivot_05.index:
    intensity_corr, _ = spearmanr(intensity_pivot_05.loc[frame_id],
intensity_reference_05)
    pseudo_corr, _ = spearmanr(pseudo_pivot_05.loc[frame_id],
pseudo_reference_05)

    ordering_rows_05.append({
        "frame_id": frame_id,
        "intensity_rank_corr_vs_mean_profile": float(intensity_corr),
        "pseudo_rank_corr_vs_mean_profile": float(pseudo_corr),
    })

ordering_df_05 = pd.DataFrame(ordering_rows_05)

print("Class-ordering consistency across frames")
display(ordering_df_05)

print("\nAverage rank correlation")
print("Intensity           :",
round(ordering_df_05["intensity_rank_corr_vs_mean_profile"].mean(),
4))
print("Pseudo-reflectivity :",
round(ordering_df_05["pseudo_rank_corr_vs_mean_profile"].mean(), 4))
```

```
Class-ordering consistency across frames

  frame_id  intensity_rank_corr_vs_mean_profile  \
0   000000                               0.8901
1   000001                               0.9165
2   000002                               0.9297
3   000003                               0.9648
4   000004                               0.9824
5   000005                               0.9868
6   000006                               0.9912
7   000007                               0.9648
8   000008                               0.9516
9   000009                               0.9560

   pseudo_rank_corr_vs_mean_profile
0                            0.9473
```

```
1                              0.9560
2                              0.9736
3                              0.9868
4                              0.9868
5                              0.9956
6                              0.9473
7                              0.9385
8                              0.9253
9                              0.9253


Average rank correlation
Intensity           : 0.9534
Pseudo-reflectivity : 0.9582
```

Visualizing ordering consistency across motion

In this code, we plot the frame-wise rank-correlation values for:

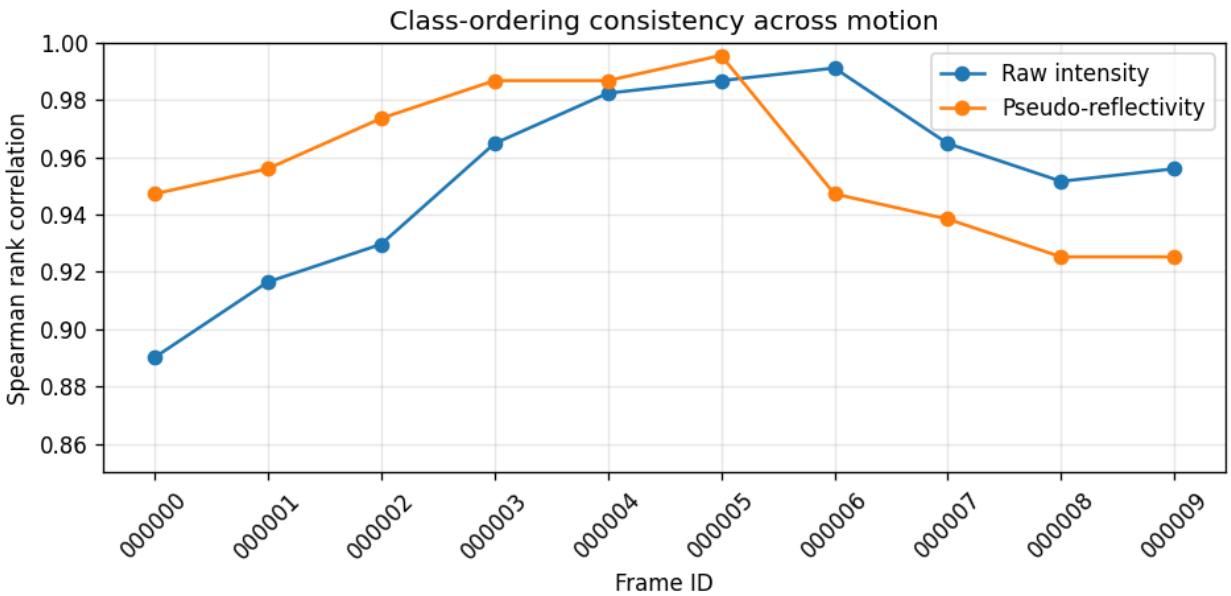
- raw intensity,
- pseudo-reflectivity.

This gives a compact visual check of how stable the semantic class ordering remains as the platform moves.

The goal is to make the consistency result easier to read across the 10-frame window and to see whether the pseudo-reflectivity advantage is steady, weak, or erratic.

```
plt.figure(figsize=(8, 4))

plt.plot(
    ordering_df_05["frame_id"],
    ordering_df_05["intensity_rank_corr_vs_mean_profile"],
    marker="o",
    label="Raw intensity"
)

plt.plot(
    ordering_df_05["frame_id"],
    ordering_df_05["pseudo_rank_corr_vs_mean_profile"],
    marker="o",
    label="Pseudo-reflectivity"
)

plt.ylim(0.85, 1.00)
plt.xlabel("Frame ID")
plt.ylabel("Spearman rank correlation")
plt.title("Class-ordering consistency across motion")
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
```

```
plt.legend()
plt.tight_layout()
plt.show()
```



Summarizing the semantic-consistency result numerically

This gives us a neat final quantitative statement for the short-window semantic-consistency recap without bloating the analysis.

The goal is to summarize:

- average rank consistency,
- minimum and maximum consistency,
- how often pseudo-reflectivity is better,
- how often raw intensity is better,
- and the mean advantage of pseudo-reflectivity over raw intensity.

```
intensity_vals_05 =
ordering_df_05["intensity_rank_corr_vs_mean_profile"].values
pseudo_vals_05 =
ordering_df_05["pseudo_rank_corr_vs_mean_profile"].values

summary_05 = {
    "num_frames": len(ordering_df_05),
    "intensity_mean_rank_corr": float(np.mean(intensity_vals_05)),
    "pseudo_mean_rank_corr": float(np.mean(pseudo_vals_05)),
    "intensity_min_rank_corr": float(np.min(intensity_vals_05)),
    "pseudo_min_rank_corr": float(np.min(pseudo_vals_05)),
    "intensity_max_rank_corr": float(np.max(intensity_vals_05)),
    "pseudo_max_rank_corr": float(np.max(pseudo_vals_05)),
```

```
    "pseudo_better_frames": int(np.sum(pseudo_vals_05 >
intensity_vals_05)),
    "intensity_better_frames": int(np.sum(intensity_vals_05 >
pseudo_vals_05)),
    "tied_frames": int(np.sum(np.isclose(pseudo_vals_05,
intensity_vals_05))),
    "mean_advantage_pseudo_minus_intensity":
float(np.mean(pseudo_vals_05 - intensity_vals_05)),
}

for k, v in summary_05.items():
    if isinstance(v, float):
        print(f"{k}: {v:.4f}")
    else:
        print(f"{k}: {v}")
```

```
num_frames: 10
intensity_mean_rank_corr: 0.9534
pseudo_mean_rank_corr: 0.9582
intensity_min_rank_corr: 0.8901
pseudo_min_rank_corr: 0.9253
intensity_max_rank_corr: 0.9912
pseudo_max_rank_corr: 0.9956
pseudo_better_frames: 6
intensity_better_frames: 4
tied_frames: 0
mean_advantage_pseudo_minus_intensity: 0.0048
```

Final takeaway of the short-window semantic-consistency recap

At this point, we have:

- built the 10-frame semantic-consistency window,
- computed class-wise temporal summary statistics,
- restricted the analysis to persistent classes,
- measured class-wise temporal stability,
- measured class-order consistency across motion,
- and summarized the relative advantage of pseudo-reflectivity.

The goal now is to lock the correct scale of conclusion:

pseudo-reflectivity does not become true calibrated reflectivity, but it preserves short-window class-wise semantic structure at least as well as raw intensity, with a mild average advantage in this window.

This gives the final recap notebook its proper endpoint: the baseline idea survives contact with motion, semantics, and temporal consistency strongly enough to justify the actual final-stage extensions that follow in Notebook 2 and beyond.

```python
print("SHORT-WINDOW SEMANTIC CONSISTENCY RECAP: FINAL SUMMARY\n")

print("[Window Setup]")
print(f"number of frames                    =
{summary_05['num_frames']}")
print(f"persistent classes                  =
{persistent_classes_05}")
print(f"number of persistent classes        =
{len(persistent_classes_05)}")
print()

print("[Class-Ordering Consistency]")
print(f"mean raw intensity rank correlation  =
{summary_05['intensity_mean_rank_corr']:.4f}")
print(f"mean pseudo rank correlation        =
{summary_05['pseudo_mean_rank_corr']:.4f}")
print(f"minimum raw intensity rank corr     =
{summary_05['intensity_min_rank_corr']:.4f}")
print(f"minimum pseudo rank corr            =
{summary_05['pseudo_min_rank_corr']:.4f}")
print(f"maximum raw intensity rank corr     =
{summary_05['intensity_max_rank_corr']:.4f}")
print(f"maximum pseudo rank corr            =
{summary_05['pseudo_max_rank_corr']:.4f}")
print()

print("[Frame-Wise Comparison]")
print(f"pseudo better frames                =
{summary_05['pseudo_better_frames']}")
print(f"intensity better frames             =
{summary_05['intensity_better_frames']}")
print(f"tied frames                         =
{summary_05['tied_frames']}")
print(f"mean pseudo-minus-intensity advantage =
{summary_05['mean_advantage_pseudo_minus_intensity']:.4f}")
print()
```

```
SHORT-WINDOW SEMANTIC CONSISTENCY RECAP: FINAL SUMMARY

[Window Setup]
number of frames                    = 10
persistent classes                  = [0, 10, 40, 44, 48, 50, 51, 52,
60, 70, 71, 72, 80, 99]
number of persistent classes        = 14

[Class-Ordering Consistency]
mean raw intensity rank correlation  = 0.9534
mean pseudo rank correlation        = 0.9582
minimum raw intensity rank corr     = 0.8901
minimum pseudo rank corr            = 0.9253
```

```
maximum raw intensity rank corr      = 0.9912
maximum pseudo rank corr             = 0.9956

[Frame-Wise Comparison]
pseudo better frames                 = 6
intensity better frames              = 4
tied frames                          = 0
mean pseudo-minus-intensity advantage = 0.0048
```

Final Conclusion of Notebook 01

This notebook has successfully re-established the preliminary project baseline in a compact and trustworthy form.

Across this recap, we verified that:

- the SemanticKITTI dataset structure is intact and usable,
- the frame-loading and label-loading pipeline still works correctly,
- the raw LiDAR signals and semantic labels remain aligned and numerically sensible,
- the single-frame reflectivity-style analysis reproduces the key preliminary behavior,
- the short-window motion analysis reproduces the main temporal trends,
- the single-frame semantic analysis reproduces the class-wise semantic effect,
- and the short-window semantic-consistency analysis confirms that the main semantic story survives motion with a mild average advantage for the pseudo-reflectivity proxy.

This is the correct stopping point for the recap notebook.

The goal here was not to regenerate every heavy preliminary artifact again. Those richer visual outputs, including the earlier GIF-based artifacts, already belong to the frozen midterm archive in the preliminary notebook folder and remain part of the preserved project record.

So the meaning of this notebook is clear:

the dataset is intact, the original preliminary pipeline logic still works, the core baseline results have been reproduced in a compact final-pipeline form, and the project is ready to move forward from the end of the preliminary stage.

In other words, this notebook completes the baseline lock.

The midterm work has been recovered cleanly enough for continuation. The next notebook can now begin the actual final-stage extension work.